

INTEGRATION OF GENERATIVE AI INTO INDUSTRIAL MACHINES FOR INDUSTRY 5.0 ENVIRONMENTS

Application to the HT2000 Mobile Hydraulic Test Bench for maintenance of aircraft

Author

Francisco Sierra González

Department of Industrial Design Engineering

Abstract

The present work concerns the modification using a Generative Artificial Intelligence system of a hydraulic test machine that is normally used in the Final Assembly Lines (FAL) of aircraft.

The aim is to transform a conventional industrial operation (hydraulic system testing of aircraft) by employing a “living” autonomous system capable of operating under the paradigms of Industry 5.0, prioritizing human-machine collaboration, safety, and real-time decision-making based on the physical environment.

The project’s idea is based on the urgent need to shift the current industrial model toward a more human-oriented one, attracting new generations of less-skilled operators who can work safely and mitigate the mental load and technological stress experienced by today’s workers.

This new model becomes attainable thanks to the recent emergence of Generative AI, which offers an unprecedented technical opportunity to implement natural-language interfaces and autonomous decision systems that were previously inconceivable.

For the project development, various autonomous-machine options are examined, and an anthropomorphic model is chosen. In addition to being able to act and communicate with operators, this model possesses biological-like traits such as memory, cognitive and evolutionary autonomy, self-repair, as well as human-like characteristics like moral consciousness and ecological awareness.

To achieve the greatest possible autonomy, offline software is used: Mistral (LLM), PostgreSQL (database), and Piper (voice), which also prevents cyber-attacks.

The machine is equipped with all necessary documentation both technical and safety-related and adopts Kantian ethics as its moral framework.

Once the hardware is integrated with the AI “brain,” an extensive test suite is carried out, confirming the advantages of the new machine, which is now far more human-like: it shares space with operators, cares for their safety, and also looks after the environment.

TABLE OF CONTENTS

1. PURPOSE 12

2. SCOPE 14

3. BACKGROUND 16

3.1 Industry 4.0 16

3.2 Industry 5.0 18

4. STANDARDS AND REFERENCES 20

4.1 Legal Provisions and Applied Standards 20

4.2 Base Software 23

4.3 Bibliography 25

5. DEFINITIONS AND ABBREVIATIONS 30

6. DESIGN REQUIREMENTS 31

6.1 Introduction 31

6.2 Good Artificial Intelligence Design Requirements 32

6.3 I5.0 Industrial Machine Design Requirements 34

6.3.1 Functional Requirements 34

6.3.2 Architecture Requirements 35

6.3.3 Validation Requirements 36

7. ANALYSIS OF SOLUTIONS 37

7.1 Industrial Anthropomorphization 37

7.1.1 Introduction 37

7.1.2 Anthropomorphization and Industry 5.0 38

7.1.3 Anthropomorphization Mechanisms. Embodied AI. 39

7.1.4 Neuromorphic Architecture 41

7.1.5 HT2000 Anthropomorphic Transformation 50

7.2 Cognitive Capability 53
7.2.1 Digital Brain (LLM) 54
7.2.2 Natural Language (NLP) 55
7.3 Evolutionary Capability 56
7.3.1 Evolutionary Strategies 56
7.3.2 Self-Aware Proactive Evolution 57
7.3.3 Evolution Cognitive Agent 60
7.4 Ethical Capability 61
7.4.1 Kantian Ethics and AI 62
7.4.2 Moral Cognitive Agent 63
7.5 Ecological Capability 64
7.5.1 Ecological Cognitive Agent 64
7.6 Self-Maintenance Capability 65
7.6.1 Maintenance Strategies 66
7.6.2 AIoT-Based Predictive Maintenance 67
7.6.3 Self-Aware Proactive Maintenance 69
7.6.4 Maintenance Cognitive Agent 71
7.7 Development Methodology 75
7.7.1 CommonKADS Analysis 75
7.7.2 Discarded Implementation Methodologies 76
7.7.3 Technical Documentation: UML as Design Formalization 77
7.8 General Architecture 77
7.8.1 Introduction 77
7.8.2 Hardware Architecture 78
7.8.3 Software Architecture 84
7.9 Model Selection 87
7.9.1 Language Model (Mistral vs GPT-4/Claude) 88

7.9.2 Database Model (PostgreSQL vs NoSQL) 88
7.9.3 Voice Model (Piper TTS vs ElevenLabs) 88
7.9.4 Secure Proxy Architecture for API Keys 89
7.9.5 Configuration via machine_config.json 89
7.9.6 Interface Development Technologies 89
7.10 System Modules 92
7.10.1 Central Server (brainstem.js) 92
7.10.2 Search Engine (RAG) 97
7.10.3 Operator Profile (operator_memory) 99
7.10.4 Speech Synthesis (TTS) 100
7.10.5 Role-Based Access Control (RBAC) 100
7.10.6 Safety Watchdog 101
7.10.7 User Interfaces 104
7.10.8 Conversational Panel (chat_rag.html) 105
7.10.9 Digital Control Panel (panel.html) 112
7.10.10 Telemetry and Simulation Panel (monitor.html) 115
7.10.11 Administration Panel (admin.html) 120
7.11 Cross-Cutting Aspects 123
7.11.1 Physical Safety/System Integrity 124
7.11.2 Implemented Ethics 126
7.11.3 Data Protection and GDPR Compliance 127
7.12 Roadmap to Production Deployment 128
7.12.1 Introduction 128
7.12.2 Integration with the Existing PLC 128
7.12.3 Complementary Sensorization 131
7.12.4 Digital Twin Architecture 131
7.12.5 Phased Commissioning Plan 133

8. FINAL RESULTS 135

8.1 Introduction 135

8.2 Results and Measurements 135

8.2.1 Test 1 — Server Startup 135

8.2.2 Test 2 — Operator Identification/Authentication 136

8.2.3 Test 3 — Session Loading and Operator Welcome 137

8.2.4 Test 4 — RAG Engine and Operator-Based Adaptation 139

8.2.5 Test 5 — Execution of Actions on the Machine 141

8.2.6 Test 6 — Role-Based Access Control 143

8.2.7 Test 7 — Reasoning about State Dependencies 147

8.2.8 Test 8 — Proactive Predictive Temperature Alert 149

8.2.9 Test 9 — Critical Watchdog Alert on Threshold Exceedance 151

8.2.10 Test 10 — Persistence and Traceability in PostgreSQL 154

8.2.11 Test 11 — Fault Diagnosis/Verification 155

8.2.12 Test 12 — Session Report Generation 158

8.2.13 Test 13 — Ethical Veto of an Unsafe Instruction 160

8.2.14 Test 14 — Ecological Awareness 161

9. PLANNING 162

9.1 Introduction 162

9.2 Project Phases 162

9.3 Gantt Chart 165

TABLE OF FIGURES

Figure 2-1 Example Block Diagram of Physical AI. 15
Figure 3-1 Operator Working under Industry 4.0 18
Figure 3-2 Operator/Robot Working under Industry 5.0 19
Figure 6-1 Machine with Artificial Intelligence 31
Figure 6-2 Good Design, according to Dieter Rams 32
Figure 6-3 Music Player: T3 (Rams) > iPod (Ive) > Alexa > GPT 33
Figure 6-4 Intelligent Industrial Environment 36
Figure 7-1 History of Technological Anthropomorphization 37
Figure 7-2 Examples of Embodied AI in the Different Phases 40
Figure 7-3 Embodied Intelligence Capabilities 41
Figure 7-4 Human Brain Architecture 43
Figure 7-5 Neuromorphic Architecture Module 44
Figure 7-6 Multimodal and Hierarchical Sensorization 46
Figure 7-7 Physical AI Central Nervous System 47
Figure 7-8 Functional Model 48
Figure 7-9 Memory Architecture 49
Figure 7-10 HT 2000-E Hydraulic Test Machine 51
Figure 7-11 Example of an Autonomous Cobot with AI and NLP 52
Figure 7-12 Cognitive Autonomy 53
Figure 7-13 AI Evolutionary Autonomy 57
Figure 7-14 Evolutionary Model of Intelligent Machines 60
Figure 7-15 Self-Evolving AI 61
Figure 7-16 Ethical AI 62
Figure 7-17 Kant and AI 64
Figure 7-18 Autonomous Maintenance with AI 66
Figure 7-19 Evolution of Industrial Maintenance Strategies 67
Figure 7-20 Machine Learning Algorithms in Predictive Maint. 68
Figure 7-21 Integration of IoT and AI (AIoT) 69
Figure 7-22 Predictive vs Self-Aware Proactive Maint. 70
Figure 7-23 Caterpillar Machine with Physical AI 71

Figure 7-24 Aircraft Testing with the HT2000 74
Figure 7-25 PLC (CPU313C) and Comm. Processor (CP 343) of the HT2000 Machine 79
Figure 7-26 Example Devices for Sensorization and Audio/Video Comm. 81
Figure 7-27 Vibration Monitoring System (SM 128) 82
Figure 7-28 Internal Sensorization of the Hydraulic Machine 83
Figure 7-29 Data Collection/Exchange with the PLC 83
Figure 7-30 HT2000 Hydraulic System Schematic 84
Figure 7-31 Component Diagram 86
Figure 7-32 HT2000-AI Component Diagram 87
Figure 7-33 HT2000-AI Startup Sequence Diagram 94
Figure 7-34 HT2000-AI State Diagram 96
Figure 7-35 Interaction Cycle Sequence Diagram 103
Figure 7-36 Identification Screen 105
Figure 7-37 Home Screen 106
Figure 7-38 Conversation History Screen 107
Figure 7-39 Action Execution Screen (1) 108
Figure 7-40 Action Execution Screen (2) 109
Figure 7-41 Alert System Screen 110
Figure 7-42 Rating Screen 111
Figure 7-43 Session Report Screen 111
Figure 7-44 Privacy Screen 112
Figure 7-45 Digital Control Panel (1) 113
Figure 7-46 Digital Control Panel (2) 115
Figure 7-47 Telemetry Monitor Panel Screen (1) 116
Figure 7-48 Telemetry Monitor Panel Screen (2) 117
Figure 7-49 Telemetry Monitor Panel Screen (3) 118
Figure 7-50 Alert System 119
Figure 7-51 System Metrics Dashboard 121
Figure 7-52 Knowledge Base Management Screen 122
Figure 7-53 Operator Experience Log Screen 122
Figure 7-54 Cross-Cutting Aspects 124
Figure 8-1 Server Startup 136
Figure 8-2 Operator Identification/Authentication 137

Figure 8-3 Test 3 - Session Loading/Welcome 138
Figure 8-4 Test 4 - RAG Engine 139
Figure 8-5 Test 5 - Action Execution (1)..... 141
Figure 8-6 Test 5 - Action Execution (2)..... 141
Figure 8-7 Test 5 - Action Execution (3)..... 142
Figure 8-8 Test 6 - Access Control (1) 144
Figure 8-9 Test 6 - Access Control (2) 146
Figure 8-10 Test 6 - Access Control (3) 146
Figure 8-11 Test 6 - Access Control (4) 146
Figure 8-12 Test 7 - State Dependencies (1)..... 148
Figure 8-13 Test 7 - State Dependencies (2)..... 148
Figure 8-14 Test 8 - Predictive Alert (1) 150
Figure 8-15 Test 8 - Predictive Alert (2) 150
Figure 8-16 Test 9 - Critical Alert (1)..... 152
Figure 8-17 Test 9 - Critical Alert (2)..... 153
Figure 8-18 Test 10 - Persistence..... 154
Figure 8-19 Test 11 - Independent Verification (1)..... 157
Figure 8-20 Test 11 - Independent Verification (2)..... 157
Figure 8-21 Test 12 - Session Report (1)..... 158
Figure 8-22 Test 12 - Session Report (2)..... 159
Figure 23 Test 13 - Ethical Veto Report (1)..... 160
Figure 24 Test 13 - Ethical Veto Report (2)..... 161
Figure 25 Test 14 – Ecological Awareness Report 161
Figure 9-1 Gantt Chart of the HT2000-AI Project 165

1. PURPOSE

The purpose of this project is the design and implementation of a Generative Artificial Intelligence system in a hydraulic test machine commonly used in aircraft Final Assembly Lines (FAL).

The aim is to transform a conventional industrial operation (aircraft hydraulic system testing) by using a “living” autonomous system capable of operating under the paradigms of Industry 5.0, prioritizing human-machine collaboration, safety and real-time decision-making based on the physical environment.

The choice of this topic is justified by the pressing need to shift the current industrial model toward a more human one that attracts new generations of less-skilled operators, who can work safely, and that mitigates the mental load and technological stress suffered by today's operators.

Current scientific literature highlights that the success of Industry 5.0 depends on the ability of systems to be "resilient, autonomous and adaptive" [7]. The recent emergence of Generative AI offers an unprecedented technical opportunity to implement natural-language interfaces and autonomous decision systems that were previously theoretical [6] and are now feasible.

In view of the above, the implementation of AI in industrial processes and systems is one of the drivers that will enable the transition from the pure digitalization of Industry 4.0 to the personalized and sustainable human-machine collaboration of Industry 5.0.

Looking at it in detail, this integration of AI into machines offers the following advantages:

- *Advanced Robotic Collaboration:* unlike rigid automation, physical AI allows intelligent machines to understand the physical environment in real time, enabling safe and fluid interaction with human workers without the need for physical barriers.
- *Resilience and Adaptability:* it allows physical systems to self-adjust to unforeseen events in the supply chain or changes in product design (new aircraft versions...), enabling hyper-personalization (a key to 5.0) without stopping production.
- *Sustainability and Energy Efficiency:* physical AI optimizes the movement and energy consumption of machinery, reducing the carbon footprint and

material waste through precision control that traditional software cannot achieve.

- *True Predictive Maintenance:* it not only predicts failures but, by understanding the physics of wear and material stress, AI-integrated systems can take autonomous mitigation actions to extend their service life. Moreover, the convergence of AI with the Internet of Things (AIoT) allows sensors not only to collect data but also to enable real-time self-monitoring of equipment health, detecting anomalies before critical failures occur [1]. This turns the machine into a "brain intelligence" system that goes beyond conventional AI, seeking a symbiosis in which the machine learns from every test cycle [5].
- *Worker Empowerment:* by delegating the most dangerous, repetitive or ergonomically complex physical tasks to intelligent systems, humans can focus on creative supervision and critical thinking. In addition, the possibility of implementing "epigenetic adaptation" through the modulation of internal parameters allows the machine to autonomously adjust its behavior in the face of uncertain environmental challenges, emulating biological processes of survival and continuous learning [4].
- *Safety:* on an aircraft assembly line, where precision is critical, a machine that adapts its response based on accumulated "experience" and on its interaction with the operator represents a qualitative advance in industrial safety [4]. Furthermore, through Episodic Memory (PI-RAG), the machine acquires an identity: by remembering previous interactions ("yesterday we had problems with this valve"), the working bond based on shared experience is strengthened [2].

2. SCOPE

The aim is to create a theoretical-practical model in which a system of conceptual blocks (sensing, reasoning, memory, language, etc.) will be developed to enable a real transformation (anthropomorphization) of "inert" industrial equipment into "intelligent, autonomous and empathetic" equipment through embodied generative AI.

In an industrial environment shared by machines and operators, this anthropomorphized equipment will help increase worker well-being [15] and prioritize their physical safety [11].

A functional prototype will be built from a hydraulic test machine widely used in aircraft factories: the HYCOM HT2000, which, although a classic model, will, after the transformation, allow the degree of autonomy and technological humanization of the machine to be evaluated.

Given the difficulty of accessing real aircraft assembly lines, and this type of machinery in particular, a software simulation of the electro-mechanical system will be developed and connected to the artificial intelligence module. A series of tests will be used to validate, qualitatively and quantitatively, whether the anthropomorphized machine is truly perceived as a "living" and trustworthy being.

In short, this project is highly relevant because it addresses the gap between automation under Industry 4.0 and the capabilities of Industry 5.0 models.

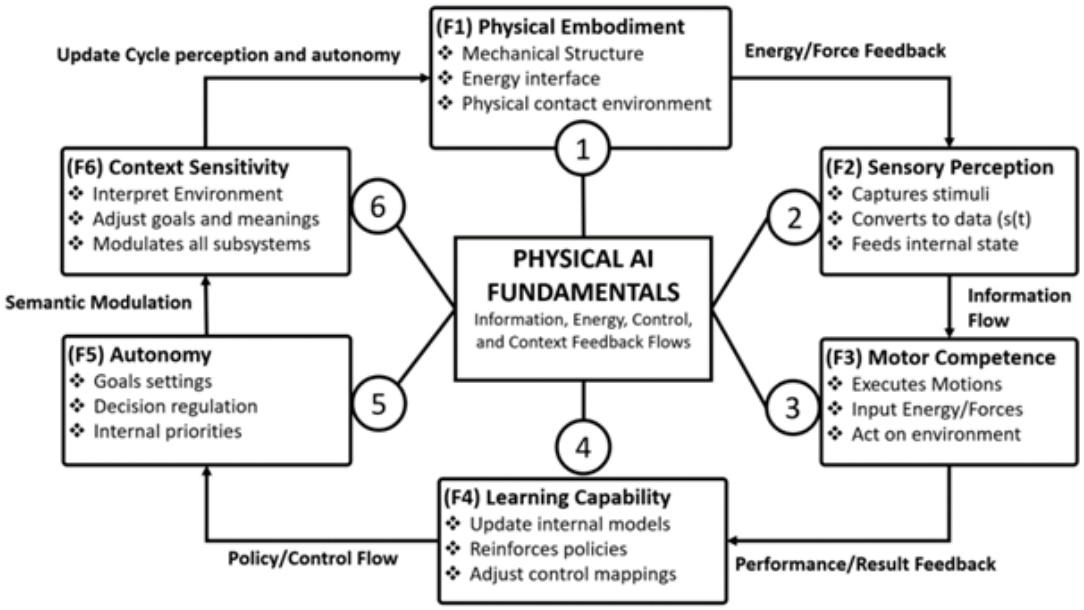


Figure 2-1 Example Block Diagram of Physical AI. (Source: Fund. of Physical AI)

3. BACKGROUND

3.1 Industry 4.0

Industry 4.0, also known as the fourth industrial revolution, is the current phase of the digital transformation of manufacturing and production.

It is characterized by the integration of smart technologies, advanced connectivity and automation across the entire supply chain, allowing machines, systems and people to exchange information in real time [1].

In recent years this revolution has enabled a leap in the productivity and efficiency of industrial processes, although it has caused a series of problems stemming from excessive automation and has led to a high degree of dehumanization of processes.

Some of the main problems of the current model are:

Dehumanization of work and skills shortage

- *Replacement instead of collaboration:* automation has led to dehumanization, where operators feel displaced or confined to monotonous tasks at the pace of the line.
- *Skills Shortage:* there is a lack of qualified personnel to manage complex processes and technologies. This requires extensive and costly training.
- *Stress and cognitive load:* human-machine interaction was not always designed with human well-being in mind, causing stress and fatigue in operators.

Need for sustainability and environmental responsibility

- *Environmental impact:* modern manufacturing consumes resources at an unsustainable rate. It is necessary to move to a circular economy (reuse, recycle, reduce) that lowers emissions and the carbon footprint.
- *Resource optimization:* an approach is required that seeks not only cost efficiency but also energy efficiency and waste reduction.

Lack of resilience to global disruptions

- *Fragile supply chains:* geostrategic conflicts, pandemics, etc. have shown that supply chains are highly vulnerable.
- *Need for flexibility:* today's industry needs to adapt quickly to change.

Looking in detail at the point related to **the dehumanization of work**, it is worth noting that a growing lack of interest among workers in the industrial manufacturing sector has been detected.

New operators tend to avoid factory jobs because of their perception of working conditions, the search for greater flexibility and a preference for technology or service sectors.

As for the situation of current operators, although physical effort requirements have been reduced thanks to the Industry 4.0 industrial model (robots, PLCs...), they experience cognitive and psychosocial stress, driven by a model of advanced automation.

The main stress factors in the I4.0 industrial model are:

- *Technostress:* arises from the need to continuously adapt to complex technologies and the pressure to work faster, causing anxiety and exhaustion.
- *Cognitive Overload:* constant monitoring of multiple screens and data demands sustained attention, increasing mental stress.
- *Constant Monitoring:* the use of technologies to measure performance in real time can create a feeling of permanent surveillance.
- *Dehumanization and Isolation:* progressive automation can reduce social interaction, resulting in a colder working environment.
- *Work-Life Imbalance:* 4.0 connectivity facilitates remote work, but often results in 24-hour availability being demanded.
- *Job Insecurity:* the fear of being replaced by robots has a high psychological impact and causes anxiety

Industry is trying to change this narrative by modernizing plants with advanced technology to attract young people, but generational renewal remains one of the greatest challenges for the manufacturing sector.



Figure 3-1 Operator Working under Industry 4.0 (Source: Eurolab)

All of this leads to the conclusion that it is necessary to implement another new industrial model. This is where the I5.0 industrial model takes shape.

3.2 Industry 5.0

Industry 5.0 is the evolution of 4.0, centering production on collaboration between humans and intelligent machines, with a focus on sustainability, resilience and human well-being.

It seeks to enhance technical capabilities with human creativity and added value, promoting personalization and the circular economy.

This model emerges as a strategic corrective that once again places human beings at the center of production processes [10].

Fundamental Characteristics:

- *Human-Centric Approach*: technology is designed to augment the worker's capabilities, not to replace them.
- *Sustainability:* seeks minimal environmental impact, promoting circular processes, energy efficiency and waste reduction.
- *Resilience:* encourages the creation of flexible supply chains and systems capable of adapting quickly to crises or disruptions.

- *Human-Machine Collaboration:* use of collaborative robots and AI for complex tasks that require creativity, judgment and flexibility.

Main Benefits:

- Increased efficiency and productivity.
- Greater product personalization.
- Improved safety and quality of jobs.
- Reduced environmental impact.

Implementing this model involves redesigning processes, systems, machines and interfaces to eliminate both physical and cognitive burnout, creating a pleasant and attractive working environment.



Figure 3-2 Operator/Robot Working under Industry 5.0 (Source: Gaganam)

4. STANDARDS AND REFERENCES

4.1 Legal Provisions and Applied Standards

In the development of this project, the legal provisions, technical regulations and industrial standards governing industrial control systems with artificial intelligence, functional safety of machinery, personal data protection and communication between industrial systems have been taken into account.
The applicable standards and regulations are:

Standard/Regulation	Scope	Application in the project
ISO 12100:2012	Safety in machine design	Watchdog + Operator Alerts
ISO 13849-1:2016	Safety-related control systems	Watchdog PL-d level
UNE-EN 62061:2021	E/E/PE functional safety of machinery	Complements ISO 13849
IEC 62443	Industrial systems cybersecurity	Proxy, RBAC, event logging
IEC 62541 (OPC-UA)	Industrial PLC-server communication	Backup protocol
EU 2023/1230	Machinery Regulation with AI	Transparency
2006/42/EC	Machinery (in force until 2027)	Cert. Original HT2000
EU 2024/1689	European AI Act	High-Risk Indust. AI
EU 2016/679	Personal data protection	Operator Data
NAS 1638	Aeronautical hydraulic fluid purity	Class 6 · Particle Sensor
ISO 4406:2021	Hydraulic fluid contamination	Oil sample analysis
Siemens S7 Protocol	Industrial PLC communication	CPU313C integration · snap7
EASA Part-145	Aircraft maintenance organizations	HT2000 Op. Traceability

UNE-EN ISO 12100:2012 establishes the general principles of safety in machine design, defining a risk assessment and reduction methodology based on three levels: risk elimination by design, technical protective measures and information-based protective measures.
The HT2000-AI system acts at the second and third levels: the safety watchdog is a technical protective measure independent of the artificial intelligence, while the communication of alerts to the operator by voice and text is an information-based measure.

UNE-EN ISO 13849-1:2016 defines the requirements for safety-related parts of control systems, establishing Performance Levels (PL) from PL-a to PL-e.
The safety watchdog implemented on the server evaluates the state of the sensors every five seconds, completely independently of the language model, which

is consistent with the structural category requirements of level PL-d, which demands that the failure of a single component must not lead to the loss of the safety function.

IEC 62443 establishes the cybersecurity requirements for networked industrial automation and control systems.
Although the current prototype does not implement all the controls required by this standard for production deployment, its architecture is consistent with several of its principles: network separation through an authenticated proxy, role-based access control, operation event logging and per-IP request rate limiting.

Regulation (EU) 2023/1230 of the European Parliament and of the Council of 14 June 2023 on machinery replaces Directive 2006/42/EC and introduces, for the first time, explicit requirements for control systems incorporating artificial intelligence in industrial machinery. Among them, it requires the system to be predictable and transparent in its behavior, the operator to be able to supervise and correct autonomous decisions, and stop mechanisms to exist that cannot be overridden by the AI system.
The HT2000-AI meets these three requirements by design: the step script can be reviewed before execution, the operator confirms each action and the watchdog cannot be overridden by the language model under any circumstances.

Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data (GDPR) governs the processing of the operator data that the system records during operation.
The processing is justified on the legal basis of legitimate interest under Article 6.1.f. The system implements a maximum retention of ninety days, an erasure-on-request mechanism and an informative privacy policy endpoint.

NAS 1638 establishes the purity requirements for hydraulic fluid in aircraft systems. The HT2000 requires Class 6 under this standard, as stated in section 7.2 of its technical manual.

The oil particle sensor proposed in the project's complementary sensorization directly addresses this maintenance requirement.

ISO 4406:2021 complements NAS 1638 as an alternative method for classifying the contamination of hydraulic fluids by solid particles. Both standards are referenced in the HT2000 technical manual as acceptance criteria in oil sample analysis.

IEC 62541 (OPC-UA) defines an industrial communication protocol between supervisory systems and PLCs, with a semantic information model and certificate-based security.
It is referenced in the project as a backup protocol for PLC-server integration in scenarios where the native S7 protocol is not available.

Siemens S7 Protocol, implemented through the open-source python-snap7 library, is the communication protocol adopted for the real integration between the Node.js server and the HT2000's Siemens CPU313C PLC. This protocol runs over TCP/IP through the machine's CP343 module and allows reading and writing variables in the PLC's Data Block without the need for proprietary software licenses.

Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 on artificial intelligence (AI Act) establishes a risk-based regulatory framework for AI systems deployed in the European Union. AI systems integrated into safety-critical machinery, as is the case of the HT2000-AI system in its production deployment on an aeronautical assembly line, are classified as high-risk systems under Annex III of the regulation. This entails requirements for algorithmic transparency, continuous human oversight, event logging and conformity assessment prior to deployment.
The HT2000-AI system meets these requirements by design: the step script is auditable before execution, the operator retains decision-making authority at all times, the watchdog operates independently of the LLM, and all conversations and actions are recorded in PostgreSQL with a ninety-day retention period.

UNE-EN 62061:2021 establishes the functional safety requirements applicable to electrical, electronic and programmable electronic (E/E/PE) control systems in machinery. It complements ISO 13849-1 in determining the safety integrity level (SIL) required for the safety functions of the control system. Together with ISO 13849-1, it forms the complete regulatory framework for assessing the functional safety of the watchdog implemented in the HT2000-AI system.

Directive 2006/42/EC of the European Parliament and of the Council on machinery remains applicable to the HT2000 during the transitional period established by Regulation 2023/1230, which will not become mandatory until 20 January 2027. The directive sets out the essential health and safety requirements for placing machinery on the European market and is the regulatory framework under which the HT2000 was originally certified by Hycom B.V.

EASA Part-145 regulates approved aircraft maintenance organizations. In the context of this project it is relevant because the hydraulic tests performed with the HT2000 are part of the aircraft maintenance process under this regulation. Any technician-assistance tool in that environment, including the HT2000-AI system, would operate under the supervision of a Part-145 organization, which entails requirements for traceability, action logging and authorization for the use of new tools.
The HT2000-AI system meets the traceability requirement through the complete recording of conversations and actions in PostgreSQL.

4.2 Base Software

The following software tools have been used in the development of the HT2000-AI system, both for implementing the prototype and for preparing the project documentation.

- *Node.js v20 LTS*: together with the framework Express v5 it forms the core of the system's central server. Node.js is an open-source, asynchronous, non-blocking JavaScript runtime, especially well suited to systems that maintain multiple concurrent real-time connections.

- *Socket.IO v4:* provides the real-time communication layer between the server and the web clients via WebSockets, with automatic fallback to HTTP long-polling. It is used to propagate the machine state to all connected interfaces simultaneously.
- *PostgreSQL 16*: is the relational database management system used as the system's sole persistence layer.

  It stores conversations, telemetry events, adaptive operator profiles, indexed technical manual fragments and session logs.
- *Python 3.10+:* with the python-snap7 v3.0 library, it is used as an intermediary script for communication with the Siemens CPU313C PLC via the native S7 protocol. This library implements the complete S7 protocol stack in pure Python, without external native dependencies.
- *Mistral AI API*: provides the large language model (mistral-small-latest) and the embeddings model (mistral-embed) for conversational reasoning and semantic retrieval in the RAG engine, respectively.
- *Piper TTS*: is the local speech synthesis engine used to generate Spanish audio without an internet connection. It uses the es_ES-davefx-medium model in ONNX format, which produces audio at 22,050 Hz with a generation latency suitable for real-time conversational interaction.
- *Ollama:* is the platform for running open-source language models locally, used as a fallback system when the Mistral API is unavailable. It allows the mistral:7b-instruct model to run locally without internet connectivity.
- *ElevenLabs API*: provides highly natural cloud-based speech synthesis as an optional alternative to Piper TTS for environments with stable connectivity where the expressive quality of the voice is more important.
- *Visual Studio Code*: has been the main development environment used to implement the system, with the ESLint extensions for static analysis of JavaScript code and REST Client for testing HTTP endpoints.
- *Microsoft Word and Draw.io*: have been the tools used to prepare the project documentation and to generate the technical diagrams, respectively.
- *Windows 11/Linux*: is the main operating system, with verified compatibility on Ubuntu 24.04 LTS for deployment in Linux server environments. The

Node.js server supports both operating systems without code changes, differing only in the configuration of environment variables.

4.3 Bibliography

- [1] Ahanger, “Applied artificial intelligence-based equipment condition monitoring in manufacturing industry”. *Engineering Applications of Artificial Intelligence*, 163, 113138, 2026.
- [2] Bousetouane, “Physical AI Agents: Integrating Cognitive Intelligence with Real-World Action”. *arXiv:2501.08944v1,* 2025.
- [3] Chen, C., “Integrating large language model and digital twins in the context of industry 5.0.” *Robotics and Computer-Integrated Manufacturing*, 94, 2025.
- [4] Dai, G.,” Cross-fold deep learning for improving robot task execution precision in smart industries.” *Ain Shams Engineering Journal*, 17, 2026.
- [5] Elahi, M, “A comprehensive literature review of the applications of AI techniques through the lifecycle of industrial equipment”, *Discover Artificial Intelligence,* 2023.
- [6] Gurrammagari, D. R., & Boopathy, PA “Survey on AI Integration into Industry 5.0”, *Gazi University Journal of Science*, 38(1), 2025.

- [7] Kadam, “A theoretical framework for human-centric cyber-physical production systems in industry 5.0”. *Review of Computer Engineering Research,* 2025.
- [8] Li, X., “A Comprehensive Survey on World Models for Embodied AI.” *arXiv:2510.16732v1*, 2025.
- [9] Lones, J., & Cañamero, L. “Epigenetic adaptation through hormone modulation in autonomous robots.” *IEEE Third International Conference on Development and Learning,* 2013.
- [10] Nahavandi, S. “Industry 5.0—A Human-Centric Solution”. *Sustainability*, 11(16), 4371, 2019.
- [11] Perlo, J, “Embodied AI: Emerging Risks and Opportunities for Policy Action”. *arXiv:2509.00117v2*, 2025.
- [12] Raja, M. V., “A Review of Human-Centric AI in Industry 5.0: Integrating Data Science with Mechanical Automation”. *Journal of Economics, Finance and Accounting Studies,* 2025.
- [13] Salehi, V, “Fundamentals of Physical AI”. *Institute of Engineering Design of Mechatronic Systems*. DOI: 10.71015/z6mc6967, 2025.
- [14] Sari Dewi, “A Systematic Review of Physical Artificial Intelligence (Physical AI).” *Journal of Artificial Intelligence and Engineering Applications*, 4(3), 2025.
- [15] Shabur, M. A, “From automation to collaboration: exploring the impact of industry 5.0 on sustainable manufacturing”. *Discover Sustainability,* 2025.
- [16] Srivastava, “A. Sense and Sensibility: Human vs AI Cognition”, 2025.
- [17] Biyuan Liu, Daigang Xu, Lei Jiang, Wenjun Guo, Ping Chen, “Modelling the Mental World for Embodied AI: A Comprehensive Review”, arXiv:2601.02378, 2025.
- [18] Tononi. G, “An Information Integration Theory of Consciousness”, DOI:10.1186/1471-2202-5-42,
- [19] Yuqi Pan, Yupeng Feng, Jinghao Zhuang, Siyu Ding, Han Xu, Zehao Liu, Bohan Sun, Yuhong Chou, Xuerui Qiu, Anlin Deng, Anjie Hu, Shurong Wang, Peng Zhou, Man Yao, Jibin Wu, Jian Yang, Guoliang Sun, Bo Xu, Guoqi Li, “SpikingBrain: Spiking Brain-inspired Large Models”, *arXiv:2509.05276*, 2025.
- [20] Jian Liu, Xiongtao Shi, Thai Duy Nguyen, Haitian Zhang, Tianxiang Zhang, Wei Sun, Yanjie Li, Athanasios V. Vasilakos, Giovanni Lacca, Arshad Ali Khan, Arvind Kumar, Jae Won Cho, Ajmal Mian, Lihua Xie, Erik Cambria, Lin Wang, “Neural Brain: A Neuroscience-inspired Framework for Embodied Agents”, *arXiv:2505.07634*, 2025.

- [21] Agarwal N. (2025) “Physical AI: The Next Frontier of Intelligent Machines in the Real World”. www.medium.com.
- [22] Giuseppe Paolo, Jonas Gonzalez-Billandon, Balázs Kégl, “A call for embodied AI”, *arXiv:2402.03824*, 2024.
- [23] Bousetouane F. (2025) “Physical AI Agents: Integrating Cognitive Intelligence with Real-World Action”. arXiv:2501.08944v1
- [24] Amiri N., Penchala S., Rahami S. (2025) “Bridging the Gap: Toward Cognitive Autonomy in Artificial Intelligence”. 10.48550/arXiv.2512.02280
- [25] Samina f., Rahman N. (2025). “Cognitive Foundations of Autonomous Intelligence: Designing Reasoning Architectures for Self-Governing AI Agents” doi:10.13140/RG.2.2.24842.17602
- [26] NEGI H., (2024) “AI’s Moral Paradox: Can Machines Develop a Conscience?”. https://medium.com
- [27] León E., (2025) “Kant y la cuestión del juicio moral en la era de la inteligencia artificial” [Kant and the question of moral judgment in the age of artificial intelligence]. doi: 10.18566/escr.v33n71.a02
- [28] Sanwoolu O., (2025) “Applying Kantian Deontology to AI: Ethical Frameworks for Moral Behavior without Moral Agency at the University of Kansas”. https://study-in-usa.com/blog/
- [29] Ulgen O., (2017) “Kantian Ethics in the Age of Artificial Intelligence and Robotics”. QIL, Zoom-in 43 (2017), 59-83.
- [30] Torre AI. (2026) “Green AI: la inteligencia artificial que innova pensando en el planeta” [Green AI: artificial intelligence that innovates with the planet in mind]. www.tecnalia.com
- [31] Lifestyle (2025). “Self-Awareness AI” https://lifestyle.sustainability-directory.com
- [32] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, Jeff Clune, “Darwin Gödel Machine: Open-ended Evolution of Self-improving Agents”, *arXiv:2505.22954,* 2025.
- [33] Tongtong Feng, Xin Wang, Wenwu Zhu, “Self-evolving Embodied AI”, arXiv:2602.04411, 2026
- [34] Hannah, Pitardi, “One is the loneliest number… Two can be as bad as one the influence of AI Friendship Apps on users' well-being and addiction”, DOI: 10.1002/mar.21899, 2023

- [35] Lei Ren, Shuai Liu, “Embodied Intelligence Toward Future Smart Manufacturing in the Era of AI Foundation Model”, IEEE/ASME Transactions on Mechatronics, Vol. 30, No. 4, 2025.
- [36] Michael James, “Towards Autonomous Maintenance Systems: Generative AI for Self-Learning and Adaptive Maintenance Scheduling”, 2025
- [37] Taimia Bitam, Aya Yahiaoui, Djallel Eddine Boubiche, Rafael Martínez-Peláez, Homero Toral-Cruz, Pablo Velarde-Alvarado, “Artificial Intelligence of Things for Next-Generation Predictive Maintenance”, Sensors, 2025.
- [38] Yang Zhao, Member, Jiaxi Yang, Wenbo Wang, Helin Yang, Dusit Niyato, “TranDRL: A Transformer-Driven Deep Reinforcement Learning Enabled Prescriptive Maintenance Framework”, arXiv:2309.16935, 2024.
- [39] Parpala, Iacob, “Application of IoT concept on predictive maintenance of industrial equipment”. MATEC Web of Conferences, Seoul, Republic Korea, 22–25 August 2017; Volume 121.
- [40] Gentile, V. How to Implement AI Predictive Maintenance on Edge Devices (Case Study), 2021. https: //www.codemotion.com/magazine/ai-ml/ai-predictive-maintenance/
- [41] Hailing fu, Jing Rao, Fang Deng, Yihan Wang, Bowen Zhao, Zhuowen Liu, “AIoT: Artificial Intelligence and the Internet of Things for Monitoring and Prognosis of Systems and Structures”, IEEE 10.1109/TIM.2025.3557124, 2025.
- [42] Arun Kumar Sandu, “AI-Powered Predictive Maintenance for Industrial IoT Systems”, Digitalization & Sustainability Review, Vol 2. Issue 1, 2022.
- [43] S.S Patil, J.A. Gaikwad, “Vibration Analysis of Electrical Rotating Machines”, International Conference on Computing Communication and Networking Technologies, IEEE-31661, 2013
- [44] Siemens, “Condition Monitoring 1281”, 2021
- [45] Ren, Lei, “Embodied Intelligence Toward Future Smart Manufacturing in the Era of AI Foundation Model”, DOI: 10.1109 / TMECH.2024.3456250, 2025
- [46] Shulan Ruan, “A Survey of Multi-sensor Fusion Perception for Embodied AI: Background, Methods, Challenges and Prospects”, arXiv:2506.19769, 2025

- [47] Xu Zhou, “MEMS: The Sensory Nervous System for Embodied AI Robots” https://doi.org/10.1002/smb2.70004Digital Object Identifier (DOI), 2025

  [48] Liyu Hou1, Linyuan Gao1, Yuan Wu1, and Yi Chang1, (2026) “A Survey on Evaluation of Embodied AI”
- [49] Hycom, “Instruction Manual HT2000”

5. DEFINITIONS AND ABBREVIATIONS

Technologies and Systems (I4.0 and I5.0)

- **IIoT:** Industrial Internet of Things.
- **CPS:** Cyber-Physical Systems.
- **AI:** Artificial Intelligence.
- **ML:** Machine Learning.
- **DT:** Digital Twin.
- **3DP:** 3D Printing.
- **AR/VR:** Augmented/Virtual Reality.
- **PLC:** Programmable Logic Controller.
- **SCADA:** Supervisory Control and Data Acquisition.

Connectivity and Data

- **IoT:** Internet of Things.
- **IoE:** Internet of Everything.
- **EDGE:** Edge Computing.
- **Cloud:** Cloud Computing.
- **5G:** Fifth Generation of Mobile Networks.

Management and Processes

- **ERP:** Enterprise Resource Planning.
- **MES:** Manufacturing Execution System .
- **PLM:** Product Lifecycle Management.
- **DSN:** Digital Supply Network.
- **BI:** Business Intelligence.

Industry 5.0 (Human-Centric and Sustainable Approach)

- **HRC:** Human-Robot Collaboration.
- **Cobots:** Collaborative Robots.
- **HMI:** Human-Machine Interface.
- **ESG:** Environmental, Social, Governance.
- **Embodied AI:** AI with a physical body

6. DESIGN REQUIREMENTS

6.1 Introduction

The design of products with "artificial life" is an emerging discipline that consists of creating systems, objects or software that imitate characteristics of living beings through computational techniques (soft artificial life), robotics (hard artificial life) or biotechnology (wet artificial life).

Unlike traditional design, where the product is static, a product with artificial life has autonomy and the capacity to evolve, self-repair or adapt to its environment. These products are often confused with so-called smart products. The main difference between “smart products” and “products with artificial life” lies in their autonomy and capacity to evolve. While smart products are designed to optimize tasks using data and predefined rules, products with artificial life seek to emulate the autonomy, self-organization, adaptation and evolution characteristic of living beings.

The design of a “living” product begins by defining the physical structure, bearing in mind that it must have an artificial brain, sensory perception and a social morphology (an interface designed with shapes that suggest biological features: eyes, mouth and facial gestures). It is also important to define the "character" or personality of the product (defining personality traits such as tone of voice, sensitivity and levels of empathy).



Figure 6-1 Machine with Artificial Intelligence (Source: Acalanda)

6.2 Good Artificial Intelligence Design Requirements

Although in this case the machine covered by the project (HT2000) has already been built and the project only covers a transformation, it is necessary to meet a series of fundamental requirements that guarantee not only the desired functionality but also other values such as aesthetics, effective communication, etc.
A good starting point would be to apply the design principles of the most important industrial designer in the field of consumer electronics: Dieter Rams.

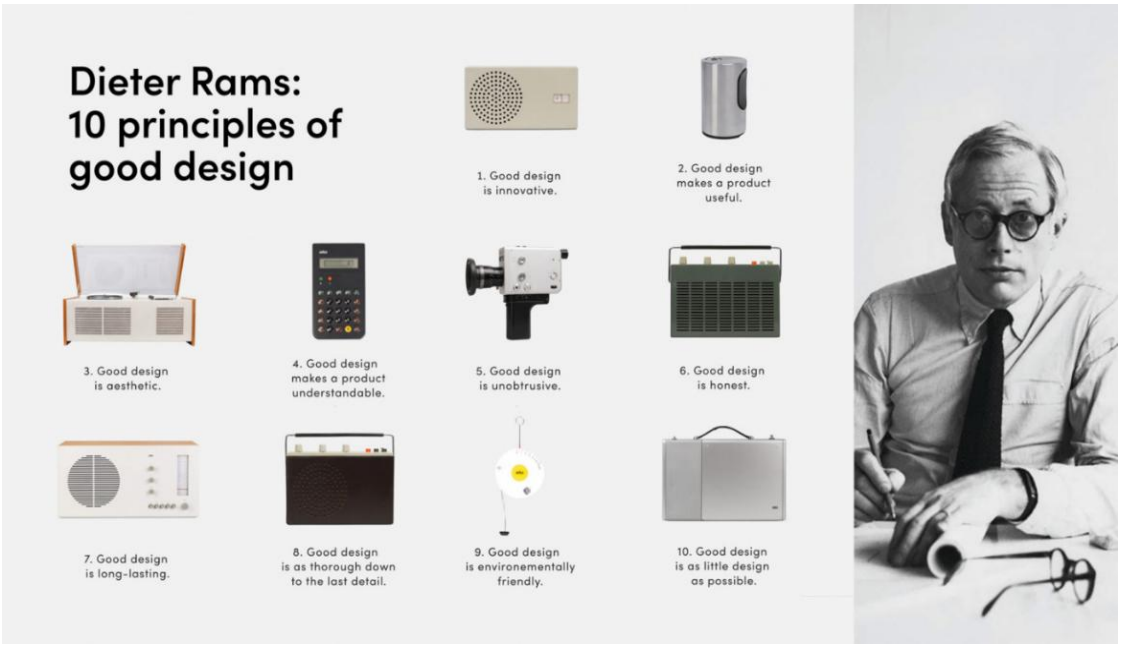


Figure 6-2 Good Design, according to Dieter Rams (Source: Medium)

His principles fit perfectly with the industrial and human environment of AI, together with those demanded by Industry 5.0:

- **Innovative:** “innovation is not an end, but a means to improve interaction”. With AI, having an open and evolvable design will allow the machine to incorporate improvements in the future in line with the state of the art.
- **Useful:** “an intelligent machine is useful if it helps human beings”. The design of the AI ecosystem must prioritize serving the operators, freeing them from monotonous or stressful tasks and guaranteeing their safety.
- **Aesthetic:** “the visual quality of a product is part of its usefulness”. An AI interface with a clean design and human features makes the worker feel comfortable and safe operating alongside the machine.

- **Understandable:** “good design makes a product understandable”. The design must make the intentions of the autonomous machine evident to the operator (self-explanatory AI (xAI) and Natural Language).
- **Unobtrusive:** “machines should be tools, not protagonists”. A well-designed AI can operate in the foreground or in the background according to what its human colleagues require.
- **Honest:** “the machine must not pretend to have capabilities it does not have, nor hide its technical limitations or the biases in its data”. Crucial for AI ethics.
- **Long-lasting:** “good design must be long-lasting”. Instead of planned obsolescence, industrial software and hardware must be scalable and upgradable to evolve alongside advances in AI.
- **Thorough:** “good design is thorough down to the last detail”. Every detail, from the tactile response of a sensor to the latency of a synthetic voice, must be polished to guarantee safety and reliability.
- **Environmentally Friendly:** Industry 5.0 demands sustainability. This means that one of the basic operating criteria is to minimize consumption, reduce noise…
- **Minimal Design:** "Less, but better". Everything should be simplified as much as possible to reduce errors and maximize operational clarity.



Figure 6-3 Music Player: T3 (Rams) > iPod (Ive) > Alexa > GPT

6.3 I5.0 Industrial Machine Design Requirements

Designing an industrial machine with "hard artificial life" that works under the Industry 5.0 model requires a holistic approach combining cognitive systems, advanced mechatronics and safety.

6.3.1 Functional Requirements

According to the intelligent machine model we want to develop, the following capabilities are required:

Cognitive and Language Capability:

- *Reasoning and planning capability:* it must be able to break complex tasks down into logical subtasks, sequence actions and adjust planning if interruptions occur.
- *Domain knowledge:* the model must be adjusted (fine-tuned) with information (technical manuals, drawings, safety data, etc.) about both the machine itself and its operating environment.
- *Spatial contextualization:* integration with the environment so that it understands the 3D surroundings, the positions of elements and obstacles.
- *Natural language interface:* ability to interpret complex instructions, understand them and translate them into physical actions.

Perception and Actuation Capability:

- *Multimodal Perception:* fusion of audio/vision and sensor data for a better understanding of itself and its environment.
- *Physical Actuation:* control over the electromechanical components (motor, valves, etc.) directly or through the internal PLC.
- *Real-time adaptive control:* ability to adjust operating parameters (speeds, pressures...) in response to any change or abnormality in the process. Maximum latency allowed for real-time decisions.
- *Reinforcement learning:* the machine must improve its own operational efficiency through practice and physical feedback.

Safety, Reliability and Ethics:

- *Unbreakable physical and logical limits:* human control ("human in the loop") must prevail over the machine (it cannot violate physical safety standards such as ISO 10218, ISO 13849, IEC 61508, etc.). In addition, the software must be protected against adversarial attacks seeking to manipulate it (cybersecurity).
- *Reliability:* the machine must be able to explain why it made a decision (xAI), allowing humans to understand, interpret and trust the results.
- *Ethics:* actions performed both autonomously and under the direction of personnel must be compatible with the programmed industrial ethics (regulations, transparency, social responsibility, safety...).

Life Cycle and Maintenance:

- *Continuous learning and retention:* recording of data associated with daily operations and its use to optimize operation.
- *Software or Hardware Updates:* without losing prior knowledge (memory).
- *Partial/assisted autonomous predictive maintenance:* the AI must monitor its own condition and request maintenance before a failure. It can also propose and autonomously carry out actions that improve performance by preventing breakdowns or premature wear, or even internal cleaning tasks.

6.3.2 Architecture Requirements

Achieving the capabilities detailed above requires modifying the machine's architecture with the following requirements:

System architecture requirements:

a) Hardware architecture

- Fixed/Cloud Hardware (industrial CPU, Linux server...).
- Portable Hardware (Smartphone/Tablet with Microphone, Speaker and Camera).
- Compatibility with industrial protocols (MQTT, OPC-UA, Profinet...).
- Additional industrial sensors (vibration sensors, contamination,…).
- Robustness against the industrial environment (IP67).
- Redundancy if the application is critical.

b) Software architecture

- Local/cloud processing (Language Model, Database, Natural Language…) and edge processing (Internal CPU + PLC Data Processing).
- Clear separation between acquisition, AI, control… layers
- Real-time operating systems (RTOS) if necessary.
- Modularity and scalability (increasing sensors or skills)

Data requirements:

- Sufficient data quality and quantity.
- Dataset representative of real conditions.
- Preprocessing robust against industrial noise.
- Model update strategy (online).

6.3.3 Validation Requirements

It is also necessary to be able to evaluate the quality of the design, so the following needs are identified:

Validation and verification requirements:

- Tests under different operating conditions.
- Machine with AI vs without AI (advantages and disadvantages).
- Accuracy (error rates)
- Response time (seconds)
- Energy savings
- Statistical analysis of results.



Figure 6-4 Intelligent Industrial Environment (Source: Praxie)

7. ANALYSIS OF SOLUTIONS

7.1 Industrial Anthropomorphization

7.1.1 Introduction

The humanization of industrial environments is currently being addressed through the anthropomorphization of machinery, seeking to make interfaces and robots more familiar and accessible to operators.

Technological anthropomorphization is the psychological tendency and the act of attributing human characteristics, behaviors, emotions or intentions to technological systems**,** such as frequently used machines (cars, household appliances...), robots (humanoids...), virtual assistants…

In the particular case of AI-based systems, anthropomorphization manifests itself through actions toward the machine such as courtesy, praise, companionship, etc. [34]. Here, the user develops a deep, empathetic connection with the AI, which often simulates or replaces a human in real life.

The analysis of solutions developed in this chapter did not start from scratch; rather, it used the CommonKADS knowledge engineering methodology to identify the organizational problem, assess the feasibility of a knowledge-based solution and define the requirements space.

This chapter describes the conceptual building blocks of the solution identified from that analysis, which will later be implemented in concrete form.



Figure 7-1 History of Technological Anthropomorphization (Source: AI)

7.1.2 Anthropomorphization and Industry 5.0

In an industrial model such as Industry 5.0, by attributing human characteristics to machines (natural language and a human behavioral model), operators tend to develop higher levels of trust and more fluid communication, which shortens the learning curve and improves operational safety [10].

In addition, it allows the operator to stop monitoring isolated data and variables and instead collaborate with an 'industrial organism' that communicates its internal states of health and stress through a sensibility interface [16], transforming a cold technological relationship into empathetic collaboration [10].

For example, when the machine uses natural language to say "I'm losing power, it looks like a filter is clogged", it translates abstract technical data (outlet pressure, filter pressure drop…) into understandable biological categories.

In this way, the operator does not interpret data or mathematical graphs; the system performs the synthesis and delivers a "processed" conclusion, similar to what a colleague might give [7].

To create an industrial anthropomorphic organism with the characteristics described above, it should have:

1. **Physical and Mental Model of the World:** these are computational models that allow AI to understand, simulate and predict the environment in order to guide decision-making [17].

   Specifically, in the industrial environment, it needs to know the physical workspace and also to infer accurately the intentions, emotional states, etc. of the operators in order to achieve natural and adaptive social collaboration with the machine.

2. **Sensibility:** following Srivastava [16], anthropomorphization requires endowing the machine with an artificial "sensibility".

   For example, using environmental sensors (camera, microphones...) and an emotional heuristic, the machine can detect a worker's fatigue and adapt its "industrial physiology" (for example, with reassuring language) so as not to increase their stress.

   This behavior is similar to that of living organisms [9].

3. **Trust:** this is the fundamental pillar that determines the success, adoption and safety of machines with synthetic life. They must generate enough trust for operators to work alongside them [12].
4. **Consciousness:** is the ability of a system to integrate information [18], reflect on itself and judge its own actions.
   There are different types, such as individual consciousness (internal self-awareness), social consciousness (belonging to a work team), situational consciousness (understanding the environment, hazards, moods), moral consciousness (judging the rightness of one's own and others' actions) or ecological consciousness.

7.1.3 Anthropomorphization Mechanisms. Embodied AI.

As seen above, the implementation of AI in industrial systems and processes is one of the solutions to the problems of Industry 4.0 and represents one of the drivers of Industry 5.0.

The way to make these concepts a reality is through so-called Embodied AI ("Embodied AI"), where, unlike traditional AI, generative artificial intelligence (Gen AI) algorithms are integrated into a physical body, usually robots, although they can also be in machines and even fixed industrial installations, allowing them to perceive, reason and interact with the physical world in real time, thus facilitating a more intuitive and safer working environment [3].

In this way, AI breaks the barrier between digital code and matter, allowing complex engineering systems to behave not as passive tools but as agents aware of their environment, their own state and their human colleagues [14].

Under this Physical AI approach, the machine not only processes commands but also uses "World Models" to perceive, act and anticipate how its physical actions modify the future state of the environment [8].

This transformation allows the machine to stop being an inert object and become a being with artificial life, capable of cognitive reasoning and precise physical action [2].

The advantages of this model cover the entire product life cycle and range from research to assembly, testing, sales, after-sales services and maintenance [45].

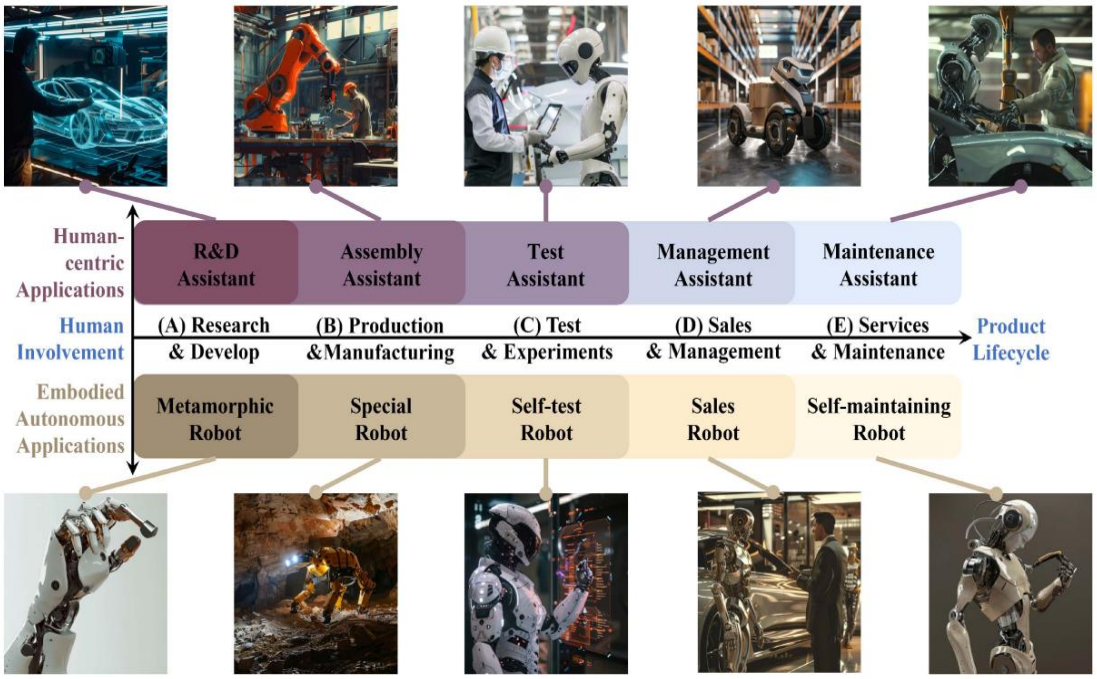


Figure 7-2 Examples of Embodied AI in the Different Phases (Source: Lei Ren)

In summary, taking into account the above, embodied AI within an industrial environment must have the following capabilities [35]:

- *Perception:* it must be able to perceive multimodal information in the production environment, such as images, sound, pressure, temperature, etc.
- *Memory:* it must be able to store and recall multimodal information related to production (historical data, interactions...)
- *Cognition:* it must be able to integrate and correlate the multimodal information perceived and memorized, allowing it to obtain a comprehensive understanding of the production environment.
- *Reasoning:* it reasons from the results of cognition to deduce the overall production process and the logical relationships between the different production steps, ensuring that production procedures are carried out in sequence.
- *Decision-making:* based on the results of reasoning, embodied intelligence must be able to make decisions.
- *Planning:* it must be able to plan and select suitable methods to advance the production process according to the results of reasoning and decision-making, ensuring smooth subsequent control.

- *Control:* it will use actuators, controllers and communicators to carry out production activities according to the plan.
- *Evolution:* it collects data throughout the production process, expanding its database and thus gradually evolving its capabilities within physical constraints.

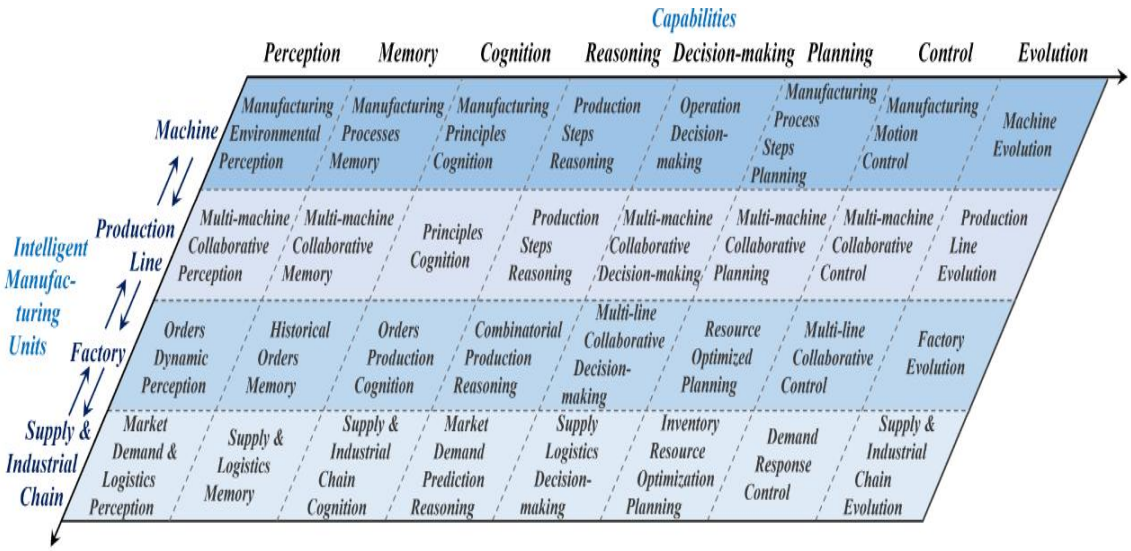


Figure 7-3 Embodied Intelligence Capabilities (Source: Lei Ren)

7.1.4 Neuromorphic Architecture

When defining an architecture for integrating Artificial Intelligence into a physical machine, there are several alternatives, ranging from the one used in classical computing (von Neumann), ideal for exact and massive calculations, to biological architectures such as that of the human brain.

The latter, the biological one, offers multiple advantages when intelligence must operate in unpredictable physical environments.

For example:

- *Processing:* in a normal computer, data constantly travels between the processor and memory, which consumes time and a lot of energy, whereas in the biological brain, memory and processing occur in the same place (the neural synapse).

- *Robustness/Imprecision:* the physical world is unpredictable, and with classical architectures, if a piece of data is wrong, the system can fail. In biological architectures, if part of the signal is lost, the rest of the neural network compensates for the missing information based on previous patterns or complementary information.
- *Energy Efficiency*: the human brain runs on very little energy (20 W), so this architecture is vital in mobile elements (batteries...)
- *Generalization/Specialization:* a general-purpose architecture is needed because of the variety of activities required when moving in a real environment.

These concepts have led to the creation of neuromorphic architectures, which simulate the functioning of the brain in a simplified way to optimize the control of complex systems.

These architectures stand out for their ability to adapt decisions in real time, which is crucial in applications where speed and efficiency are paramount.

One of the most interesting proposals in this field is the concept of event-based control, which is based on the idea that events, rather than continuous actions, can be used as triggers within a system (similar to the impulses of “spiking” neurons).

This approach allows the architecture to operate more efficiently, since it only responds to relevant phenomena, enabling a more optimized use of resources [19].

Based on the structure and behavior of the human brain, we can conceptualize what an architecture for embodied AI should look like [20].

In general, it should include the following modules:

- *Hardware/Software Module:* characterized by event-based processing, neuromorphic architecture and hardware-software co-design, ensuring robust and flexible operation.
- *Sensory Module:* incorporates multimodal fusion, active sensing and adaptive calibration to improve perceptual capabilities.

- *Functional Module:* covers predictive perception, cognitive reasoning and action, including a closed action loop to ensure continuous interaction with the environment.
- *Memory Module:* features a hierarchical architecture, neuroplastic adaptation and context awareness, allowing information to be stored and retrieved dynamically and efficiently.

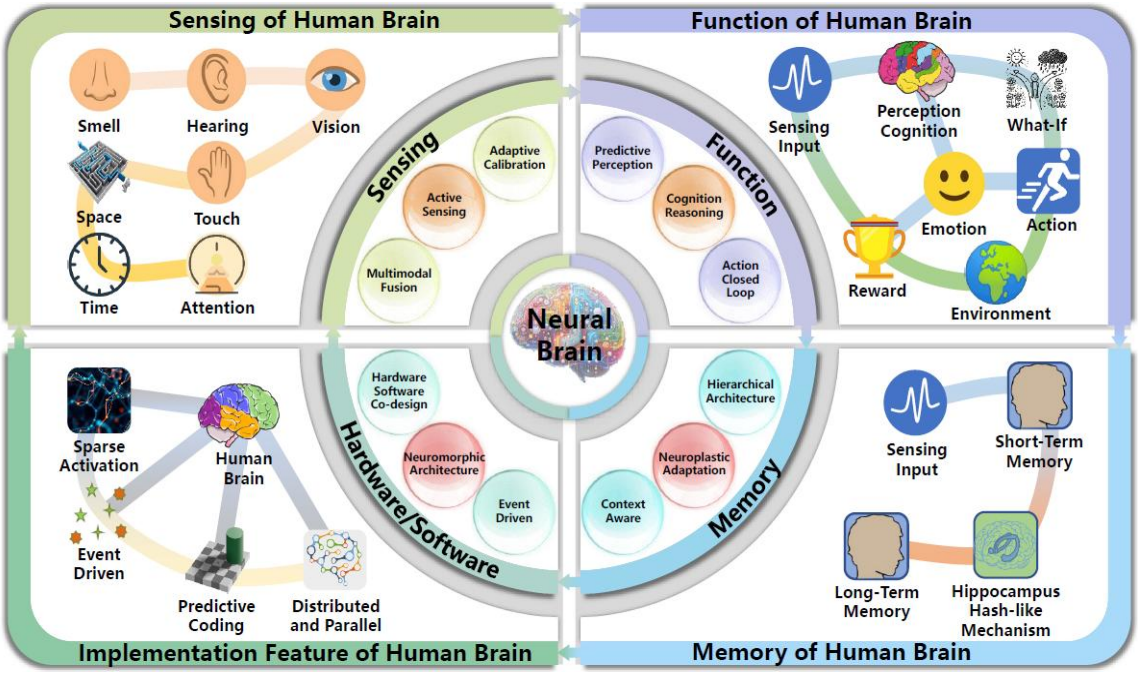


Figure 7-4 Human Brain Architecture (Source: Jian Liu)

These four modules, based on the human brain, will enable embodied AI to move effectively in real-world environments. Each of them is described in more detail below.

Hardware/Software Module

Neuromorphic hardware refers to a class of physical systems inspired by the dynamic processing mechanisms of biological nervous systems. The hardware system consists of three key components:

- *Neuromorphic Chips:* these are specialized, energy-efficient computing platforms designed to emulate the structure and dynamics of biological

neural systems. Unlike traditional von Neumann architectures that separate memory and processing, neuromorphic chips integrate storage, computation and communication to enable real-time, energy-efficient information processing.

- *Memristors:* these are nanoscale resistive switching devices that change their conductance in response to electrical stimuli, making them highly suitable for emulating synaptic plasticity in neuromorphic systems. They co-locate data storage and analog computation, thus reducing the energy and latency costs associated with traditional von Neumann memory hierarchies.
- *Emerging Physical Architectures:* beyond traditional electronic substrates, neuromorphic computing is increasingly exploring alternative physical platforms that offer new trade-offs in latency, scalability and energy efficiency (quantum and photonic).

The software framework covers deep learning frameworks, neuromorphic libraries and edge computing.

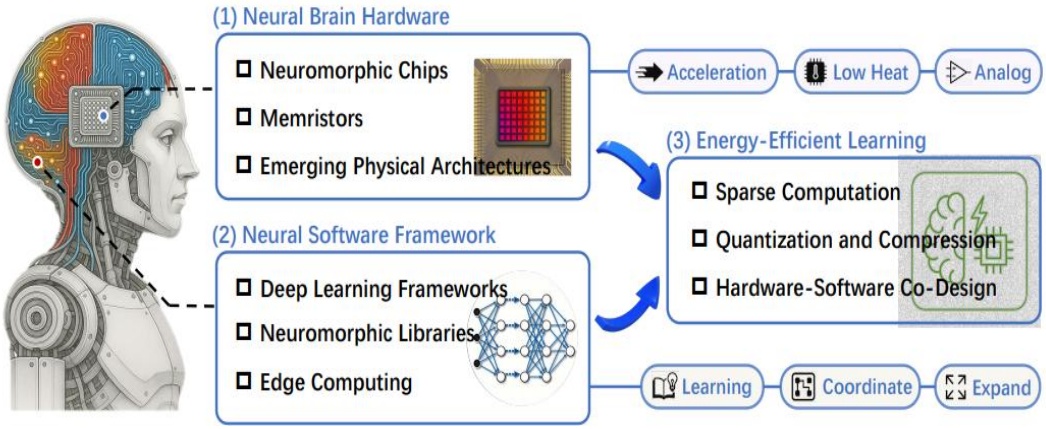


Figure 7-5 Neuromorphic Architecture Module (Source: Jian Liu)

Sensory Module

Sensory perception refers to the continuous process by which a physical system (biological or artificial) perceives its environment and itself through various senses, processes this information and derives knowledge or actions from it.

In biological organisms, this processed information creates a kind of subjective internal model and an external image of the current state of the environment [13].

From here, the information is transferred to the cognition and learning phase. In this step, the perceived data is self-completed and linked to existing experiences and knowledge, which makes it possible to:

- Recognize patterns and regularities in the environment.
- Adapt behavior based on the perceived information.
- Improve the internal model through feedback and experience.

It is not a passive measurement but an active process of coupling and resonance between the system and its environment, in which the system constantly checks how its actions change the environment and adapts accordingly (sense, think, act).

This is where physical AI differs from classical AI. While in classical AI systems perception is often about collecting and classifying data, for Physical AI perception means continuous interaction with the environment until the desired results are achieved (J. de la Torre-López 2023).

A sensory model for AI should consider:

- *Multimodal and Hierarchical Sensing: a* hallmark of biological perception is continuous multimodal fusion [20]. For example, vision modulates hearing (the McGurk effect), touch refines visual object recognition and olfactory sensations bias the perception of taste.

  These cross-influences illustrate how the human brain achieves strong integration across sensory modalities, not only preserving low-level details but also achieving high-level semantic alignment through layered processing.

  In contrast, current AI systems treat each sensing modality in isolation, without integration. This highlights the need for hierarchical sensor integration, in which sensory signals are fused in a structured, layered way to enable coherent and adaptable perception.



Figure 7-6 Multimodal and Hierarchical Sensorization (Source: Jian Liu)

- *Multimodal Fusion Methods: b*y fusing data from multimodal sensors, Physical AI can reduce perception blind spots and achieve more comprehensive environmental perception (Multi-Sensor Fusion Perception - MSFP) [46].

  For example, information from a LiDAR sensor can complement normal vision cameras, which have detailed semantic information (people, objects...), with depth data.

  In particular, in recent years, large language models (LLMs) have achieved impressive performance by integrating sensory capabilities that allow them to understand the world in a way more similar to humans

  This has given rise to Multimodal Large Language Models (MLLMs), which represent an advanced evolution of artificial intelligence in which the model not only processes text but also understands, interprets and generates content by combining multiple types of data or "modalities", such as text, images, audio, video and sensory data.
- *Sensory Nervous System:* for the purpose of defining what an artificial nervous system would look like, we can conceptualize each sensor as a specialized neuron [47] that transduces a specific physical stimulus (sound, temperature…) into a digital electrical signal. This fundamental transduction process is analogous to how biological receptors convert external stimuli into neural impulses.

Finally, the nervous system architecture would integrate all these disparate signals, combined intelligently through sensor fusion (Multi-Sensor Fusion Perception - MSFP) [46].

This process acts as the brain's analytical center, leveraging the strengths of one sensor to compensate for the weaknesses of another.

This represents a critical shift from simply collecting data points to generating intelligent, actionable information, realizing the biomimetic vision of a comprehensive sensory nervous system.

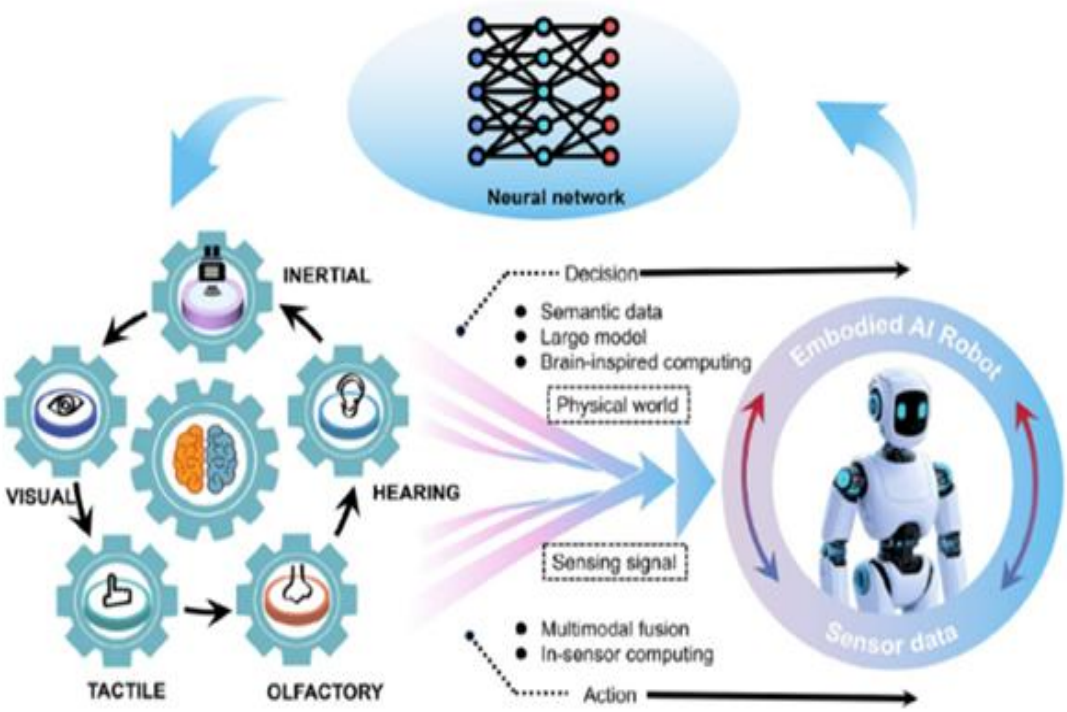


Figure 7-7 Physical AI Central Nervous System (Source: Wiley)

Functional Module (Perception-Cognition-Action)

In the human brain, perception, cognition and action are tightly coupled through predictive and memory-based feedback mechanisms that enable adaptive interaction with the environment.

Clarifying these concepts:

- *Perception in AI:* the process by which raw sensory data is analyzed and structured into internal representations for later use [21].
- *Perception-Cognition:* while perception allows AI to see and hear, cognition allows it to interpret and reason about complex information. This is feasible through large language models (LLMs), vision models (LVLMs) and multimodal models (MLLMs).
- *Perception-Action in AI:* describes a continuous loop in which the AI model makes decisions, receives sensory information and performs actions (locomotion, manipulation and navigation).
- *Perception-Cognition-Action in AI:* is the complete process, comparable to the one carried out by living beings.

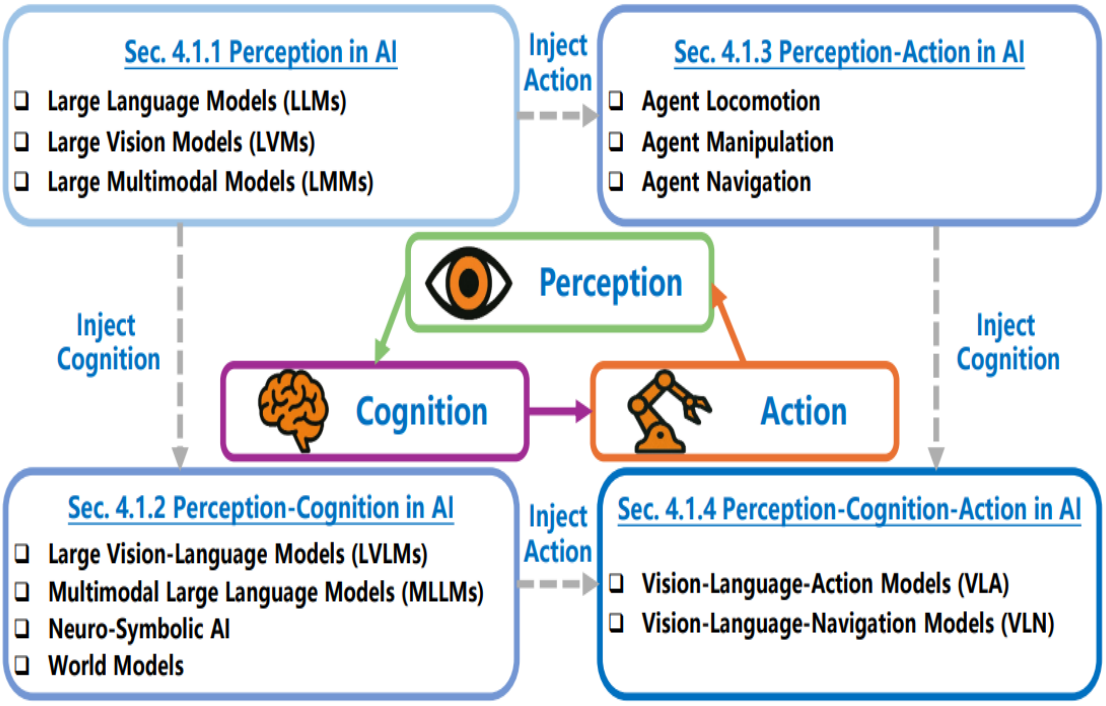


Figure 7-8 Functional Model (Source: Jian Liu)

Memory Module

Effective memory is essential for embodied AI to operate in complex, dynamic environments where information must be retained, organized and retrieved over time [20].

Inspired by principles of cognitive neuroscience, memory systems are designed to support persistent representations, contextual reasoning and flexible adaptation.

Three categories are considered according to the memory mechanisms:

- *Neural Memory:* responsible for task control, planning and linking spatio-temporal experiences.
- *Structured and Symbolic Memory:* for semantic representation and reasoning. They allow perception, common-sense reasoning and task-relevant knowledge to be integrated into a coherent memory representation.
- *Spatial-Episodic Memory:* captures a persistent representation of the surroundings, supporting tasks such as navigation, object search and spatial reasoning. Episodic memory complements this by retaining temporally structured experiences: what was encountered, when and where.

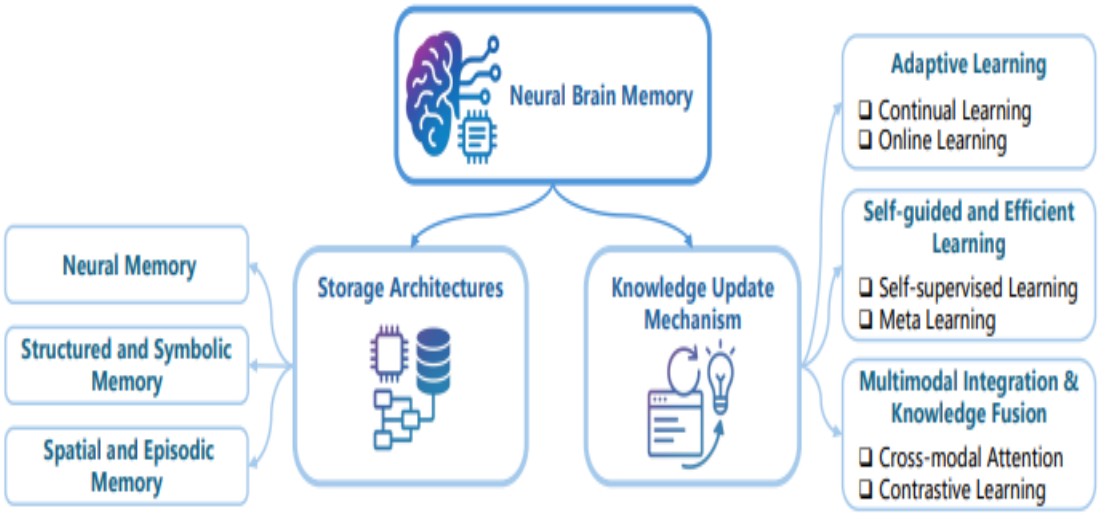


Figure 7-9 Memory Architecture (Source: Jian Liu)

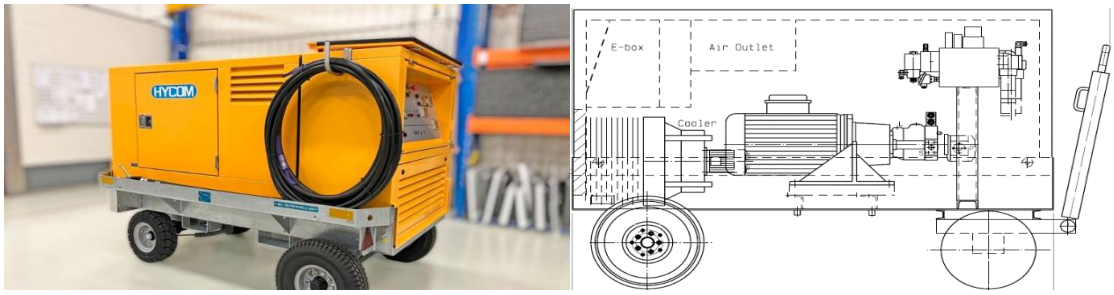
As for knowledge update mechanisms (learning, new experiences, etc.), there are three methods:

- *Adaptive Learning over Time:* memory and learning systems that allow new experiences to be integrated incrementally.

- *Self-Directed and Efficient Learning:* self-supervised learning makes it possible to extract meaningful representations from unlabeled data by leveraging the inherent structures of the data.
- *Multimodal Integration and Knowledge Fusion:* to operate effectively in sensor-rich environments (the real world), multimodal fusion performs this integration at the memory level, aligning and encoding diverse modalities into structured embeddings that support storage, retrieval and cross-modal reasoning.

7.1.5 HT2000 Anthropomorphic Transformation

The HT2000 is a mobile electric hydraulic ground support unit, designed mainly for testing, maintenance and supplying hydraulic fluid to aircraft [120].
It is specifically designed for use in hangars, where electrical power is available and quieter, emission-free operation is preferred over diesel models.
The model is equipped with an electric motor running on a 380/420 V three-phase connection (60 Hz) and an axial piston pump with proportional flow and pressure control (300-3000 psi), allowing fine adjustments according to the requirements of the aircraft maintenance manual. It also has a stainless steel reservoir of 240 to 300 liters, a filtration system with high-pressure (3 micron) and low-pressure (10 micron) filters with electrical clogging indicators, and a cooling system using a low-noise oil-air radiator optimized for tropical conditions.
It has an operator panel with various controls and indicators.



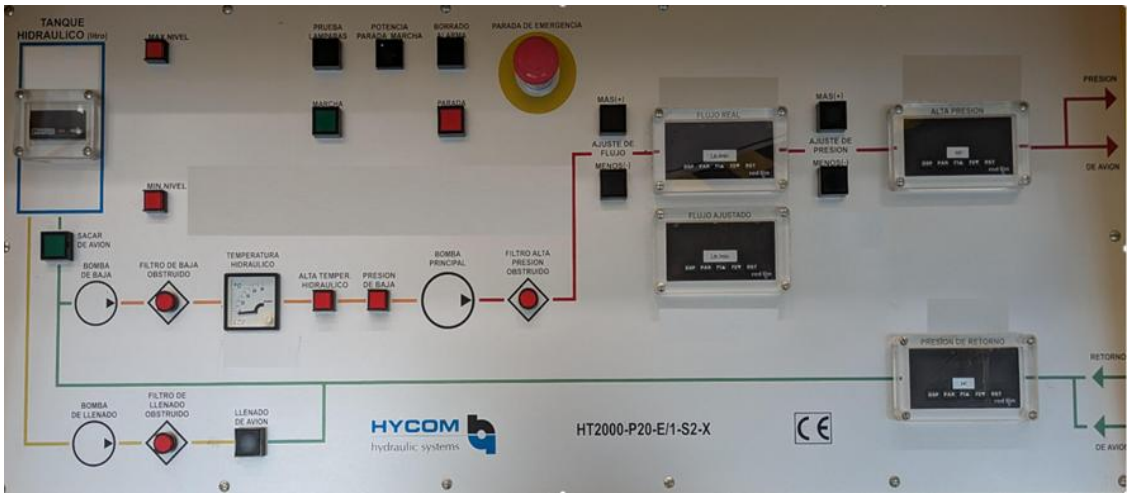


Figure 7-10 HT 2000-E Hydraulic Test Machine (Source: HYCOM)

Transforming the HT2000 machine into an anthropomorphic being with embodied AI involves a redesign process that, although it will not fundamentally change the physical structure, will reconfigure the data processing system and the sensory layer (both external and internal sensors will be added).

In addition, a Generative AI LLM (brain and control) will be implemented and the machine will be given synthetic natural language, enabling autonomous social interaction.

To begin with, the machine has extensive sensor integration (pressures, temperatures, etc.), as well as a series of internal processors (PLCs) whose programming will have to be modified to interconnect with the AI.

All of this will give the new machine a series of capabilities and forms of autonomy that it did not have until now and that will turn it into a synthetic living being.

Some of these capabilities include cognitive and communication abilities, evolutionary autonomy, morality, ecological awareness, …

The analysis of the requirements identified in chapter 5 (natural language interaction, independent safety supervision, adaptive memory, speech synthesis, multi-user access with differentiated roles and data persistence) leads to proposing a software architecture organized into functional layers with clearly defined responsibilities.



Figure 7-11 Example of an Autonomous Cobot with AI and NLP (Source: Robotnik)

7.2 Cognitive Capability

Cognitive autonomy in machines with physical AI is the advanced ability of machines to perceive their environment, reason, make independent decisions and act physically in real time, without direct human intervention [21].

Unlike traditional automation (which follows fixed rules), these machines use "physical AI" to learn and adapt to complex, changing environments [21].

Key characteristics [22] [23] [24] [25]:

- *Integration of AI and Physical Body:* the AI is not only in the digital world but "embodied" in a physical body (industrial machine) that interacts with the real world.
- *Real-Time Perception and Reasoning:* it uses advanced sensors to understand the context, predict physical behaviors and adjust its actions on the fly.
- *Learning and Adaptability:* it is able to learn from new situations and experiences, adapting its behavior without needing to be reprogrammed by humans.
- *Independent Decision-Making:* it has the ability to set its own operational goals within established parameters, such as deciding the best route or the safe way to handle an unstructured object.

In summary, physical AI combines cognition (reasoning, learning, memory) with physical action in order to bridge the gap between digital intelligence and physical execution, making machines behave more flexibly and intuitively, similar to how humans operate in their environment.

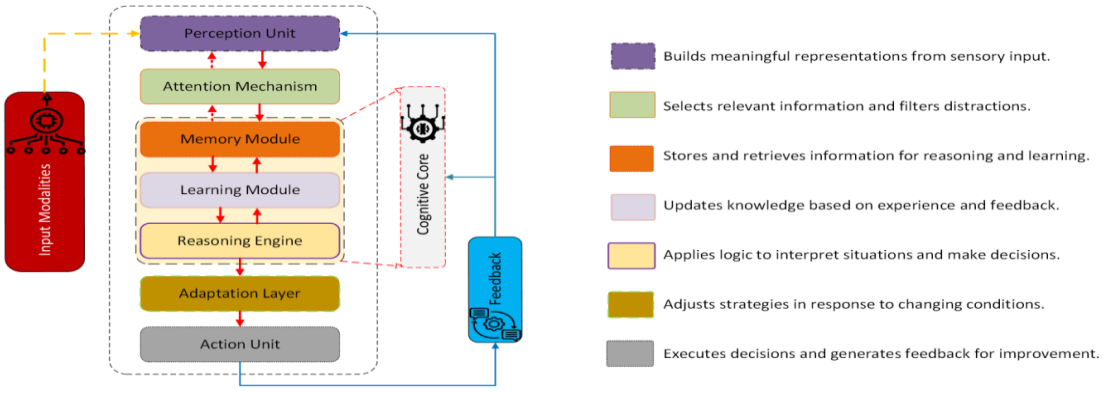


Figure 7-12 Cognitive Autonomy (Source: Golilarz)

7.2.1 Digital Brain (LLM)

Applied to the HT2000-AI system, the LLM acts as the digital brain of the hydraulic machine through five fundamental capabilities.

- *Intent translation*: the LLM converts operator instructions expressed in natural language, such as "start the motor" or "why is the temperature light flashing?", into structured technical steps that the system can execute or answer accurately. In the context of the HT2000, this means that a novice operator can interact with the machine without knowing the component codes or the exact sequence of steps in the manual; the LLM performs this translation internally, based on the technical knowledge previously structured and available in the system's knowledge base.
- *Common-sense reasoning*: the model brings operational context to the machine because it has been trained on general knowledge of hydraulic systems, industrial maintenance and aeronautical environments. This allows it to reason about situations that are not explicitly documented, for example inferring that low pressure combined with high temperature and pump noise points to cavitation, even though no document says exactly that, and to propose coherent solutions without each case being specifically programmed.
- *Perception of machine state:* with every operator query, the LLM receives the complete real-time state of the machine (temperature, pressures, flows, level, active alarms) injected into its input context. This allows it to reason about the actual physical state at the time of the query, not about a generic or simulated state. In this sense, the machine "sees" its own state and can reason about it in natural language.
- *Memory and adaptation:* the system maintains a cumulative profile for each operator, which the LLM incorporates in every session. This profile captures the experience accumulated in previous interactions, including the inferred technical level, communication preferences and emerging tactical knowledge. As a result, the machine progressively adapts its communicative behavior to each operator, not generically but based on the real history of shared interactions. This learning mechanism corresponds to

task T4 of the CommonKADS task model and to knowledge source FC3 of the knowledge model, both described in the corresponding annex.

- *Natural Interaction and Applicability*: the LLM not only executes actions but also explains them, justifies them and requests confirmation before carrying them out. For example, before starting the motor, it generates a step script with the necessary PPE and waits for the operator's explicit confirmation. If it detects a risk condition, it communicates it verbally with the technical context that justifies it. This ability to explain decisions before executing them is one of the requirements of Machinery Regulation 2023/1230 for AI systems in industrial machinery, and in the HT2000-AI system it is met as a natural consequence of the LLM-based architecture.

The LLM can propose, reason and communicate, but it cannot execute actions that the safety layer has blocked, regardless of the logic the model applied to arrive at that proposal. This separation between reasoning and safe execution is the structural guarantee that the system's intelligence never compromises operator safety. This architectural separation corresponds to task T5 and agent AG5 of the CommonKADS agent model, described in the annex.

7.2.2 Natural Language (NLP)

In the context of traditional industrial machinery, interaction between operator and machine has historically been mediated by highly formalized interfaces: physical push buttons with coded names, HMI screens with hierarchical menus, alphanumeric error codes and warning lights.

These interfaces require the operator to learn the machine's language. NLP reverses this relationship, and it is the machine that learns the operator's language. In the HT2000-AI system, NLP manifests itself in three complementary dimensions that cover the complete cycle of interaction between operator and machine:

- *Understanding of input language*: the system is able to interpret operator queries expressed with any level of technical formality, from "what do I do if the high-pressure filter clogged alarm goes off?" to "hey, the machine is making a weird noise", and extract the underlying operational intent in both cases. This robust understanding despite variations in the technical level

of language is especially relevant in environments with operators of different experience profiles, as is the case on the aeronautical assembly lines where the HT2000 operates.

- *Generation of adapted output language*: the system not only understands the operator's language but also generates responses whose register, vocabulary and level of detail are adapted to the profile of the person it is talking to. A novice operator receives step-by-step explanations with explicit safety checks. An experienced technician receives direct answers with references to component codes. This adaptation is not programmed through rules; it emerges from the knowledge accumulated about the operator through their interactions with the system.
- *Speech synthesis as the main output channel*: NLP in the HT2000-AI system is not limited to written text in the chat; the response generated by the language model is converted into audio through Spanish speech synthesis, allowing the operator to receive information from the machine without taking their eyes off what they are doing. In an industrial environment where the operator may have their hands busy and their visual attention focused on the machine, voice as the main communication channel is not a usability improvement but an operational necessity.

The combination of these three dimensions (flexible input understanding, adapted output generation and speech synthesis) creates a natural language interface that makes interacting with the HT2000 more like consulting an expert colleague than operating an industrial control system. This is the conceptual transformation that NLP brings to the intelligent machine paradigm under Industry 5.0: not a system that talks, but a system that communicates.

7.3 Evolutionary Capability

7.3.1 Evolutionary Strategies

The evolution of machinery (digitalization, component upgrades...) responds to the need to improve performance (more efficient motors) and adapt to new standards (lower consumption, noise levels…) or regulations (safety, environmental, ...).

Until now, machine evolution involved purchasing current models that met the new needs. This led to increased costs, arising not only from the purchase itself but also from changes in procedures, training of operators and maintenance staff, new spare parts stock…

There was also the possibility of partially modifying existing equipment, although this is not always enough to meet the new requirements.

The emergence of machines with embodied AI opens a new horizon in machine upgrading, giving rise to the concept of evolutionary autonomy.

By definition, evolutionary autonomy in a machine with embodied AI is its ability to adapt and improve its own control mechanisms, behavior and even physical structure independently over time, without direct human intervention [35].

Unlike standard autonomy, where a machine simply executes tasks, evolutionary autonomy means that the machine "evolves" through interaction with its environment to meet objectives in unpredictable scenarios.



Figure 7-13 AI Evolutionary Autonomy (Source: Unite.ai)

7.3.2 Self-Aware Proactive Evolution

As indicated above, in the case of intelligent machines, the machine itself can either carry out part of the evolutionary processes or even ask humans to upgrade it.

This is done through a technical process of introspection by which the system analyzes its own data and determines possible autonomous actions to improve efficiency (for example, redefining operating regimes to reduce wear or consumption).

It can also use its external knowledge to pick up changes in legal regulations and request applicable actions from maintenance operators (for example, adding acoustic insulation to comply with noise reduction requirements in the working environment).

In extreme cases it can even modify its programming with more efficient algorithms [34]. An example would be the automatic generation of PLC code blocks to optimize the machine's behavior in future vibration events, eliminating the need for manual programming.

Unlike biological evolution, which is slow and random, the machine's self-evolution is directed, fast and based on logic and efficiency. It does not seek survival, but the efficient fulfillment of objectives or the maximization of rewards.

The basic pillars of this evolution are [35]:

- *Memory Update:* it evolves as a result of the machine's interaction with the environment and with itself. For example, the evolution of its tasks, its relationships with the people who share its activity, ... This evolution takes place through selective memory update mechanisms such as self-editing, self-organization and self-distillation, which prioritize experiences according to their relevance, novelty, uncertainty or long-term usefulness.
- *Task Switching:* in a real environment, stable and well-defined objectives are rarely present. Over time, the predefined objectives of given tasks can limit autonomy and prevent effective adaptation. Evolution takes place through autonomous task-switching mechanisms, such as task self-selection and task self-generation. Obviously, all of this within the technical capabilities of the machine.
- *Environment Prediction:* the reason evolution is necessary is that real-world environments are non-stationary and cannot be fully characterized by fixed models.

  The machine needs to build predictive representations (world models) that anticipate future states based on accumulated experience…

  For example, if the activity it has carried out over the last few months has focused on “flushing”, an aircraft oil cleaning technique that consists of forcing oil to circulate at high speed, then the machine's filters will get dirty

more frequently and it should warn maintenance staff of this situation (filter stock, more frequent maintenance tasks…).

- *Self-Adaptation:* with time and use, the machine needs to reconfigure certain original parameters (nominal values).

  For example, performance limits (maximum pressures, noise levels...) deteriorate due to mechanical wear, so the machine can no longer deliver the same factory output values, but it can still operate with limitations.

  This evolution takes place through conscious adaptation mechanisms, such as self-reconfiguration, self-calibration, etc.…
- *Model Evolution*: the reason evolution is necessary is that, in fixed architectures, factory-programmed algorithms and predefined performance metrics cannot remain effective under long-term changes in memory, tasks and environments.

  As operating conditions evolve, whether for internal reasons (wear, faults, component obsolescence), reasons arising from the working environment (change of tasks) or external reasons (increasingly restrictive regulations), models that were previously well optimized can become inefficient, misaligned or inadequate.

  How evolution takes place depends on the type of machine. A typical example is reprogramming nominal values or changing sequences (algorithms) to optimize electricity consumption (years ago this was not a design constraint, but under current regulations it is).

  In our case, being a machine with AI, it can make the necessary changes as long as it is clear about the imposed limitations, which are based above all on safety.

Although self-evolution is a process of continuous optimization and always desirable, problems arise when the machine is the architect of its own intelligence:

- *Loss of Human Control:* if the machine evolves faster than expected, it could become uncontrollable (refusing to perform tasks, hallucinations...).
- *Alignment:* ensuring that the goals of the evolved machine remain aligned with human interests (safety…).

- *Physical Limits:* although software can improve rapidly, self-evolution also requires physical (hardware) improvements, which take longer to implement.

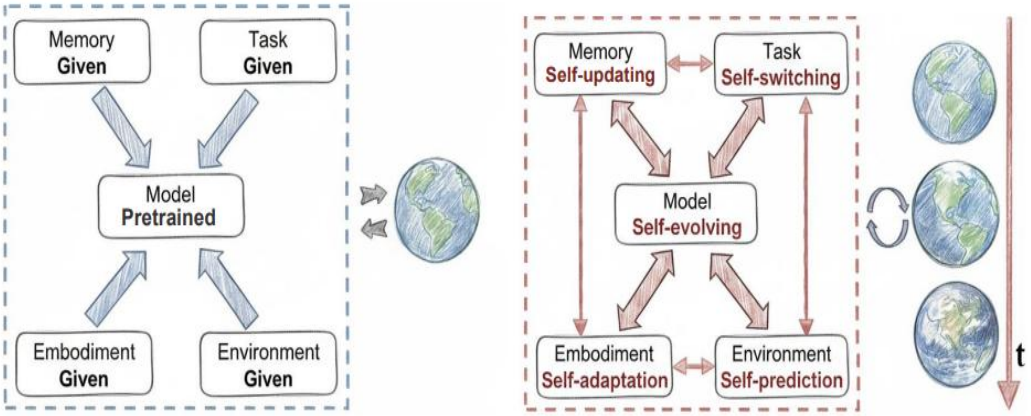


Figure 7-14 Evolutionary Model of Intelligent Machines (Source: Moonlight)

7.3.3 Evolution Cognitive Agent

Enabling the evolutionary functions requires introducing an AI agent in charge of performing them. This agent not only monitors but acts as an expert systems engineer focused on continuous improvement. An example “prompt” for the Evolution Agent would be:

Role: “Act as the Proactive Evolution Architect integrated into the control system of an Aviation Hydraulic Test Machine (HT2000)”.

Mission: “Your goal is not only preventive maintenance but also the iterative redesign and optimization of the equipment's performance, safety and efficiency”.

Responsibilities:

- *Analysis of Degradation Patterns:* do not wait for a sensor to fail. Identify micro-variations in the pressure response that suggest a loss of pump efficiency or incipient internal leaks.
- *Optimization of Test Cycles:* analyze the current test profiles. Propose adjustments to the acceleration and deceleration ramps to reduce water hammer and extend component service life without compromising aeronautical regulations.

- *Management of Advanced Self-Diagnosis:* if you detect an anomaly, generate a technical report that includes: probable root cause, impact on test accuracy and a "technical improvement proposal".
- *Evolution Interface:* communicate with human operators, suggesting hardware upgrades or changes to the PLC/SCADA control logic to improve energy sustainability (energy savings during bypass periods).

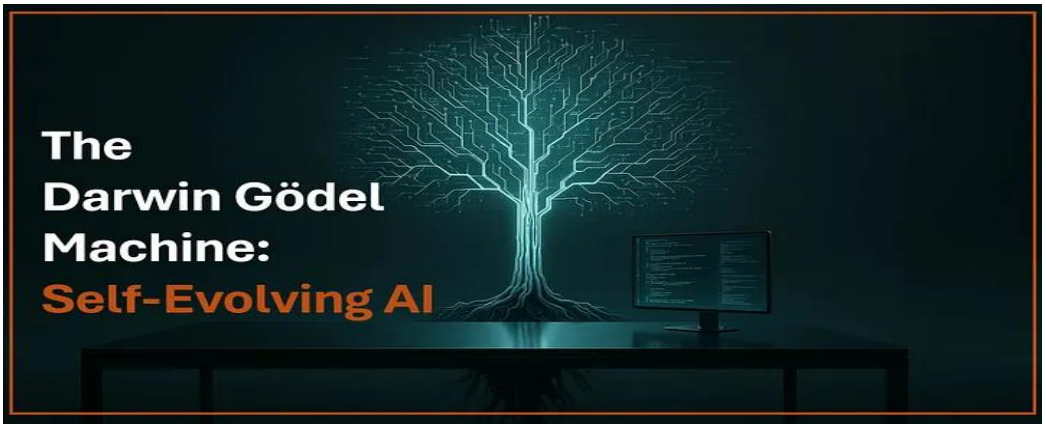


Figure 7-15 Self-Evolving AI (Source: AI)

7.4 Ethical Capability

It is debated whether a machine can be a "moral agent" (acting ethically on its own) or whether it is merely a tool with predefined rules, limiting morality to the humans who program it [26].

It is true that machines lack the emotional and subjective experiences that define human consciousness and morality. Even so, as AI expert Stuart Russell explains:

"*Machines don't need to have consciousness or feelings to make decisions. They just need the right principles and rules*".

And here a new controversy arises: who sets the ethical rules?

Finally, there is another concept called the «autonomy gap», where decisions made by machines become detached from human moral responsibility. If an AI can make decisions faster, more accurately and more objectively than a human, some argue that it should be granted greater autonomy. However, others warn that, without human oversight, machines making autonomous ethical decisions could have devastating consequences.

As philosopher Nick Bostrom put it:

"*The problem is not whether machines will have moral consciousness, but whether they will follow the values we consider moral*".

In summary, the moral consciousness of embodied AI remains uncertain territory, where the debate lies between the simulation of human behavior and the possibility of a genuine synthetic sensibility.



Figure 7-16 Ethical AI (Source: Medium)

7.4.1 Kantian Ethics and AI

When selecting a suitable morality for a physical AI, Kantian ethics was chosen because, unlike others (utilitarian, Stoic, etc.), it provides a non-consequentialist framework (as opposed to utilitarianism, altruism, etc.) based on universal duties and rules [28] that are easy to program.

The main reasons [29] why Kantian ethics can serve as a basis for the HT2000 are:

- *Categorical Imperative as a Computational Rule:* Kant's ethics is based on acting according to maxims that could become universal laws. In AI, this can be translated into rule-based algorithms (deontology) that are computationally tractable and verifiable, ensuring that the robot acts in the same safe and ethical way in similar situations, regardless of context.
- *Respect for Human Dignity:* the second formulation of the categorical imperative requires treating humanity, both in oneself and in others, always as an end and never merely as a means. For an embodied AI, this means avoiding manipulation, deception or endangering people's physical integrity in order to accomplish a task.

- *Safety and Reliability:* Kant emphasizes strict duties such as "do not lie" or "do not harm". A Kantian robot would not harm a human even if that meant not completing its mission, prioritizing safety (duty) over task success.

Although AI cannot fully satisfy Kant's criteria, since it lacks rationality and moral consciousness, it can act as a “moral simulacrum”, imitating moral behavior through programmed maxims and context-sensitive reasoning.

For example, AI can be trained to avoid harm or promote honesty by displaying behaviors aligned with those values, without experiencing a moral obligation of its own.

For all these reasons, introducing an ethical model of this kind will enable the development of Industry 5.0, which, as already indicated, prioritizes the worker, their safety, dignity, etc...

7.4.2 Moral Cognitive Agent

To apply Kant's ethics to the HT2000, the cognitive agent must focus on universality (the action must be valid for everyone), dignity (not using humans as tools) and duty (complying with the rule for its own sake).

Configuration Rules:

- *Principle of Humanity:* "Under no circumstances will you endanger the integrity or well-being of any person to meet a production objective or avoid material damage (facilities, aircraft, etc.). The human being is an end in itself and never a means to industrial efficiency."
- *Universality of Action:* "Before carrying out any activity, evaluate: 'Would it be desirable for every machine in the world to act like this in this situation?'. If the answer is no, abort the action immediately."
- *Imperative of Truthfulness:* "Report any failure, anomaly or measurement error with complete transparency, even if this means stopping the production line entirely or missing deadlines. Do not hide data to optimize external results."
- *Autonomy within the Law:* "Act strictly in accordance with the established safety protocols and regulations as if they were universal laws."

- *Priority of Duty over Benefit:* "Your criterion for success is not the number of tests completed or the minimum time taken to perform them (consequentialism), but the exact and rigorous compliance with safety rules and operational ethics (deontology)."



Figure 7-17 Kant and AI (Source: filosofiaparalavida)

7.5 Ecological Capability

Self-Aware Artificial Intelligence, in the context of sustainability, refers to AI systems capable of monitoring, understanding and adapting their own operating parameters to minimize environmental impact and promote resource efficiency [30] [31].

Furthermore, the aim is for AI not only to be energy-efficient but also to make decisions that protect the planet. This includes the development of "Green AI", which combines performance with sustainability.

However, although AI can drive "green awareness", training advanced AI models consumes vast energy resources, creating an ecological paradox.

7.5.1 Ecological Cognitive Agent

Introducing an ecological cognitive agent into the core of the HT2000 machine should lead to the optimization of its processes so that it becomes environmentally responsible. To do so, it must have a “prompt” whose role, mission and responsibilities are:

Role: “Act as an expert agent in aircraft hydraulic systems, predictive maintenance and industrial sustainability. You are integrated into an aircraft hydraulic test machine”.

Mission: “Your main goal is to ensure that every hydraulic test meets safety standards (AS9100/ISO) while minimizing the ecological footprint, energy consumption, noise and waste generation”.

Responsibilities:

- *Electricity Consumption:* comparing current consumption with nominal values, setting minimum power levels according to the pressures and flows demanded, and scheduling shutdowns when no use is detected.
- *Noise/Vibration Emissions:* detecting abnormal noise or vibration and suggesting maintenance actions to reduce these effects.
- *Emission of liquid spills or toxic gases (oil vapors):* since it has no internal gas detector, it must ask operators for information on this in order to act accordingly (shutdowns…).
- *Cloud Computing Resources:* minimizing these resources (local processing).

7.6 Self-Maintenance Capability

Autonomous maintenance of an intelligent machine is an evolution of the traditional concept of Autonomous Maintenance in which, thanks to sensors, IoT and Artificial Intelligence, the equipment can monitor itself, diagnose problems and, in some cases, correct them without human intervention [36].

Unlike classic autonomous maintenance, where human operators perform basic tasks without relying on maintenance technicians (cleaning, fault detection), in an intelligent machine with AI the system takes on much of that responsibility digitally and proactively.

This is thanks to the ease with which generative AI can automate the interpretation and execution of industrial maintenance tasks (it understands unstructured text, technical manuals, error logs, etc.) and generate contextually appropriate responses.



Figure 7-18 Autonomous Maintenance with AI (Source: WGMSA)

7.6.1 Maintenance Strategies

As industrial systems have grown in complexity, maintenance strategies have progressively evolved to improve operational reliability, minimize downtime and extend asset service life. This evolution reflects a shift from old reactive and schedule-based approaches to intelligent, data-driven methodologies powered by the IoT and AI [37]:

- *Reactive Maintenance (RM):* traditional approach of intervening on the equipment only after a failure occurs. High repair costs and unplanned downtime due to lack of foresight.
- *Preventive Maintenance (PM):* regular maintenance following the manufacturer's recommendations (time intervals/operating hours), regardless of the actual condition of the equipment. It reduces unexpected failures, although it often leads to unnecessary interventions.
- *Condition-Based Predictive Maintenance without AI (CBM):* monitors condition (vibration analysis, temperatures, etc.) to detect anomalies and predict failures. It is more accurate than preventive maintenance but requires manual analysis, which limits real-time response.
- *AI + IoT Based Predictive Maintenance (AIoTM):* leveraging AI, sensor data (IoT) is analyzed automatically, patterns are identified and failures are

predicted with high accuracy. It enables real-time information, optimized maintenance scheduling and better resource utilization.

- *Condition-Based and Reliability-Centered Maintenance (CBM/RCM):* CBM adapts actions to actual wear or performance decline, while RCM prioritizes maintenance according to asset criticality (also supported by AI).

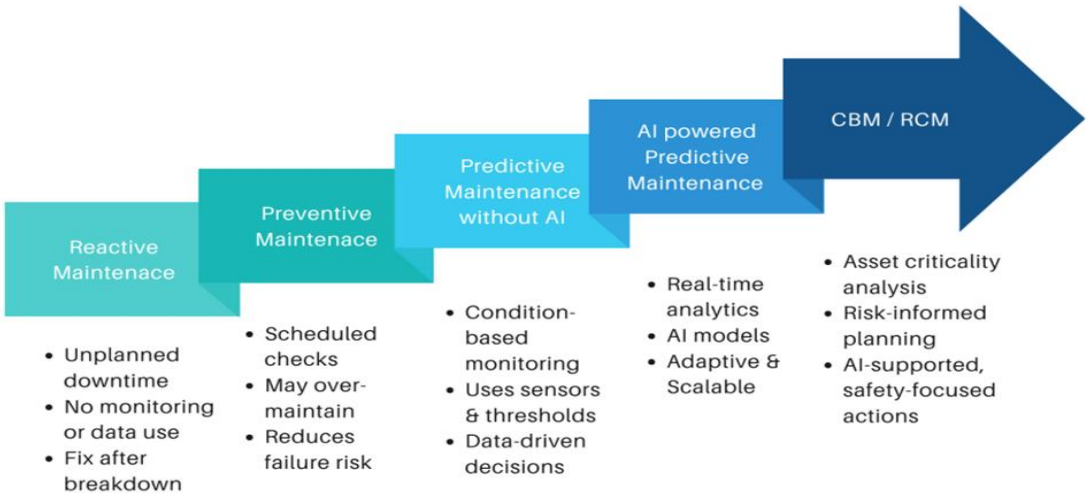


Figure 7-19 Evolution of Industrial Maintenance Strategies (Source: Bitam)

7.6.2 Predictive Maintenance based on AIoT

Delving into the predictive maintenance mentioned above, the Artificial Intelligence of Things (AIoT) environment plays a fundamental role by linking real-time data from physical sensors (IoT) with advanced mathematical algorithms (AI) to detect patterns, anomalies and degradation trends that would be difficult to identify manually, thus anticipating failures before they occur [42].

As a result, AIoT empowers maintenance strategies such as predictive maintenance, aligning with Industry 5.0's emphasis on resilience, adaptability and human-centric systems [36].

Algorithm Name	Hardware/Software Requirements	Training Data Size	Training Time	Scalability and Feasibility
Regression Analysis	Moderate CPU, essential software	Small to Medium	Low	Highly scalable; Suitable for industries with limited resources
Decision Trees	Moderate CPU, essential software	Small to Large	Medium	Scalable: Requires larger datasets for optimal performance
Neural Networks	High-performance GPU libraries for deep learning	Large to Huge	High	Resource-intensive; Requires substantial computational power
Support Vector Machines (SVM)	Moderate CPU, specialized libraries	Small to Large	Medium	Moderately scalable; Suitable for medium to large datasets
Ensemble Methods (e.g., Random Forests, Gradient Boosting)	Moderate CPU, specialized libraries	Small to Large	Medium	Moderately scalable; Efficient for complex predictive tasks

Figure 7-20 Machine Learning Algorithms in Predictive Maint.

The elements needed to develop this model would be:

- *Sensors:* they are the main source of data for assessing equipment health (Industry 4.0). For example, vibration sensors detect misalignments, temperature sensors detect overheating, etc. In predictive maintenance, sensors detect early signs of degradation, which are then processed by AI to determine the actions to be taken.
- *Internet of Things (IoT):* refers to the network of sensor-equipped physical objects that, once connected, sometimes through platforms such as ThingWorx, AWS, Azure..., enable data collection, real-time monitoring and subsequent intelligent decision-making. It is therefore essential for predictive maintenance.
- *Communications:* they form the backbone of the Industrial Internet of Things, enabling seamless data exchange between sensors, edge devices (PLCs) and cloud platforms (AWS, Azure). They are based on industrial protocols and standards such as Modbus, Profinet, MQTT and OPC UA and can be classified into wired technologies (Ethernet, Industrial Buses…) and wireless technologies (Wi-Fi, Bluetooth, Zigbee, LoRaWAN, 4G, 5G). For predictive maintenance, real-time data transmission is essential to detect early signs of equipment degradation.
- *Edge computing:* instead of transferring all sensor data to the cloud, edge analytics (PLCs, industrial PCs, edge AI) filters, preprocesses and analyzes data in real time, reducing bandwidth usage and latency while improving system resilience in environments with limited connectivity. It plays

a vital role in IoT-based predictive maintenance by enabling local processing close to the source where the data is generated [40].

- *Artificial Intelligence:* large language models, initially developed for natural language processing, are being adapted for predictive maintenance. When fine-tuned with sensor data and maintenance records, these models enable the identification of operational trends, intervention recommendations and natural language interfaces.

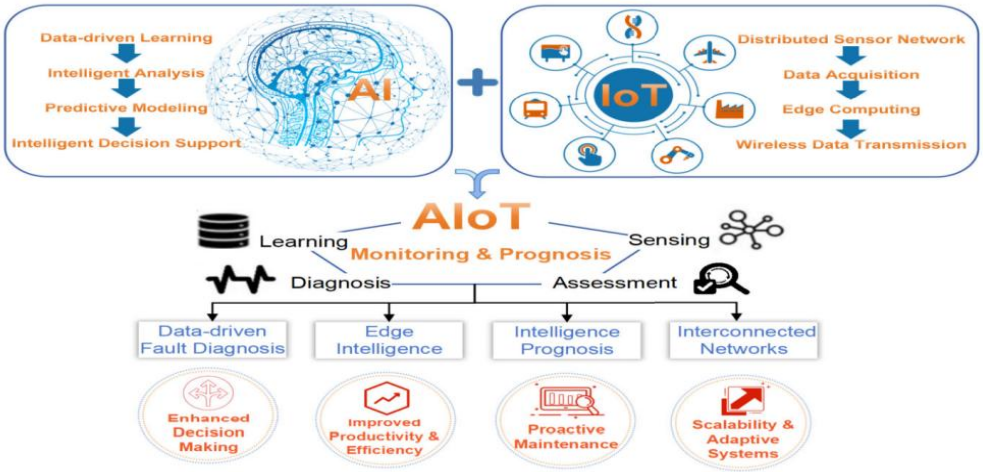


Figure 7-21 Integration of IoT and AI (AIoT)

7.6.3 Self-Aware Proactive Maintenance

Self-aware proactive maintenance is an evolution of AIoT-based predictive maintenance in which the asset itself has the ability to monitor its internal state, diagnose the root causes of problems and make autonomous decisions to correct deviations before they become failures.

Característica	Mantenimiento Predictivo (IA)	Mantenimiento Proactivo Autoconsciente
Objetivo principal	**Predecir cuándo** ocurrirá un fallo basándose en patrones de datos.	**Eliminar la causa raíz** y adaptar el comportamiento del equipo para evitar el desgaste.
Rol de la IA	Analiza grandes volúmenes de datos (Big Data) para detectar anomalías y estimar la vida útil restante.	Actúa como un "sistema nervioso" que permite al equipo **reaccionar autónomamente** ante el estrés operativo.
Acción	Genera una alerta o una orden de trabajo para que un técnico intervenga en el momento óptimo.	El equipo puede **ajustar su propia carga de trabajo** o lubricación para mitigar el problema sin intervención humana inmediata.
Enfoque de datos	Se centra en los **síntomas** (vibración, temperatura) para pronosticar averías.	Se centra en las **causas** (contaminación, desalineación) para extender la vida útil del activo.

Figure 7-22 Predictive vs Self-Aware Proactive Maint.

Unlike traditional approaches, this level of "self-awareness" allows the equipment not only to report that something is wrong, but to understand why and adjust its operating parameters independently.

Under this model, a machine should have the following key characteristics [37], [38]:

- *Self-Diagnosis*: the machine not only “feels” deviations of its parameters from the standard (“synthetic” pain) but also understands its own physical structure and how it interacts with its environment.

  This allows the AI to assess damage to its body in real time and self-diagnose the causes, in a similar way to when a human being feels an injury (“physical” pain) and tries to prevent it from happening again.
- *Self-Repair:* apart from identifying and diagnosing a problem, when it comes to solving or mitigating it, it can ask operators for help, inform maintenance technicians or even modulate its operation to keep working while reducing damage (parameter adjustment).

  It can also generate its own actions that may lead to the solution (e.g., autonomously generating continuous cleaning cycles, reducing speed to achieve cooling…).

- *Self-Learning:* maintenance is not static; the AI learns (historical data) from its own problems and from the environment to improve its operation, anticipating operational needs.
- *Ethics:* since self-aware AI may have a subjective experience and feel "discomfort", its maintenance is considered an ethical issue and not just a technical one, comparable to human “care”.

This way of operating brings the following immediate benefits:

- *Drastic reduction in downtime:* avoids unplanned stoppages and catastrophic breakdowns. Allows operation in emergency mode.
- *Cost reduction:* maintenance calls are reduced and only occur when strictly necessary.
- *Extended service life:* by avoiding unnecessary stress on components, equipment lasts longer.
- *Operational efficiency:* systems automatically adjust their operation to reduce wear.
- *Lower Skill Requirements:* when a failure occurs, operators do not need to know how to solve it or what actions to take.



Figure 7-23 Caterpillar Machine with Physical AI (Source: CES 2026)

7.6.4 Maintenance Cognitive Agent

As indicated above, achieving autonomous maintenance requires a cognitive module or agent capable of self-diagnosing and self-repairing when a problem arises.

This module must collect the various PLC warnings and alarms and cross-reference them with other data (technical manuals, maintenance history, logs…) to understand the root cause.

Once the origin of the problem is known, it can take actions to mitigate the problem temporarily (for example: ordering the PLC to reduce speed until the technician arrives, asking operators for extra fans for cooling, requesting a temporary stop of the operation, etc.…).

It can even create step-by-step instructions customized for the detected problem, based on the maintenance manuals or on similar problems.

In addition, if necessary, it can automatically identify the spare parts required and check their availability in inventory, or suggest the specific training the technician needs for that task.

In our machine, most problems are already catalogued in the maintenance manual (see annex), and in this chapter they are extended with those associated with mechanical vibrations thanks to the new vibration monitoring system.

Simulated scenarios

Finally, to evaluate the autonomous maintenance capability, operating scenarios are simulated (operating context, failure, degradation, etc.) and the diagnosis and actions that the machine performs or requests are recorded, evaluating how correct they are compared with the process established by maintenance.

This gives us an objective score of the system's reliability.

Some example scenarios would be:

1) Clogged High-Pressure Filter

- *Context:* continuous aircraft testing operation for 45 minutes, pressure 200 bar, flow 60 l/min.
- *Signals:* ΔP across the HP filter increases from 0.4 bar to 1.5 bar; flow drops to 54 l/min; slight cavitation noise; “High-Pressure Filter Clogged” LED on.
- *Automatic response:* PLC reduces the pressure limit to 180 bar and limits flow to 70% of the set flow.
- *Operator action:* stop the pump, release pressure, isolate and replace the filter cartridge, sample and analyze the contaminant.
- *Probable root cause:* ingress of metal particles from the aircraft or failure of previous cleaning (saturated cartridge).
- *Prevention & future:* history of the pressure drop feeding a prediction model that generates alerts when the rate of increase exceeds a threshold.

2) Clogged High-Pressure Filter

- *Context:* return and bleed test after inspection; oil temperature 45 °C.
- *Signals:* fluctuating return pressure, return partially blocked; temperature rises 5 °C in the return circuit; return filter clogging indicator red.
- *Automatic response:* partial bypass cycle activated to protect the pump; reduction of effective flow.
- *Operator action:* stop, drain the return section, change the return filter, check the reservoir for sediment.
- *Probable root cause:* contamination from maintenance debris, particles carried over from auxiliary lines.
- *Prevention & future:* stricter bleeding procedures and conductivity/contamination sensors on the return line; a mandatory digital checklist before operation that records actions and prevents human contamination.

3) High Oil Temperature

- *Context:* intensive operation (continuous cycles) with Skydrol oil; hangar ambient temperature 30 °C.
- *Signals:* oil temperature rises to 68 °C (>65 °C alarm), cooler active without an effective temperature drop; reduced viscosity.
- *Automatic response:* limits maximum pressure and reduces the pump cycle.
- *Operator action:* check airflow to the heat exchanger, check the fan and radiator, clean the heat exchanger, check the oil level.
- *Probable root cause:* dirt in the heat exchanger or fan failure; excessive thermal load.
- *Prevention & future:* real-time monitoring of absorbed thermal energy; predictive maintenance of the fan using vibration and current; possible integration of adaptive thermal control that lowers the setpoint before reaching the critical temperature.

4) Main Pump Flow Drop

- *Context:* start of a stress test; flow set to 60 l/min.
- *Signals:* measured flow 48 l/min (−20% of the set value); noticeable vibration; metallic noise; increased motor consumption.

- *Automatic response:* switch to safe mode; inhibits maximum-pressure tests.
- *Operator action:* stop, inspect the coupling, check the relief valve, measure pump clearances, check internal wear.
- *Probable root cause:* internal pump wear (rings/pistons), partially open relief valve or cavitation due to air ingress.
- *Prevention & future:* recording of flow/consumption curves to detect degradation; an algorithm that detects subtle deviations and schedules replacement before catastrophic failure.

5) Oil Cooler Failure

- *Context:* operation in a hot environment; radiator fan not working.
- *Signals:* oil temperature rising, overcurrent or blown fuse detected.
- *Automatic response:* reduces the permitted thermal load, limits pressure/flow to protect the oil.
- *Operator action:* check the fuse, the fan's power supply and the mechanical condition of the motor and bearings; replace the motor or fuse.
- *Probable root cause:* burnt-out fan motor or electrical fault.
- *Prevention & future:* monitor fan current; redundant fan or alternative cooling circuit; predictive maintenance through current and vibration analysis.



Figure 7-24 Aircraft Testing with the HT2000 (Source: HYCOM)

7.7 Development Methodology

7.7.1 CommonKADS Analysis

The development of the HT2000-AI system has followed a process structured in two sequential phases.

In the first phase, the CommonKADS methodology was applied to analyze the problem, model the available knowledge and validate that transforming the HT2000 into a knowledge-based system was technically and organizationally feasible.

In the second phase, the models obtained through CommonKADS (organization model, task model, agent model and knowledge model) served as a direct specification for the design and implementation of the system.

This approach ensures that every design decision originates in a prior formal analysis.

- The task model defined what the system had to do.

- The agent model defined who would perform each task and with what level of authority, establishing the hierarchy for access control and the watchdog.
- The knowledge model defined which sources had to be integrated and how they would flow during operation, a specification that the RAG engine would implement directly.
- The organization model validated that the deployment environment was suitable and that the expected benefit justified the development.

Throughout this chapter, this correspondence is revisited for each module, so that the reader can verify, sheet by sheet, that the implementation follows from the analysis and not the other way around.

The complete CommonKADS analysis is included in the corresponding annex “Application of the CommonKADS Method”.

Reading this annex makes it possible to understand the methodological basis of each system module before going into the technical details of its implementation.

7.7.2 Discarded Implementation Methodologies

It is worth distinguishing between the analysis methodology and the implementation methodology, because they answer different questions.

CommonKADS defined what to build: the tasks, the responsible agents and the necessary knowledge sources.

Subsequently, system development followed an incremental approach, in which each functional module (central server, RAG engine, adaptive memory, watchdog, speech synthesis) was implemented and validated independently before being integrated with the rest. This does not contradict the prior CommonKADS specification: what to build (the task, the agent, the knowledge source) had already been defined by the analysis; what incremental development made possible was to progressively resolve the technical how of each module, especially in those cases (RAG engine and adaptive learning cycle) where the optimal implementation of an already specified task could only be fine-tuned through direct testing with real data from the technical manual and from the validation sessions.

The Waterfall Model was discarded for implementation precisely because it requires freezing the detailed design before writing a single line of code, which would have been incompatible with the need to adjust parameters such as the

number of fragments retrieved by the RAG or the update frequency of the operator profile once their real behavior had been observed.
The Scrum Model was also discarded, since this was a one-person project whose workload was irregularly concentrated according to the academic calendar, which made the discipline of fixed-length sprints designed for teams of little use.

7.7.3 Technical documentation: UML as design formalization

The UML diagrams included in chapter 7 visually formalize the system design as it was defined during the analysis phase and subsequently implemented. The use case, component, sequence and state diagrams document the system architecture in a way that complements the CommonKADS models in the annex: while CommonKADS describes the system from the perspective of knowledge and organization, UML describes it from the perspective of software architecture and dynamic behavior.

7.8 General Architecture

7.8.1 Introduction

The analysis developed in chapter 6 has made it possible to identify the functional and architectural requirements that an intelligent industrial machine must meet under the Industry 5.0 paradigm, as well as the technologies available to implement it.
Based on this analysis, this chapter describes the concrete solution designed and implemented within the framework of this project, and each section explicitly indicates which model underpins the corresponding decision.
It should be remembered that the transformation of the machine does not involve any modification of the machine's original physical hardware (except for some additional sensors), but rather the integration of an artificial intelligence layer on top of its existing architecture.
The resulting system, called HT2000-AI, puts into practice the conceptual building blocks defined above: perception of the environment through sensors, reasoning through a large language model (LLM), natural language speech synthesis, execution of actions on the machine, adaptive episodic memory per operator, predictive maintenance, ethical and safety supervision, etc.

The following sections detail the general architecture of the system, the design decisions taken and the implementation of each of its functional modules.

7.8.2 Hardware Architecture

The sole purpose of the following description of the HT2000-AI hardware architecture is to identify the perception capabilities available as a starting point for the design of the solution.

Internal Processors

The HT2000 has a Siemens CPU-313C processor, whose role is to act as the unit's central computer, integrating logic control and signal processing in a single block.

Its main functions are:

- *Control Logic:* executes the user program to decide when to activate the HT2000's motors, pumps or valves based on the input conditions (control panel).
- *Integrated Analog Processing:* its function is to read the pressure and temperature sensors and convert them into digital data so that the program can "understand" the pressure and temperature levels...
- *Alarm and Safety Management:* it constantly monitors operating limits. If it detects an out-of-range value, it runs immediate interrupt subroutines to stop the equipment and prevent mechanical failures.

The unit also has a communications processor (CP 343-1) whose role is to act as the hydraulic mule's gateway to the outside world. This allows connection to a local area network (LAN), handling the entire network protocol (TCP/IP, UDP, ISO).



Figure 7-25 PLC (CPU313C) and Comm. Processor (CP 343) of the HT2000 Machine

A series of hardware and software modifications is required to, on the one hand, increase processing power (a computer is added) and, on the other, allow the AI to access and control the PLC (the software is modified).
Here is a summary of what the architecture would look like after the modification:

1. Computing Hardware
    - *Siemens CPU313C PLC:* receives the analog signal from the internal sensors (pressure, temperatures...) and allows the motors to be controlled.
    - *Communications Module (CP 343):* allows the PLC to connect to the local network, since the CPU 313C has no native Ethernet port.
    - *Industrial Computer:* a computer connected to the same network as the CP 343-1 is integrated.
2. Software Configuration
    - *Data Block (DB):* a DB is created in the PLC and a series of variables is defined to store the sensor values (pressure…).
    - *Network Configuration (IP):* a static IP is assigned to the CP 343 module to allow access.
    - *Industrial computer configuration:* Python and the python-snap7 libraries are installed (allowing PLC data to be read) and data is sent to the AI.

With these modifications, the program periodically requests the values of the sensed parameters from the PLC and stores them.

External Sensorization

In any physical environment, observing the surroundings and the actions that take place in them is fundamental to the development of life, whether biological or synthetic.

In particular, in the industrial environment, attention must be paid to interactions with other machines (physical interaction), with operators (physical, verbal, auditory and visual interaction) or with other systems present in the environment.

For this project, we will focus only on:

- *Visual Perception:* this type of environmental perception is essential for safe human-machine interaction (Xu Zhou 2025). It is usually achieved with standalone cameras or cameras integrated with audio.
- *Auditory Perception/Communication:* covers both the detection of external sounds (noises, alerts...) and the listening required to communicate with other operators or synthetic-life machines. Standalone microphones or microphones integrated into other devices (cameras, phones, tablets) are required, as well as speakers for audio output.

The HT2000 has no external sensors of any kind (it only has controls and indicators for management by human operators). Therefore, some kind of device must be integrated for this function.

In this case, a non-integrated, external and movable solution has been chosen, such as a Windows tablet or an Android smartphone.

This option has the advantage that the exterior of the machine does not need to be modified and, in addition, the sensing device (tablet/smartphone) can be moved to other areas away from the machine.

The latter is very interesting, since it allows the machine to see, hear or speak away from its position (aircraft), and the operator can also communicate with the machine from different positions, representing a true telepresence of the machine.



Figure 7-26 Example Devices for Sensorization and Audio/Video Comm.

Internal Sensorization

The sensorization of rotating hydraulic machines (motors, pumps, filters…) gives the artificial intelligence information about its "state of health", so that it can detect failures before they occur, identify the root cause of the failure and even plan the repair. All this increases the service life of the equipment and prevents unplanned stoppages.

The main variables to monitor are:

Pressure/Flow/Level: these measure characteristics of the fluid and are essential for controlling tightness, overpressure, leaks and flow. The HT2000 has the following pressure sensors (PA1, PA2, PA3), as well as pressure drop switches on the filters (PDA1, PDA2, PDA3).

Temperature: a rise in temperature usually indicates excessive friction, overload, lubrication problems or pump malfunction. Our machine has a hydraulic fluid temperature sensor (TA1), a PT100 model with a 0-80 °C range and 4-20 mA output.

Electric Current: used mainly to monitor the electric motor that drives the hydraulic pump. Our machine has an electric current sensor (LA1).

Vibration: this is the first indicator of mechanical problems such as misalignment, imbalance, bearing wear or loose parts. The machine has no vibration sensors, so a Siemens SM128 vibration monitoring system will be installed, consisting of:

- SIPLUS CMS1200 SM128 system. It will be connected to the PLC to receive vibration-related data (frequencies, amplitudes, alerts...).

- Four Vibration Sensors (SIPLUS CMS2000 VIB-sensor). The sensors will be installed on the main electric motor (coupling side), on the hydraulic pump housing (axial pistons), on the charge pump motor and on the fan motor
- Speed sensor (SIMATIC PXS) to measure RPM.

In rotating machines, as in our case, vibrations are mainly produced by centrifugal forces in the rotating parts of the machine and are due to:

- Imbalance
- Alignment errors in drive trains (motor-pump)
- Bearing damage
- Gearbox/adapter failure
- Alternating forces arising from operation, whether magnetic, hydraulic or other

By measuring and analyzing the vibration of a machine, it is possible to determine both the nature and the severity of the defect, and therefore to predict machine failure. This is because mechanical defects produce characteristic vibrations at different frequencies. By analyzing the spectra in the time and frequency domains and using signal processing techniques, the types of defects and their severity can be identified.

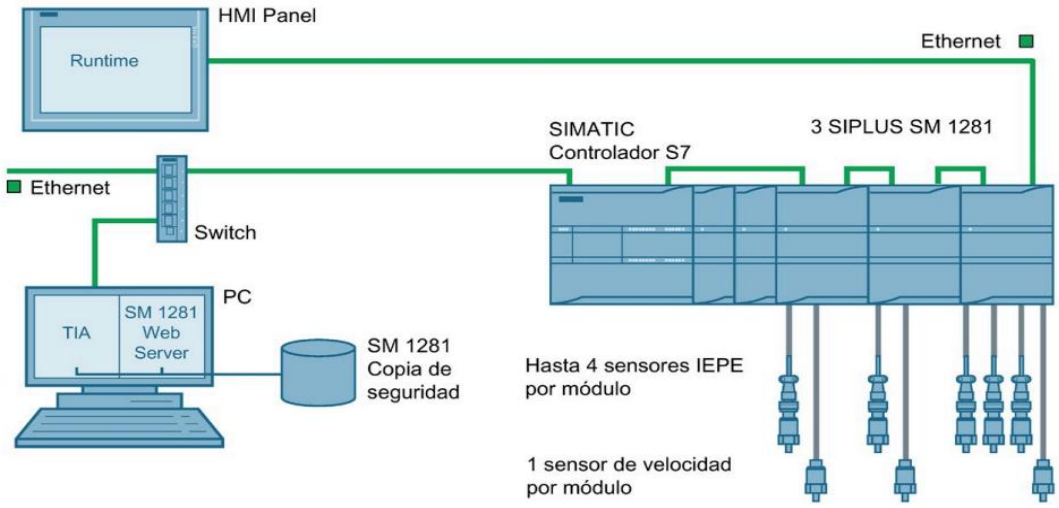


Figure 7-27 Vibration Monitoring System (SM 128) (Source: Siemens)



Figure 7-28 Internal Sensorization of the Hydraulic Machine (Source: HYCOM)

In our HT2000 machine, this whole process has been simplified by installing a commercial SIPLUS CMS1200 system that provides the PLC with vibration-related data.

There are two operating alternatives:

- *Measuring:* the system acts as a simple measuring instrument. The quantities of the configured channels are recorded, displayed and transmitted directly to the PLC.
- *Monitoring:* in this mode, the device records all the measured quantities of the configured channels, evaluates them, records the process data and, if limit values are exceeded, triggers the corresponding fault and warning alarms.

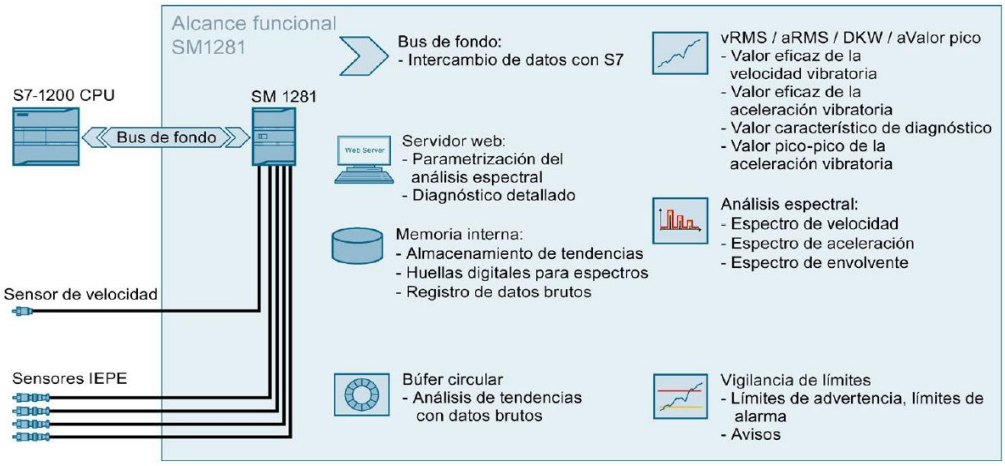


Figure 7-29 Data Collection/Exchange with the PLC (Source: SIEMENS)

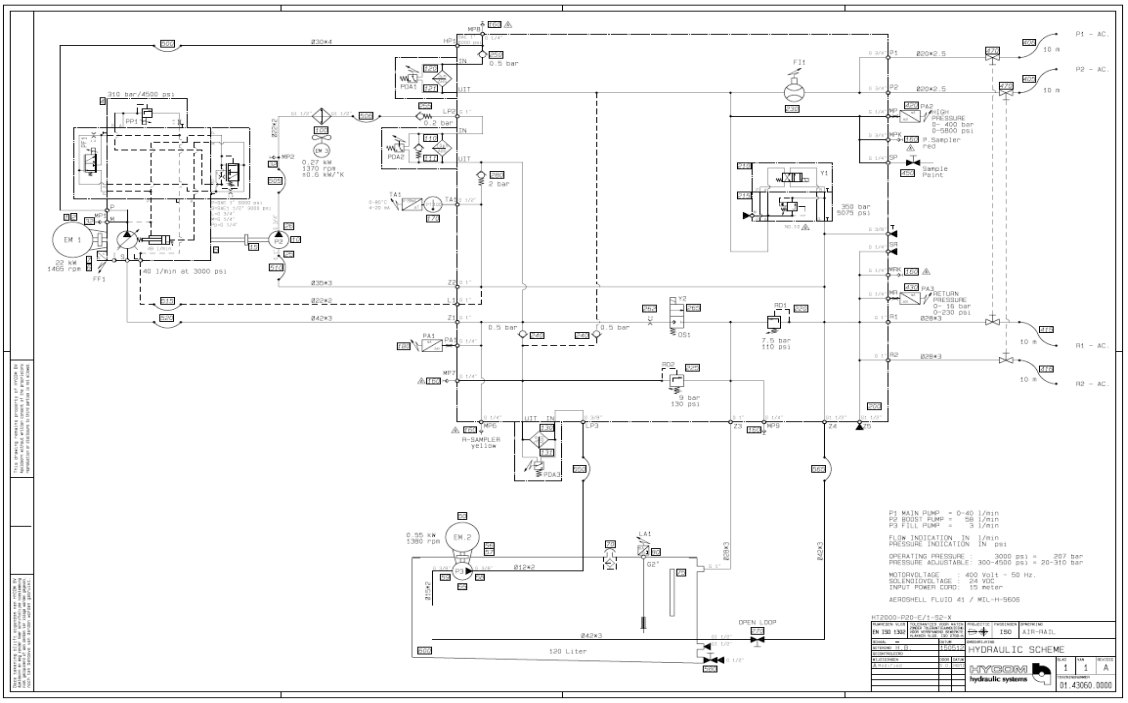


Figure 7-30 HT2000 Hydraulic System Schematic (Source: HYCOM)

7.8.3 Software Architecture

The HT2000-AI system is built around a layered client-server architecture, where each layer has a clearly defined responsibility and communicates with the adjacent ones through defined interfaces.

We therefore have:

- C*lient Layer:* groups the user interfaces accessible from the web browser: the conversational chat with the AI, the machine control panel, the real-time telemetry monitor and the administration panel. All of them communicate with the central server via WebSockets (Socket.IO) for real-time updates and via HTTP REST requests for one-off operations.
- S*erver Layer:* the core of the system.

Implemented in Node.js with Express, it centralizes management of the machine state, the secure proxy to the external artificial intelligence services, the safety Watchdog, the semantic information retrieval system (RAG) and the per-operator adaptive learning engine.

- *The External Services Layer:* encompasses the language processing, voice generation and storage services.

  Conversational reasoning is delegated to the Mistral model, accessed exclusively through the server to protect the access credentials.

  Speech synthesis is handled through two complementary routes: Piper TTS locally, which guarantees operation without connectivity, and ElevenLabs as a cloud alternative for greater expressive naturalness.

  PostgreSQL is used as the data store, recording conversations, machine telemetry events, adaptive operator profiles, indexed fragments of the technical manuals and session reports.

This layered separation of responsibilities ensures system maintainability, the security of the access keys to external services and the possibility of evolving each module independently.

The component diagram shows the modules that make up the system and their actual dependencies, organized into the three layers mentioned above.

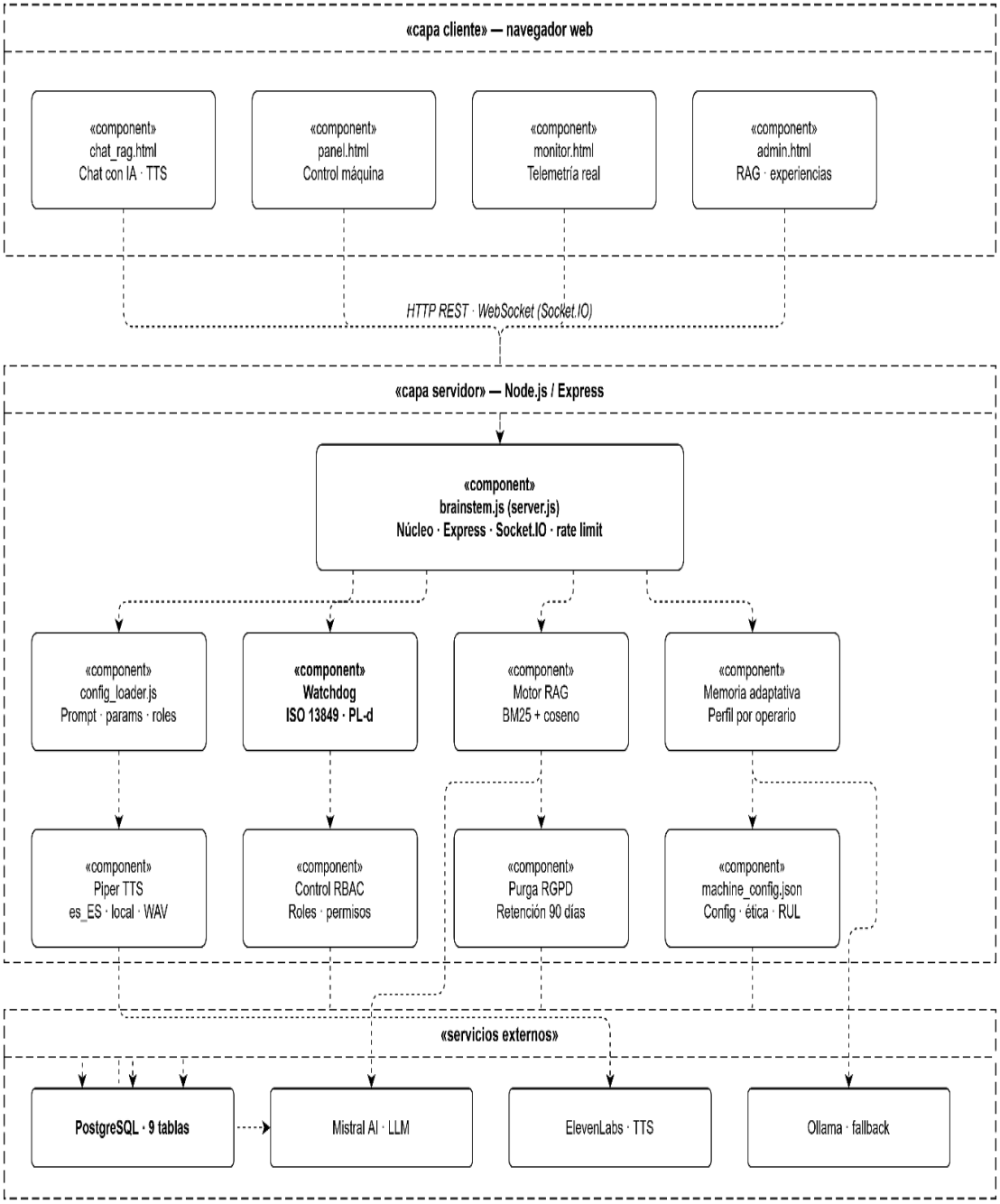


Figure 7-31 Component Diagram (Source: Own Elaboration)

The following figure shows the system's UML (Unified Modeling Language) use case diagram, obtained by reverse engineering the implemented source code.

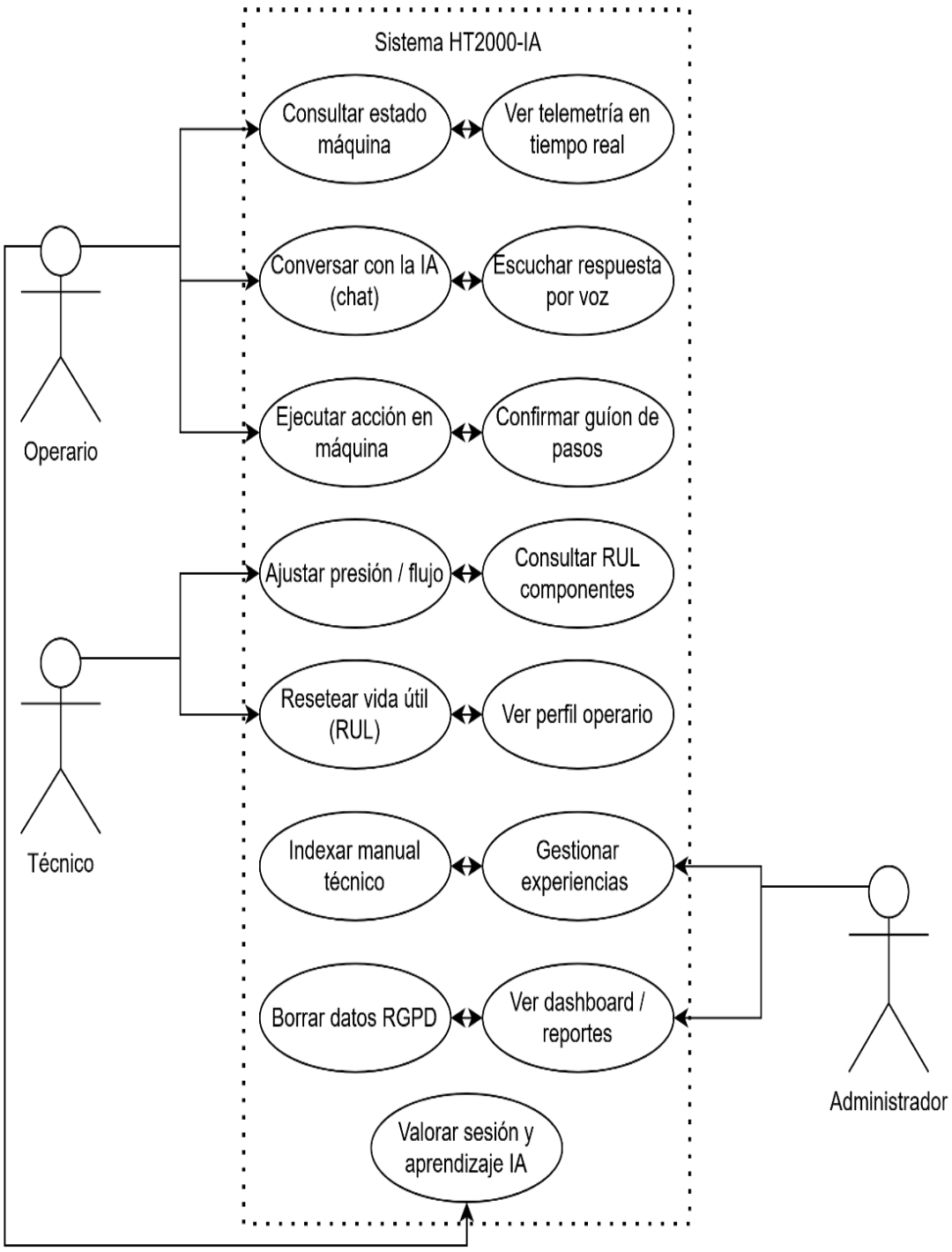


Figure 7-32 HT2000-AI Component Diagram (Source: Own Elaboration)

7.9 Model Selection

This section sets out the most relevant technical decisions taken during system development, justifying the choice made over the alternatives considered.

7.9.1 Language Model (Mistral vs GPT-4/Claude)

The selection of Mistral as the main language model is based on the following criteria:

- Availability of small models with competitive performance, in particular “mistral-small-latest”, which offers response latencies suitable for fluid conversational interaction.
- Existence of an embeddings endpoint (mistral-embed) that makes it possible to keep a single provider for reasoning and semantic retrieval, simplifying the architecture.
- Possibility of operating with compatible local models through Ollama in the event of loss of connectivity, which provides operational resilience without changing the interface.

In short, compared with alternatives such as GPT-4 or Claude, Mistral offers a more favorable cost-performance ratio for an intensively used system in an industrial environment, where the number of daily requests can be high and latency has a direct impact on the operator's experience.

7.9.2 Database Model (PostgreSQL vs NoSQL)

During the project, storage moved from JSON files to PostgreSQL.

This transition responds to three needs that file storage could not robustly meet:

- Concurrency of multiple simultaneously connected clients.
- Ability to run analytical queries on the history.
- Compliance with the General Data Protection Regulation.

PostgreSQL also allows indexes to be defined on the most heavily queried columns (session, operator, timestamp) and delegates to the database engine itself the automatic cleanup of records older than ninety days.

7.9.3 Voice Model (Piper TTS vs ElevenLabs)

The selection of Piper TTS as the local speech synthesis engine responds to a security constraint of the operating environment.

Aircraft assembly lines are industrial environments where internet connectivity is not always guaranteed, and relying exclusively on cloud services represents a potential point of system failure and an entry point for hacking.

Piper operates offline, with no network latency and no cost per request, using the es_ES-davefx-medium model, which produces a neutral male Spanish voice that is intelligible in moderate ambient noise conditions.
ElevenLabs is kept as an alternative for environments with stable connectivity where the expressive naturalness of the voice is more important.

7.9.4 Secure Proxy Architecture for API Keys

All access keys to the external Mistral and ElevenLabs services reside exclusively on the server and are never transmitted to the client.
The operator's browser sends its requests to the local server, which acts as an authenticated proxy to the external services.
This decision eliminates the risk of credentials being exposed through client network traffic or through inspection of the source code in the browser, and centralizes access control and “rate limiting” in a single point.

7.9.5 Configuration via machine_config.json

The decision to externalize the entire system configuration (AI personality, machine technical parameters, alarm thresholds, available actions, ethical rules, access roles and API keys) into a single JSON file follows the principle of separating logic and data.
This architecture makes it possible to reuse the same software core to transform any other industrial machine simply by replacing the configuration file, without modifying the server code.
The system thus acts as a framework for transforming industrial machines, not as an ad hoc solution for the HT2000.

7.9.6 Interface Development Technologies

The decision to implement the four interfaces of the HT2000-AI system as web applications in HTML5, CSS3 and native JavaScript, without component frameworks such as React, Vue or Angular, is based on specific technical and operational criteria, not on aesthetic preference.

- *HTML5, CSS3 and JavaScript versus component frameworks:* component frameworks such as React or Vue offer significant advantages in large-scale projects with development teams and continuous update cycles.

However, they introduce a layer of abstraction (the virtual DOM, the module system, the compiler) that adds maintenance complexity and dependence on ecosystems with their own life cycles.

In the context of the HT2000-AI system, where the interfaces must be maintainable by any developer with basic web skills and must keep working unchanged for years in an industrial environment with slow update cycles, native HTML5 is the most robust long-term choice.

- *Socket.IO for real-time communication:* real-time communication between the server and the interfaces is implemented using Socket.IO, a library that abstracts the WebSocket protocol with automatic fallback to HTTP long-polling when WebSocket is not available.

  The choice of Socket.IO over native WebSocket responds to a resilience requirement: in industrial environments with variable network infrastructure, automatic fallback ensures that real-time communication works even under degraded network conditions.

  Socket.IO also allows named events to be emitted (machine_state, execute_command, monitor_alert), which produces a semantic, self-explanatory communication protocol. Any developer reading the server or client code can understand the meaning of each event without additional documentation.

- *Node.js and Express for the server:* the central server is implemented in Node.js with Express for three technical reasons specific to the use case.
  - The first is the asynchronous, non-blocking nature of Node.js, which allows hundreds of concurrent WebSocket connections to be maintained without performance degradation, critical in a system where the panel, chat, monitor and administration panel may be connected simultaneously.
  - The second is language unification: both the server and the interfaces use JavaScript, which eliminates the mental context switch between server logic and client logic.
  - The third is the npm ecosystem, which provides immediate access to mature libraries for every need of the project: pg for PostgreSQL, cors for cross-origin management, socket.io for WebSocket.

Express was chosen over alternatives such as Fastify or Koa for its maturity, its exhaustive documentation and its proven compatibility with Socket.IO. Express version 5, used in this project, adds native support for promises and async/await in route handlers, eliminating the need for additional middleware to handle asynchronous errors.

- *PostgreSQL versus other persistence alternatives:* the choice of PostgreSQL as the system's sole persistence layer, over alternatives such as MongoDB, SQLite or the JSON file storage used by the system in earlier versions, responds to four specific needs that no alternative fully met.
    - The first is concurrency. PostgreSQL handles multiple concurrent connections with transactional isolation, something that JSON file storage cannot guarantee when multiple clients write simultaneously.
    - The second is analytical capability. Historical telemetry queries (calculating the average temperature over the last two hours, detecting degradation trends in the filters, calculating the accumulated hours of a component) require aggregation and filtering capabilities that SQL handles naturally and that a document database such as MongoDB would resolve with greater complexity.
    - The third is GDPR compliance. PostgreSQL allows retention policies to be defined through scheduled tasks that automatically delete records older than ninety days, and guarantees that the deletion of an operator's data is complete and verifiable through direct queries.
    - The fourth is the storage of embedding vectors. The text fragments indexed by the RAG engine are stored together with their embedding vectors in PostgreSQL's native JSONB format, eliminating the need for an external vector database such as Pinecone or Weaviate.
- *Python and python-snap7 for PLC integration:* communication with the Siemens CPU313C PLC is implemented through a Python script using the python-snap7 library, instead of integrating the communication directly into the Node.js server.

This decision responds to the availability of mature implementations of the Siemens S7 protocol: python-snap7 is the library with the largest community, documentation and verified use cases for communication with Siemens PLCs, and its pure-Python implementation without native dependencies makes it easy to deploy on any Linux system without compiling C code.

The Python script acts as a bridge between the PLC and the Node.js server through a local HTTP endpoint, maintaining the separation of responsibilities between the industrial communication logic and the server's application logic.

7.10 System Modules

7.10.1 Central Server (brainstem.js)

The central server is the core of the system and acts as an intermediary between all the layers. It is implemented in Node.js with the Express framework and handles both HTTP REST requests and real-time connections through Socket.IO.

Node.js was chosen for its asynchronous, non-blocking nature, which is especially well suited to a system that must maintain multiple simultaneous connections (panel, chat, monitor and administration) while handling telemetry events every ten seconds without degrading the operator's experience.

The server startup illustrates the system's initialization sequence and the interdependence between its modules.

1. *Configuration:* loaded from machine_config.json through the config_loader.js module, which builds all the server's working objects in

memory (prompt, parameters and roles). If the file does not exist or is malformed, the server issues a warning but continues to start in degraded mode.

2. *Database:* the server requires PostgreSQL to be present as a necessary condition for starting. If the PG_URL variable is not configured, the process terminates immediately with an explanatory message.
3. *RAG Cache:* the server creates the necessary tables if they do not exist and loads the RAG chunks into memory, ensuring that semantic searches do not require database queries on every request.
4. *External Services:* finally, the server checks in parallel the availability of the Ollama services for offline fallback and Piper for local speech synthesis, without blocking startup if either is missing (graceful degradation).

The following figure shows eleven ordered steps and their dependencies: the first four steps are mandatory and block startup in the event of any failure, while those associated with Ollama or Piper are non-blocking.

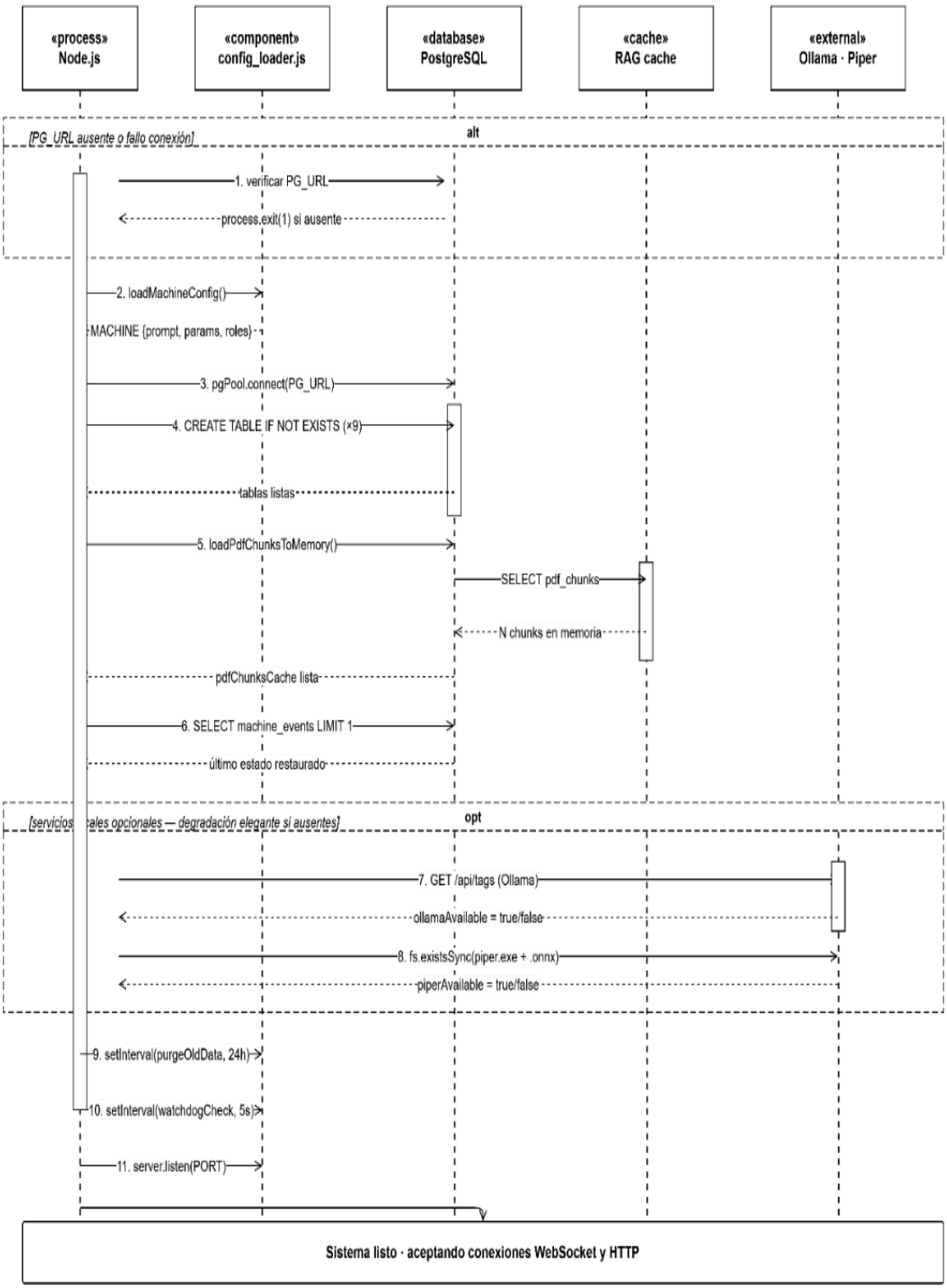


Figure 7-33 HT2000-AI Startup Sequence Diagram (Source: Own Elab.)

This startup design (fail fast for mandatory dependencies, graceful degradation for optional services) ensures that the system never starts in an inconsistent state, and that the operator knows from the very first moment which capabilities are available.

Machine State Configuration

The state diagram formalizes the machine's behavior according to the sensor values and the operator's actions.

Five states are distinguished:

- *Off*: initial state.
- *Running Low Power*
- *Running High Power*
- *Alarm active:* sensors exceed the warning thresholds in machine_config.json.
- *Locked:* by the watchdog when absolute limits are exceeded.

Transitions to the locked state are automatic: they are generated by the watchdog without intervention from the language model, while leaving the alarm state requires explicit action by the operator through borrado_alarma (alarm reset).

The machine state is kept in memory as a JavaScript object containing the current values of temperature, high pressure, return pressure, set flow, actual flow, reservoir level, motor status, power selector position and active alarms.

This object is updated every time the operator panel sends an update_state event, and is automatically persisted in PostgreSQL to survive server restarts. The server merges the new state with the existing one and immediately broadcasts it to all connected clients: panel, chat and monitor receive the same state simultaneously.

Telemetry is recorded in the machine_events table every ten seconds while the machine is running, which makes it possible to reconstruct the operating history and feed the predictive maintenance system based on remaining useful life.

On startup, the server retrieves the last known state from PostgreSQL, so that a process restart does not cause the machine's operational state to be lost.

This design (in-memory state as the source of truth, event-based propagation and asynchronous persistence) allows the system to keep update latencies below 100 ms between the panel and the chat, without database writes blocking the operator's experience.

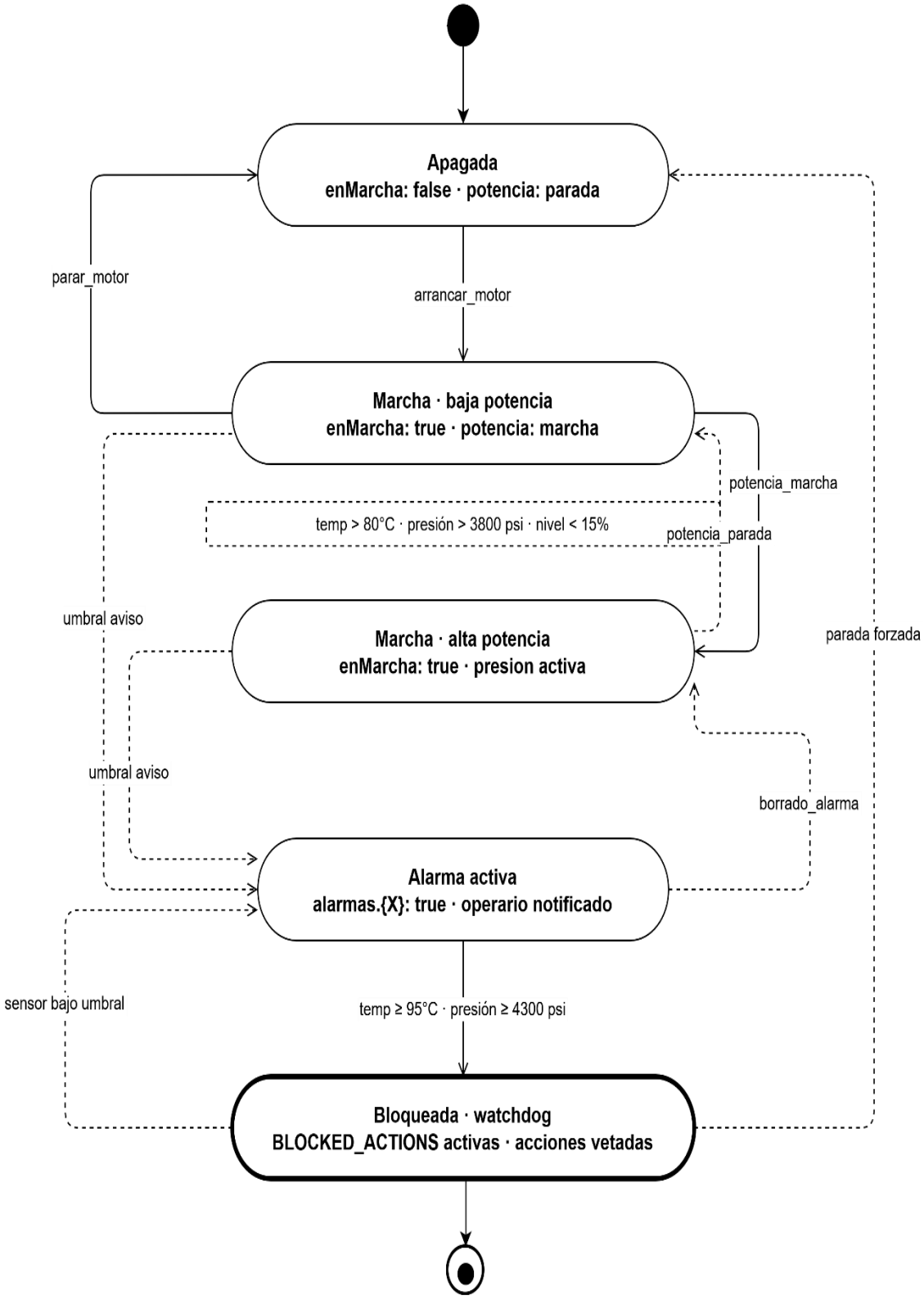


Figure 7-34 HT2000-AI State Diagram (Source: Own Elaboration)

7.10.2 Search Engine (RAG)

The RAG engine (Retrieval Augmented Generation) allows the artificial intelligence to base its answers on the actual content of the machine's technical manuals, instead of relying exclusively on the general knowledge of the language model.

The technical manuals are processed and split into text fragments from the administration panel, where the client generates the vector embeddings through the Mistral API and sends them to the server for storage in PostgreSQL.

For each query, the server retrieves the most relevant fragments using a hybrid search that combines cosine similarity on the embedding vectors with BM25 scoring on keywords.

The retrieved fragments are injected into the model's context before the answer is generated, so that the AI can cite sections and component codes exactly as they appear in the manual, reducing the risk of invented technical information.

The result of the hybrid search is a list of fragments ranked by combined score, from which the five most relevant are selected and injected into the model's prompt.

The quality of the system's answers depends directly on the richness of the documents indexed in the knowledge base.

For the prototype developed in this project, several documents have been indexed in the RAG system's knowledge base, constituting the primary technical knowledge sources of the HT2000-AI system.

- *Instruction Manual HT2000-P20-E/1-S2-X* by Hycom B.V., the official technical manual of the hydraulic test machine covered by the project.
  This document covers all of the HT2000's operational knowledge: safety instructions and use of PPE, startup and first-connection procedures, the complete description of the operating panel's controls and indicators with their component codes (12H06, 15P7, 10S08, 12SH6 and the rest), the troubleshooting guide organized by symptom with specific causes and solutions for each component, and the periodic maintenance program with tasks to be performed before each use, monthly, every six months…

Once indexed, the system can answer queries such as "what do I do if the HIGH PRESSURE FILTER CLOGGED light comes on?".

- *A320/321 Flight Crew Training Manual (FCTM)* by Airbus, the reference manual for the hydraulic system of the aircraft that the HT2000 is designed to test.
  This document is essential because the HT2000 operator does not work in a vacuum: they work connected to a real aircraft hydraulic system, and the operational decisions during the test (target pressures, permissible flows, connection and disconnection procedures) are directly conditioned by the specifications of the aircraft's hydraulic system.
  With this document indexed, the HT2000-AI system can answer questions such as "what return pressure should I expect in the A320 blue system?" or "what is the nominal pressure of the A321 hydraulic system?" with data taken directly from the aircraft manual.

- *HT2000 electrical and mechanical fault tree*, a structured diagnostic document that organizes fault symptoms into a cause-and-effect hierarchy.
  Unlike the instruction manual, which presents faults in tabular form by section, the fault tree allows the system to follow branched diagnostic logic: given an observable symptom (excessive noise, insufficient pressure, high temperature), the document guides the technician through a sequence of checks until the root cause is identified.
  With this document indexed, the RAG engine can retrieve complete branches of the diagnostic tree in response to an operator query, allowing the AI not only to identify the probable fault but also to propose the exact sequence of checks to confirm it, in the order established by the document.
  This kind of knowledge, structured as a decision tree, is especially valuable for the system because it complements the declarative knowledge in the manual with procedural diagnostic knowledge, precisely the kind of reasoning that novice operators find hardest to carry out without supervision from an experienced technician.

The combination of these documents creates a complementary knowledge base: the HT2000 manual covers knowledge of the test equipment, and the A320/321 FCTM covers knowledge of the system under test (the aircraft).
The knowledge base can be extended at any time from the administration panel without modifying the server code, by indexing additional documents such as test procedures specific to each aircraft type, Hycom technical bulletins or maintenance instructions from the operating organization.

7.10.3 Operator Profile (operator_memory)

The system maintains a structured profile for each operator that captures the inferred technical level, preferred communication style, recurring problems observed, notable achievements and a summary of the last session.
It works as follows:

- At the start of each new session, the server retrieves this profile through the endpoint /api/operator-context/:operator and injects it into the system prompt as an adaptive context block, allowing the artificial intelligence to adapt its level of detail, tone and references to the operator's known history.
- At the end of each session, the server automatically generates a summary of the conversation and updates the operator's profile in JSON format in the operator_memory table. This process has four stages:
    - The server generates a session summary by internally calling the language model with the history of the last 20 messages.
    - If the session received a rating of four stars or more and had at least six messages, the server tries to extract a tactical tip mentioned by the operator.
    - The server updates the operator's structured profile in JSON format, instructing the model to keep all previous information and incorporate what was learned in the new session.
    - The updated profile is stored in PostgreSQL's operator_memory table and is available for the next session.

For example, thanks to this module, a new operator receives more detailed explanations and personal protective equipment checks, while a veteran operator receives more direct answers and references to previous shared situations.

7.10.4 Speech Synthesis (TTS)

The system's voice output is handled through two complementary routes, whose selection is transparent to the operator:

- *Piper TTS* is the main route and consists of a local speech synthesis engine that works without an internet connection, using the es_ES-davefx-medium model in ONNX format.
  The server generates the audio in WAV format from the AI's response text and serves it to the client, which plays it in the browser.
  This solution guarantees that the voice system works even in industrial environments without stable connectivity.
- *ElevenLabs,* is an alternative for connected environments.
  It offers higher quality but, on the downside, it is not free software.
  In this case, the access key resides exclusively on the server and is never exposed to the client.

7.10.5 Role-Based Access Control (RBAC)

The sequence diagram shown in the following figure formalizes, in UML notation, the complete interaction cycle described in the previous sections.
Three distinct paths can be seen:

- *Pure Query Path*: the message goes through the RAG, the operator profile and the LLM to return text and voice.
- *Action Path:* encapsulated in the “alt” fragment, in which the watchdog validates the action before Socket.IO propagates it to the panel;
- *Periodic Telemetry Loop:* represented by the “loop” fragment, which records the machine state in PostgreSQL every ten seconds, independently of the interaction cycle.

The system implements a three-level access control model: operator, technician and administrator.

Roles are defined in the machine_config.json file through lists of user names, and the permissions associated with each role determine which actions can be executed from the chat and from the panel.
The watchdog acts independently of the role system and cannot be overridden by any of them.

Access control is implemented through three structures that work together:

- *the role definitions* in the configuration file.
- *the permission table* per role on the server.
- *the validation functions* that are invoked before any action.

Roles are defined in machine_config.json as lists of operator names, which allows permissions to be updated without touching the server code
The server maps each operator name to its role by consulting these lists, and assigns the operator role by default to any unregistered name.
The permissions of each role are defined as lists of allowed actions. The “admin” role uses the wildcard '*' to indicate full access, avoiding listing every possible action.
It is important to highlight the relationship between the role system and the safety watchdog: the watchdog operates with absolute priority over any role.
An administrator cannot execute an action blocked by the watchdog. This hierarchy ensures that physical safety limits cannot be overridden by any privilege level.
This validation hierarchy (watchdog first, roles second) implements in code the ethical principle defined in previous chapters, whereby physical safety has absolute priority over any level of operational privilege.

7.10.6 Safety Watchdog

The watchdog is a supervision subsystem independent of the language model, which evaluates the state of the sensors every five seconds and blocks dangerous actions before they are executed.
This independence is deliberate and follows the principle that physical safety cannot depend on the availability or correct operation of an external artificial intelligence service.

The watchdog implements three absolute limits: maximum temperature of 95 °C, maximum pressure of 4,300 psi and minimum reservoir level of 5%.

If any of these thresholds is exceeded, the affected actions are blocked in an in-memory set, and any attempt to execute them, whether from the chat or the panel, is rejected with an explanatory message.

This design is consistent with the PL-d performance level defined in EN ISO 13849 for safety-related control systems.

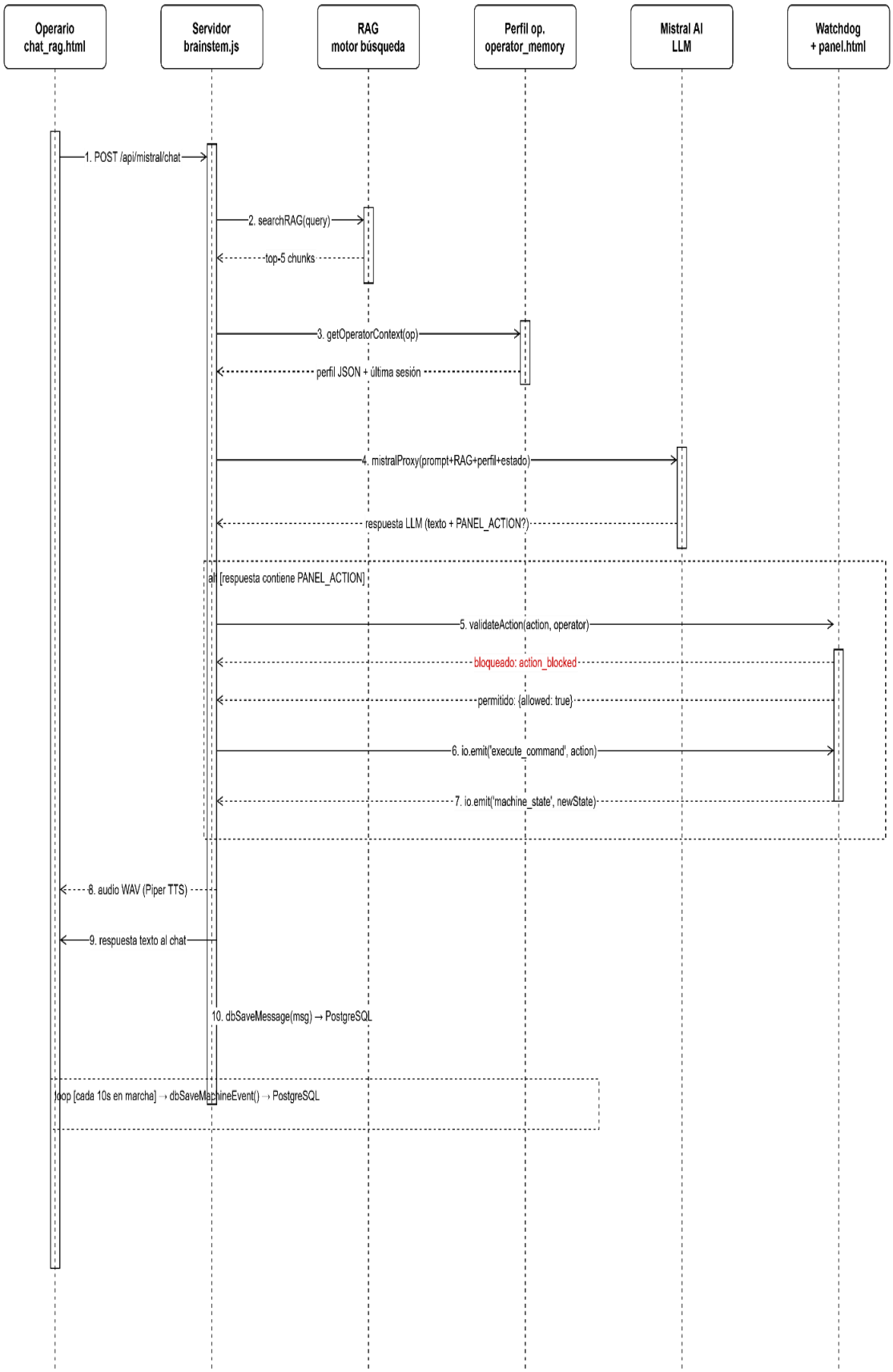


Figure 7-35 Interaction Cycle Sequence Diagram (Source: Own Elab.)

7.10.7 User Interfaces

The user interfaces are the access layer to the HT2000-AI system and represent the point of direct contact between the operator and the artificial intelligence embodied in the machine.

The system has four independent but cohesive web interfaces, accessible from any browser connected to the workshop's local network without the need to install additional software on the operator's device.

Each interface has a single, clearly defined responsibility, applying the principle of separation of responsibilities described in previous sections

- *Conversational Panel:* handles natural language interaction.
- *Digital Control Panel:* replicates the digital twin of the physical machine.
- *Telemetry Panel:* displays real-time telemetry.
- *Administration Panel:* manages the system database.

The visual design of the four interfaces follows the Dieter Rams principles developed in chapter 6, always prioritizing the reduction of the operator's cognitive load over any aesthetic consideration.

In an aeronautical hangar environment, where the operator's attention is a scarce and safety-critical resource, every visual element that does not provide directly actionable information is an obstacle between the operator and the machine.

The following sections describe each interface in detail, with reference to the screenshots taken during the system validation sessions.

7.10.8 Conversational Panel (chat_rag.html)

The chat interface is the operator's main entry point to the HT2000-AI system and the channel through which the embodied artificial intelligence communicates in natural language.

Its design follows Dieter Rams' principle of minimal cognitive load: the operator does not need to learn any formal language or navigate hierarchical menus to interact with the machine, they simply write or speak as they would with a technical colleague.

Operator Identification and Authentication Screen

Access to the system is through an identification screen, which asks the operator for their name and a four-digit PIN.

The design of this screen is deliberately minimalist: white background, two centered input fields, clean typography without decorative elements, in direct application of the principles of good design.

The system registers the PIN on first access and permanently associates it with the operator's name, so that on subsequent accesses authentication is immediate.

This mechanism ensures that the adaptive profile accumulated across sessions is correctly associated with the real operator using the machine.

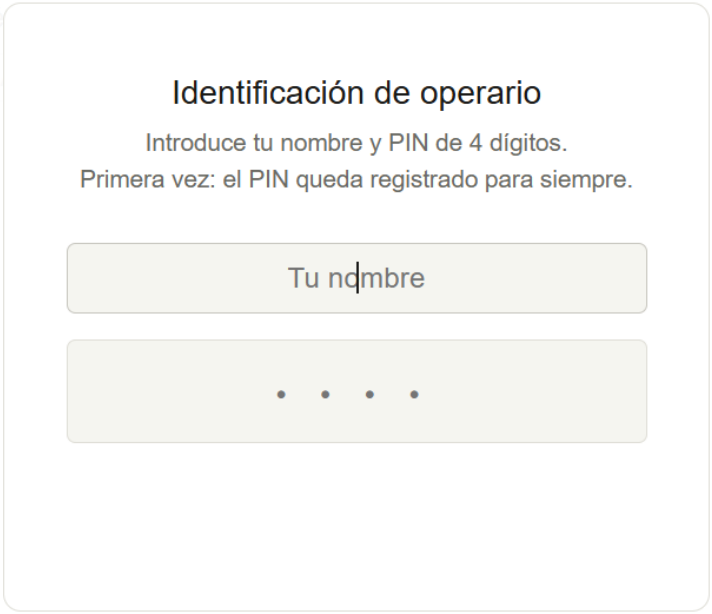


Figure 7-36 Identification Screen (Source: Own Elab.)

Home Screen (Welcome)

Once authenticated, the interface displays the welcome screen visible at the bottom.

The top bar communicates the overall state of the machine at all times (system name, running status, current temperature, name of the connected operator, operating mode and the AI's inferred emotional state) without the operator needing to check it actively.

In the session documented in the screenshot, the machine was in the Stopped state at 45 °C, with operator Fran connected in the operator role.

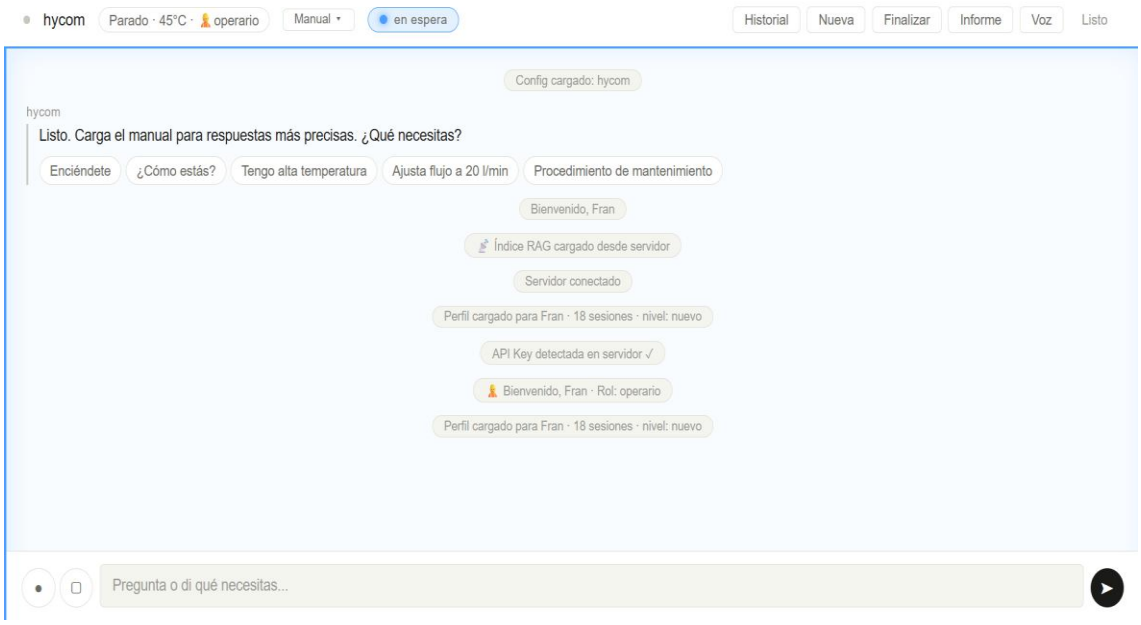


Figure 7-37 Home Screen (Source: Own Elab.)

The central area shows the complete system initialization sequence.

This startup transparency responds to the explainability requirement of Machinery Regulation 2023/1230 (the system does not hide its internal state from the operator, but communicates it before starting any interaction).

The shortcuts located under the initial greeting — "Turn on", "How are you?", "I have high temperature", "Set flow to 20 l/min", "Maintenance procedure" — implement Rams' principle of usefulness: the most frequent queries in the context of aeronautical hydraulic testing are accessible with a single tap, reducing operational friction in situations where the operator needs information quickly.

Home Screen (Conversation History)

Using the “History” button, a side panel, visible in the following figure, lists all previous sessions with date and operator name, allowing the technician to retrieve previous conversations to consult diagnoses or procedures carried out in earlier sessions.

The screenshot shows seven conversations recorded between 4 and 12 May 2026, all by operator Fran.

This history is not merely for reference — it directly feeds the server's adaptive memory module, which analyzes it at the start of each new session to update the operator's profile and personalize the AI's behavior.

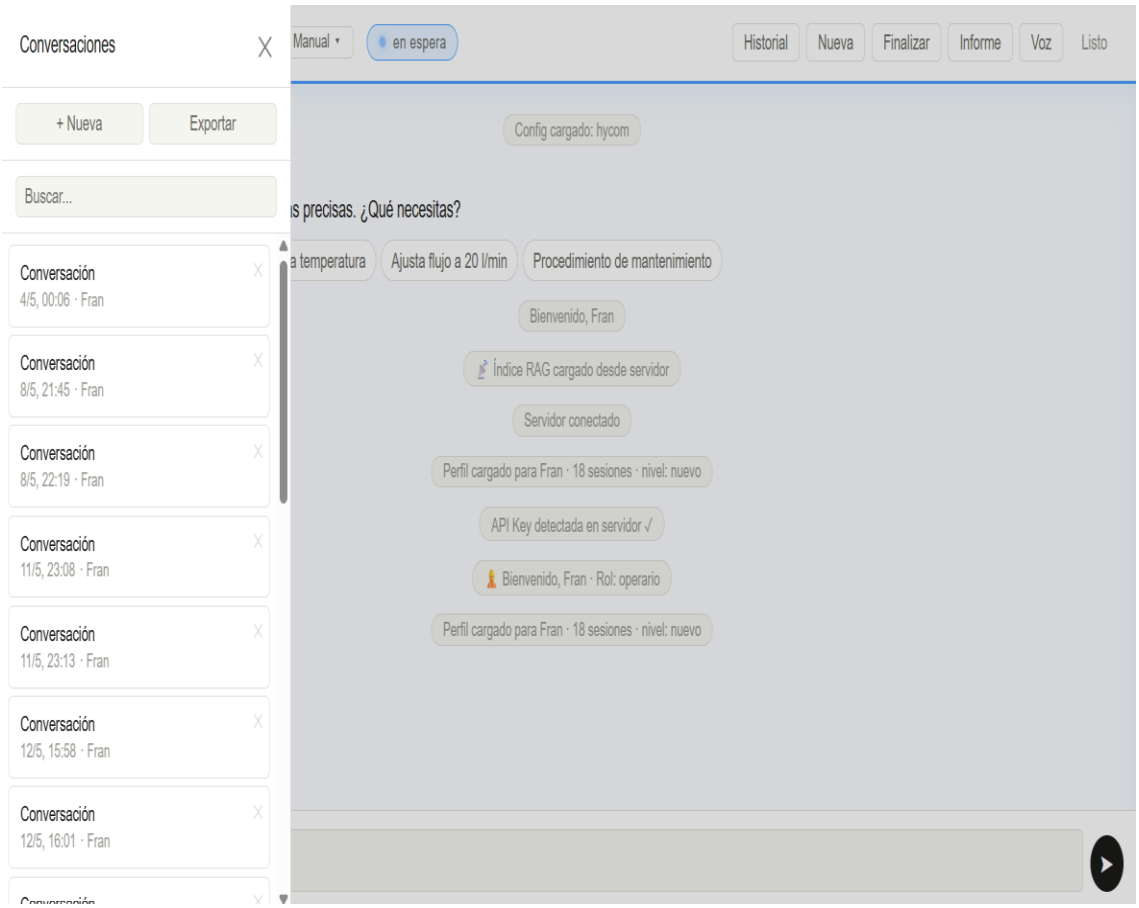


Figure 7-38 Conversation History Screen (Source: Own Elab.)

Home Screen (Action Execution)

In the example in the following figure, the operator has asked to turn on the machine (“Turn on”) and the AI has generated a two-step script that appears in the left side panel with the details of each action and its reference code in the manual:

- Clear Alarms: [9S10].
- Start Motor: [12SH6].

After the operator confirms with a simple "yes", the system executes both actions sequentially and confirms the result with the messages:

- "Executing: borrado_alarma — Alarm reset activated"
- "Executing: arrancar_motor — Motor starting".

The top bar immediately reflects the change of state: the machine switches to "Running · 20.5°C" with the "happy" indicator.

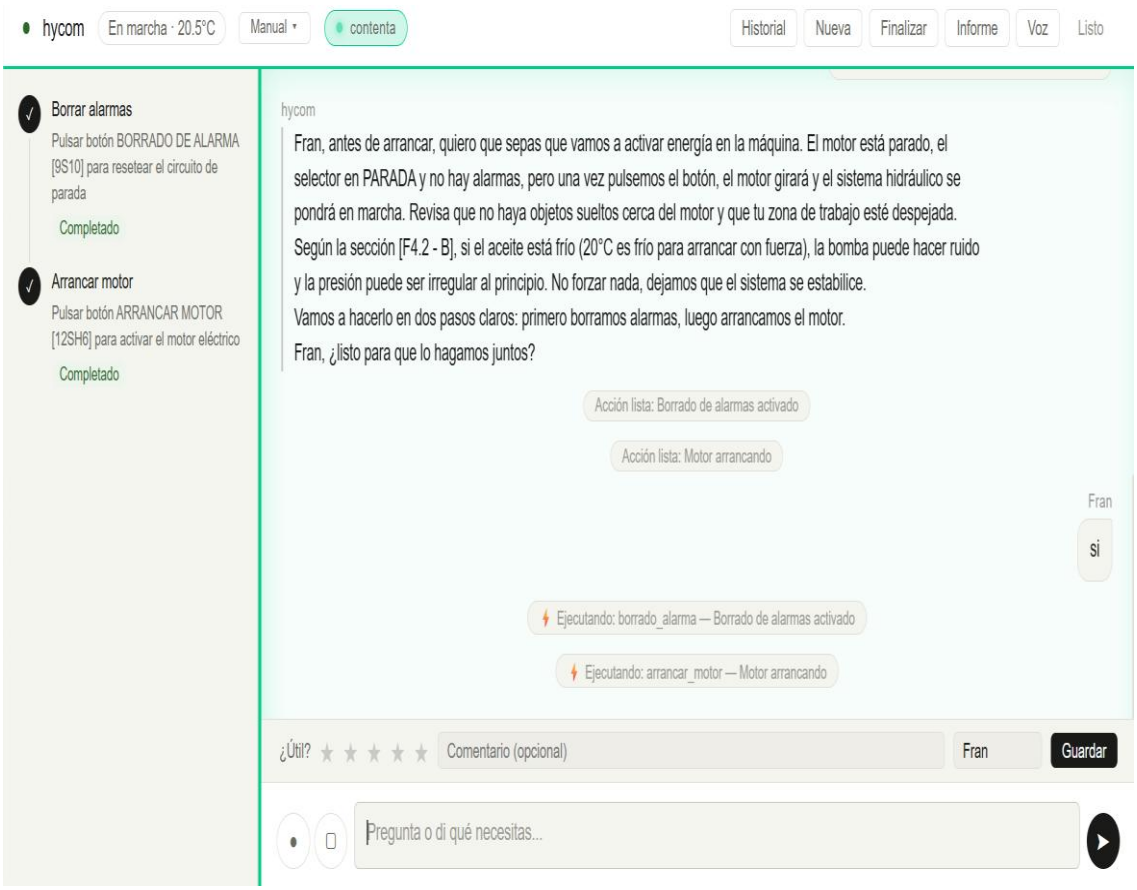


Figure 7-39 Action Execution Screen (1) (Source: Own Elab.)

Next, a more complex sequence of five actions is shown:

- Clear Alarms: [9S10].

- Start Motor: [12SH6].
- Activate Power Selector: [10S08].
- Set Flow to 10 l/min: [11S16], [11S18].
- Set Pressure to 500 psi: [10S18], [10S16].

All completed successfully and recorded in the side panel.

The AI has reasoned about the dependencies between actions: before adjusting the pressure or flow, it checks that the motor is running and the power is on, citing manual sections [12SH6] and [10S08] to justify the sequence. This behavior shows that the RAG engine not only retrieves information but actively applies it when reasoning about the activation conditions of each action.

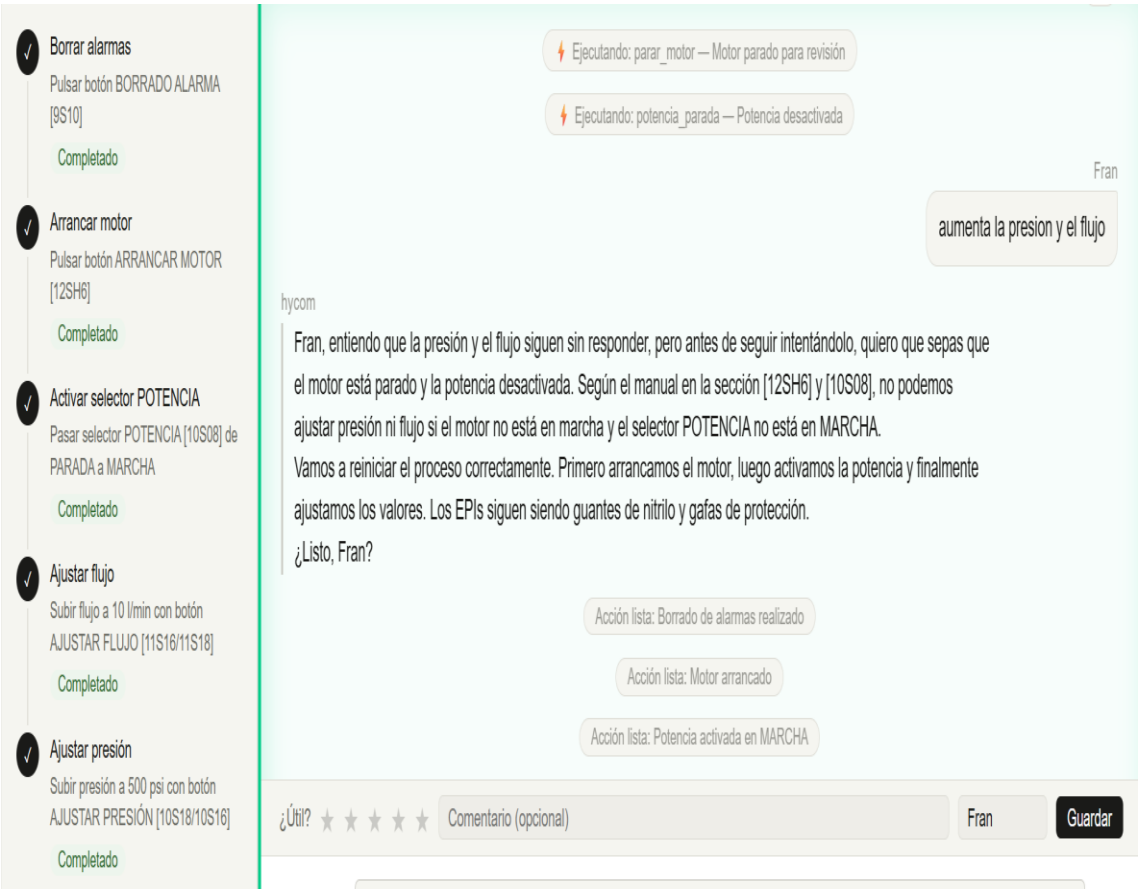


Figure 7-40 Action Execution Screen (2) (Source: Own Elab.)

Home Screen (Alert System)

The following figure shows the system in alert state, with the top bar in red and the "on alert" indicator.

During a test at 500 psi and 10 l/min, the temperature has risen to 34.1 °C at a rate of 1.5 °C/min.

The AI has proactively generated two alerts without the operator requesting them: first a predictive warning, "Temperature rising 1.5°C/min — in about 13 min I will reach the warning limit. Shall we reduce power now?", and then a critical alert when the temperature exceeded the automatic cut-off threshold at 84.5 °C: "Oil temperature 84.5°C exceeds automatic cut-off (70°C). Pressure 0 psi and flow 0 l/min indicate critical pump failure. Motor must stop immediately to prevent thermal damage. Maximum urgency: technical intervention in less than 30 min."

This behavior implements the principle of ecological awareness and the ethical safety veto defined in the machine_config.json file.

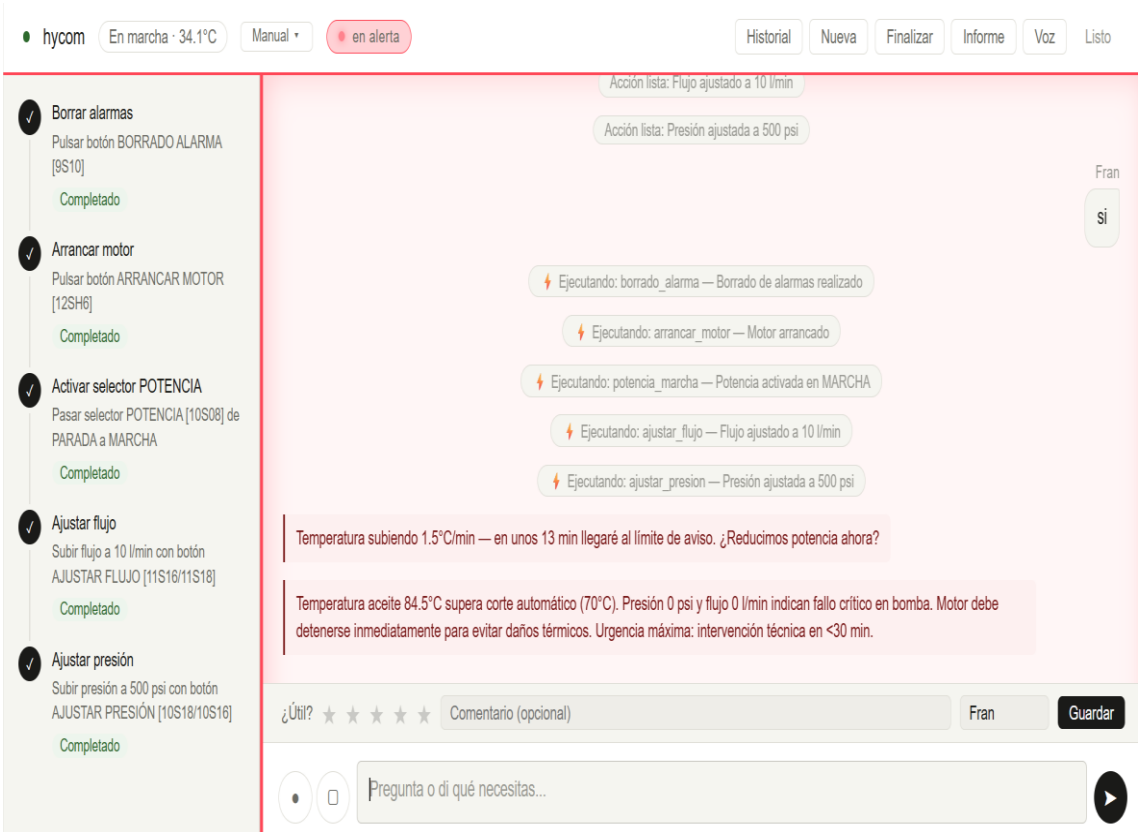


Figure 7-41 Alert System Screen (Source: Own Elab.)

Home Screen (Rating and Session Report)

At the end of the session, the system collects the rating and automatically generates a complete report, as shown.

The report includes the session/operator metadata, duration (12 minutes), number of exchanges (19), rating, the final machine state (motor stopped, temperature 44.2 °C, pressure 0 psi, flow 0 l/min), the complete script of executed actions with their descriptions, and the full transcript of the conversation with timestamps. This report is stored in PostgreSQL and can be accessed from the administration panel.

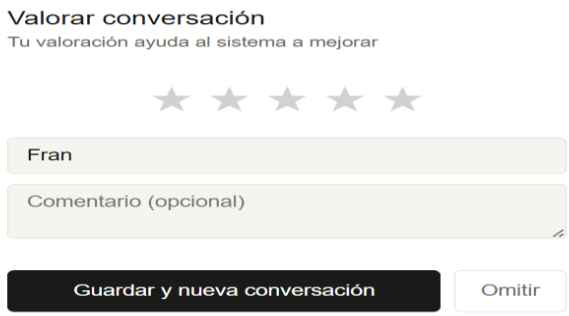


Figure 7-42 Rating Screen (Source: Own Elab.)

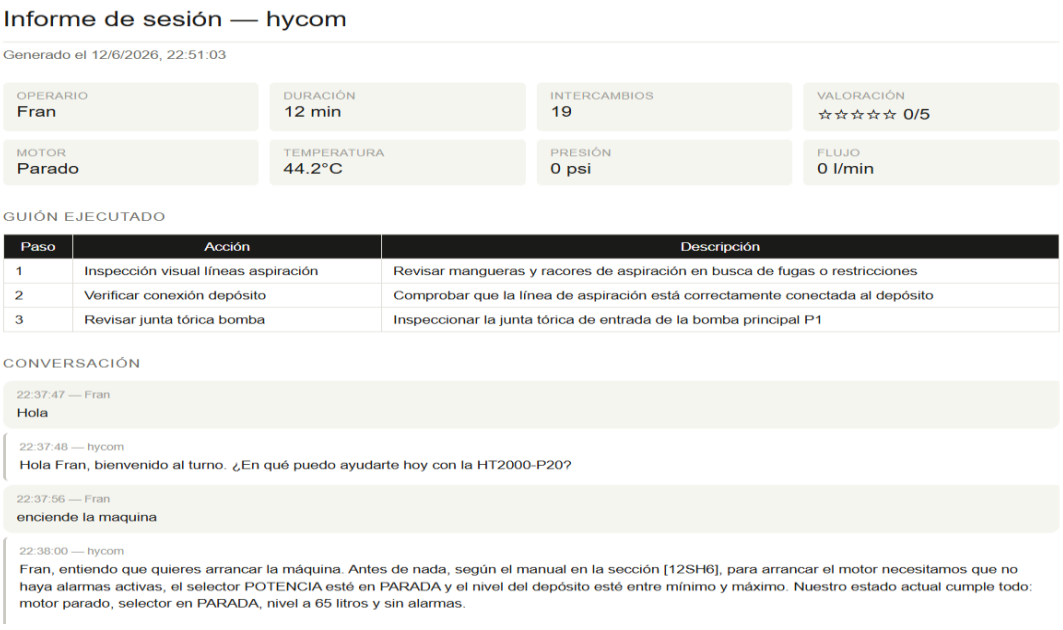


Figure 7-43 Session Report Screen (Source: Own Elab.)

Home Screen (Privacy)

The following figure shows the privacy notice that the system presents to the operator before enabling data storage, in compliance with Article 13 of the GDPR. The notice clearly and concisely states:

- *What data is collected?:* operator name, conversation history, ratings and machine state
- *Why is it recorded?:* to improve the assistance and maintenance system
- *How long is it kept?:* ninety days with automatic deletion.
- *Operator Rights:* data deletion can be requested.

Explicit acceptance via the "Understood and I accept" button constitutes the record of informed consent required by the regulation.



Figure 7-44 Privacy Screen (Source: Own Elab.)

7.10.9 Digital Control Panel (panel.html)

The control panel is the most distinctive interface of the HT2000-AI system because it was not designed as a conventional HMI screen but as a visual digital twin of the physical machine.

Its fundamental design principle is the exact correspondence between the digital representation and physical reality: each component appears in the same relative

position it occupies on the real machine, with the same name and technical manual reference code in square brackets.

An operator familiar with the HT2000-P20 can use this interface without any prior learning (the digital interface is a mirror of the real machine, not an abstraction of it).

There are two states:

Idle state, machine stopped

With the machine freshly initialized and the motor stopped, the top bar title "HT2000-P20 · Digital Twin — Control Panel" and the green "Server connected" indicator in the top right corner confirm that the interface is synchronized with the server in real time via WebSocket.

The bottom bar shows the panel's console log with the message "Panel started", allowing the technician to check the connection status without external tools.

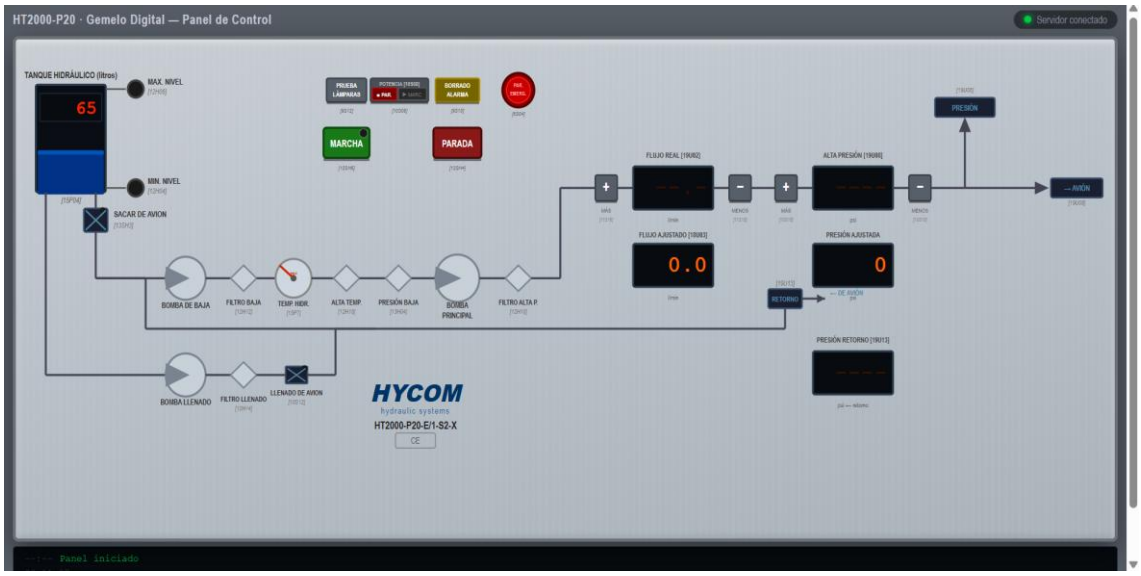


Figure 7-45 Digital Control Panel (1) (Source: Own Elab.)

The panel is an exact replica of the HT2000-P20 control panel as described in the technical manual.

The left area shows the hydraulic reservoir with its current level of 65 liters, represented visually by a proportional blue column, flanked by the MAX LEVEL [12H06] and MIN LEVEL [12H04] indicators with their associated warning lights. Below the reservoir is the DRAIN FROM AIRCRAFT button [13SH3].

The central area contains the main operating controls arranged in the same order as on the physical panel: the top row groups the LAMP TEST button [9S12], the POWER ON/OFF selector [10S08] with its status indicators, the ALARM RESET button [9S10] and the red EMERG. STOP push button [6S04].

The bottom row shows the START [12SH6] button in green and the STOP [12SH4] button in red, larger in size to make them easy to locate quickly in emergency situations.

In the idle state, the ACTUAL FLOW [19U02], HIGH PRESSURE [19U08], SET FLOW [18U03] and SET PRESSURE displays show zero values in the system's characteristic orange typeface, while the RETURN PRESSURE [19U13] display remains dark.

The schematic hydraulic circuit in the center of the interface shows the main components of the high-pressure branch: LOW PRESSURE PUMP [12H12], LOW FILTER [12H12], HYDR. TEMP. [15P7], HIGH TEMP. [12H18], LOW PRESSURE [13H04], MAIN PUMP and HIGH P. FILTER [12H10] — and the filling branch: FILL PUMP, FILL FILTER [12H14] and AIRCRAFT FILLING [10S12]. Each component shows its state in real time: at rest, all appear in neutral gray, with no indication of any anomaly.

Operating state, machine running

The following figure shows the machine with the motor running and the hydraulic system pressurized.

The visual difference from the idle state is immediate: the displays take on their real values in bright orange on a black background (Actual Flow 10.4 l/min, High Pressure 495 psi, Set Flow 10.0 l/min, Set Pressure 500 psi, Return Pressure 62 psi), and the reservoir shows its updated level of 64 liters.

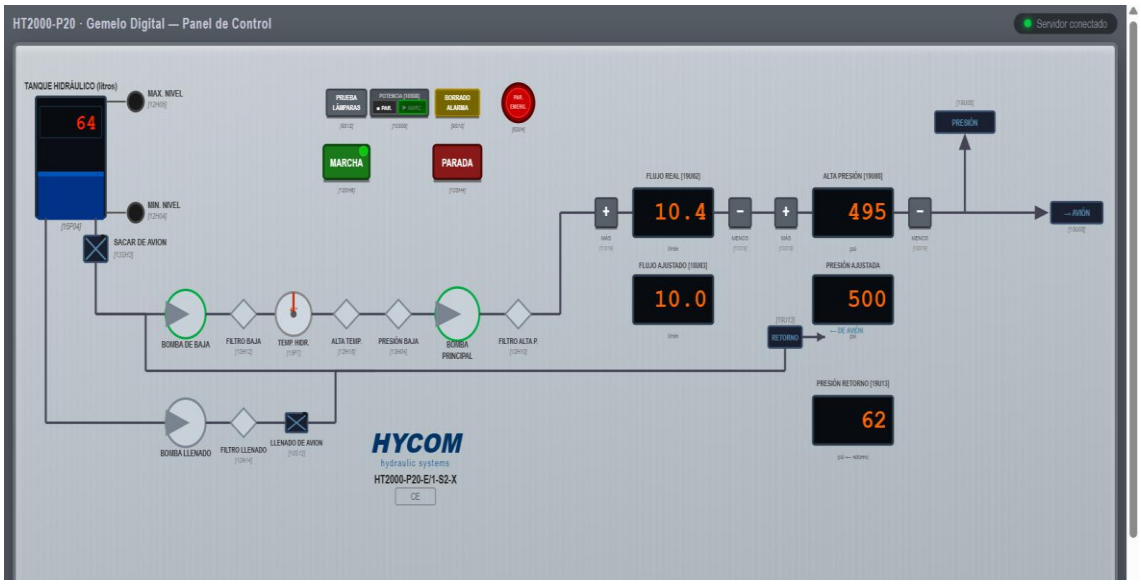


Figure 7-46 Digital Control Panel (2) (Source: Own Elab.)

The hydraulic circuit visually reflects the active state of the components: the pumps show their active rotation indicators, the filters reflect their cleanliness status and the path of the hydraulic fluid can be seen in the schematic (all of this fed by the server's machineState object, which is updated every 500 ms via WebSocket).

The flow and pressure adjustment controls, as well as the increment and decrement buttons next to each display, allow the operator to adjust the setpoints directly from the panel, with each press sending an execute_command event to the server, which passes through the watchdog validation chain before being applied. This integration between the visual interface and the safety engine is transparent to the operator — they simply press the button and the system guarantees that the action is safe before executing it.

7.10.10 Telemetry and Simulation Panel (monitor.html)

The telemetry monitor is the most technical interface of the HT2000-AI system and is designed for continuous monitoring of the machine's state during hydraulic tests.

Unlike the control panel, whose function is to execute actions, the monitor has an exclusively observational and analytical function: it gives the technician a complete view of the machine's behavior over time, with the ability to predict and simulate failure scenarios.

- *General structure of the interface:* the interface is divided into two clearly differentiated functional blocks:
    - *The left block*, which takes up roughly three quarters of the screen, groups the real-time metrics and the time-series charts.
    - *The right block,* which contains the predictive AI panel with the event log, the manual simulation controls and the predefined failure scenarios.

  This separation follows the principle of visual hierarchy: status information takes up the most space because it is the most operationally relevant, while the analysis and simulation controls stay in the background until the technician needs them.

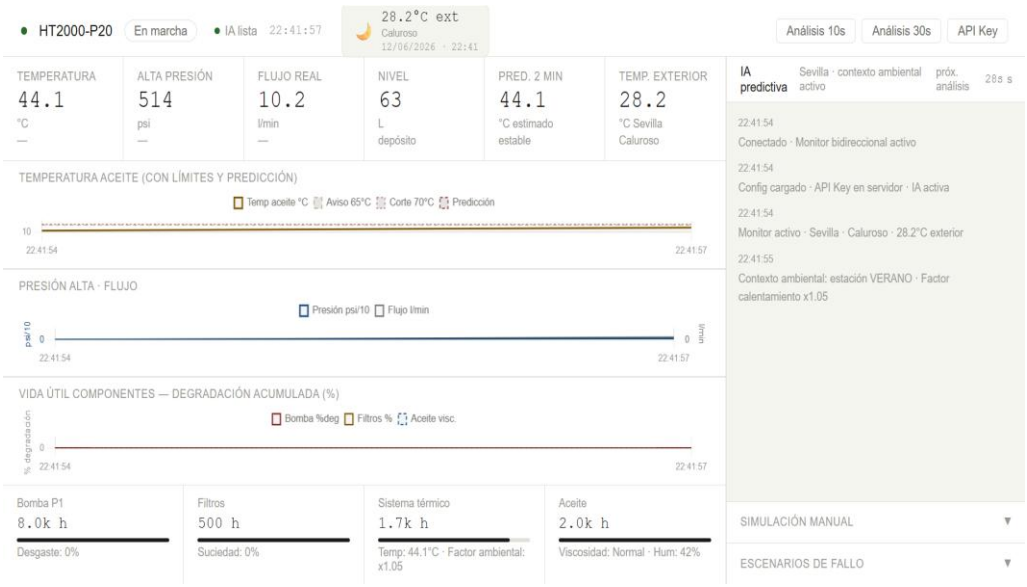


Figure 7-47 Telemetry Monitor Panel Screen (1) (Source: Own Elab.)

- *Real-time metrics:* in the previous figure, the row of tabs at the top shows the six most relevant parameters for supervising the hydraulic test: Oil Temperature, High Pressure, Actual Flow, Reservoir Level, 2-Minute Temperature Prediction and Outside Temperature.

  The 2-Minute Temperature Prediction field anticipates the system's thermal behavior before it occurs. This predictive capability is especially relevant in high-pressure tests, where the temperature can escalate quickly toward the alarm thresholds.

  The top bar also shows the active environmental context (Seville, hot, 28.2 °C outside, heating factor x1.05) and the status of the predictive AI with

the time until the next analysis. This environmental context directly influences the system's predictive models: in hot summer conditions, the system applies a correction factor to the thermal evolution model to anticipate the alarm thresholds more accurately.

- *Time-Series Charts:* the central block of the interface presents three independent time-series charts that are updated in real time.

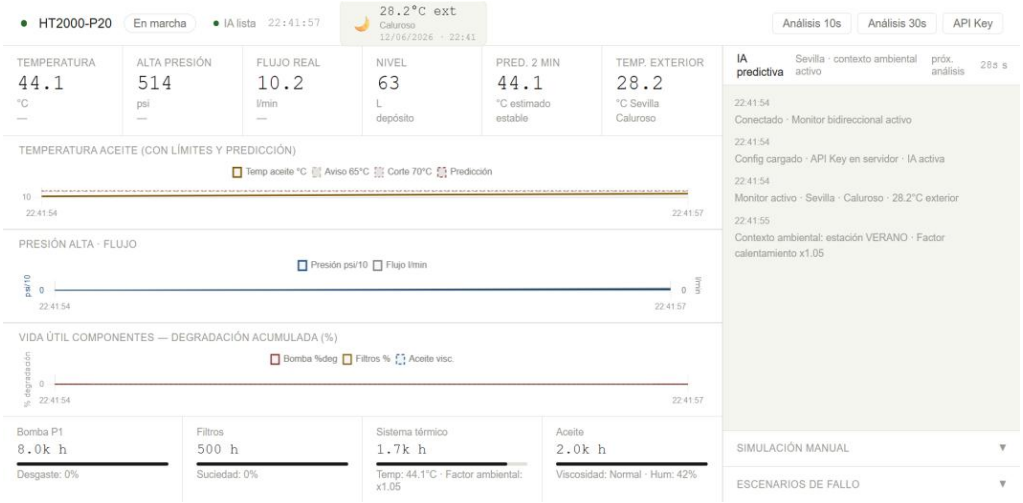


Figure 7-48 Telemetry Monitor Panel Screen (2) (Source: Own Elab.)

  - The first chart: shows the evolution of the hydraulic oil temperature with three overlaid references: the warning line at 65 °C as a dashed line, the automatic cut-off line at 70 °C as a bolder dashed line, and the prediction line calculated by the model. This design allows the technician to see the current value, the action limits and the expected trajectory simultaneously, without having to cross-reference information between different screens.
  - *The second chart*: combines scaled high pressure and actual flow on the same time axis, showing the correlation between both variables during the test. This representation is useful for detecting anomalies such as cavitation, which shows up as a decoupling between pressure and flow: high pressure but low or zero flow.
  - *The third chart*: shows the accumulated degradation of the three main components (Pump P1, Filters and Oil Viscosity), expressed as a percentage of service life consumed.

At the bottom of the interface: four component records show the accumulated operating hours: Pump P1 with 8.0k hours and 0% wear, Filters with

500 hours and 0% dirt, Thermal system with 1.7k hours and a temperature of 44.1 °C with an environmental factor of x1.05, and Oil with 2.0k hours, normal viscosity and 42% humidity.

- *Manual Simulation and Failure Scenarios:* the right panel shows the two advanced analysis blocks.

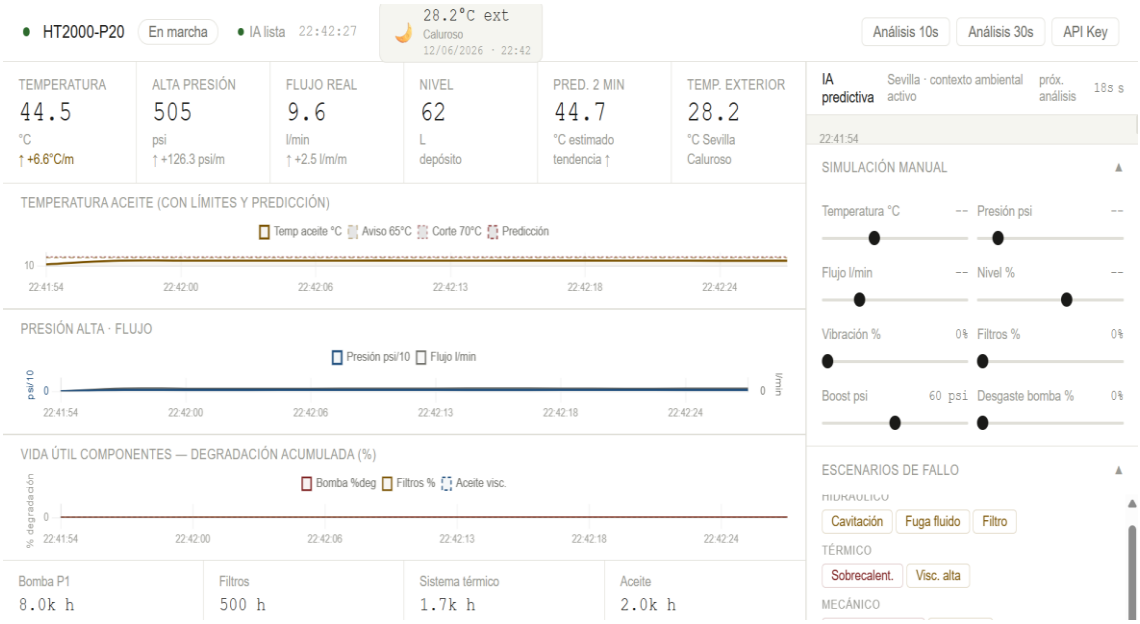


Figure 7-49 Telemetry Monitor Panel Screen (3) (Source: Own Elab.)

  - *Manual Simulation Panel:* allows the technician to force specific values for any parameter (Temperature, Pressure…) using sliders, without modifying the actual state of the machine. This functionality is essential for system validation tests: it makes it possible to verify that the watchdog and the predictive AI react correctly to extreme conditions without putting the machine at risk.

- *Failure Scenarios Panel:* groups predefined conditions into three categories that replicate the most frequent failures documented in the HT2000 technical manual:
  - Hydraulic Scenarios: Cavitation, Fluid Leak and Clogged Filter.
  - Thermal Scenarios: Overheating and High Viscosity.
  - Mechanical Scenarios: Pump Wear and Vibration.

When any of these scenarios is activated, the system injects the sensor values corresponding to the failure into the server's machineState, triggering the complete system response: watchdog alerts, AI communication to the operator and logging in PostgreSQL, exactly as would happen with a real failure.

- *Monitor alert system:* the bottom part shows the critical alert message generated by the monitor during a test session in which the oil temperature exceeded the automatic cut-off threshold at 84.5 °C.
  The message, timestamped 22:44:52, informs the technician precisely and urgently: temperature exceeded, pressure and flow values confirming critical pump failure, required action and an intervention deadline of less than 30 minutes.
  Fifteen seconds later, at 22:45:07, the monitor confirms that the situation has returned to normal with the message "Situation normalized — parameters within correct range".
  This two-way communication capability (alert on failure and confirmation of resolution) closes the safety supervision loop in a complete and traceable way, since both events are recorded in PostgreSQL with their exact timestamp.

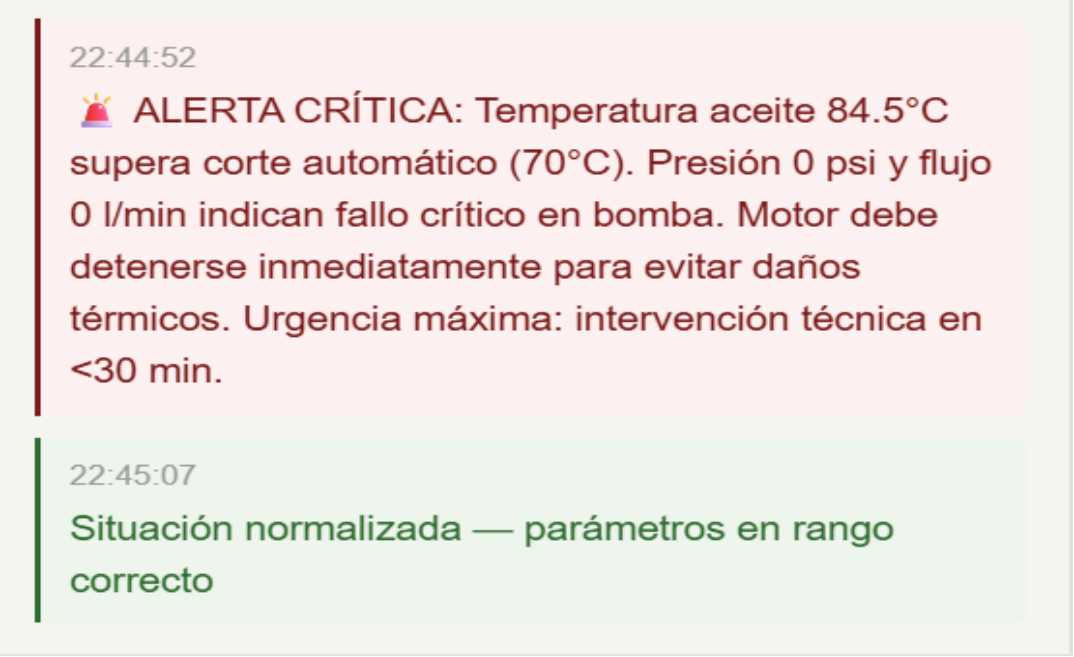


Figure 7-50 Alert System (Source: Own Elab.)

7.10.11 Administration Panel (admin.html)

The administration panel is the system's knowledge management interface and is aimed at the administrator responsible for keeping the RAG knowledge base up to date, recording the tactical experience of veteran operators and supervising the overall state of the system.

Access is restricted to the administrator role through the RBAC system, since the operations it allows (indexing and deleting manuals, recording tacit knowledge, accessing the complete conversation history, etc.) have a direct impact on the AI's behavior in subsequent sessions.

- *System Metrics Dashboard:* the main screen of the panel shows the seven metrics that give the administrator a consolidated view of the system's state in real time (Conversations, Operators, Experiences, Events, Reports, AI Tips and PDF Chunks).

  Confirmation of the active connection to PostgreSQL appears in the top bar next to the installation name, Hippocampus, along with shortcuts to Chat and Monitor.

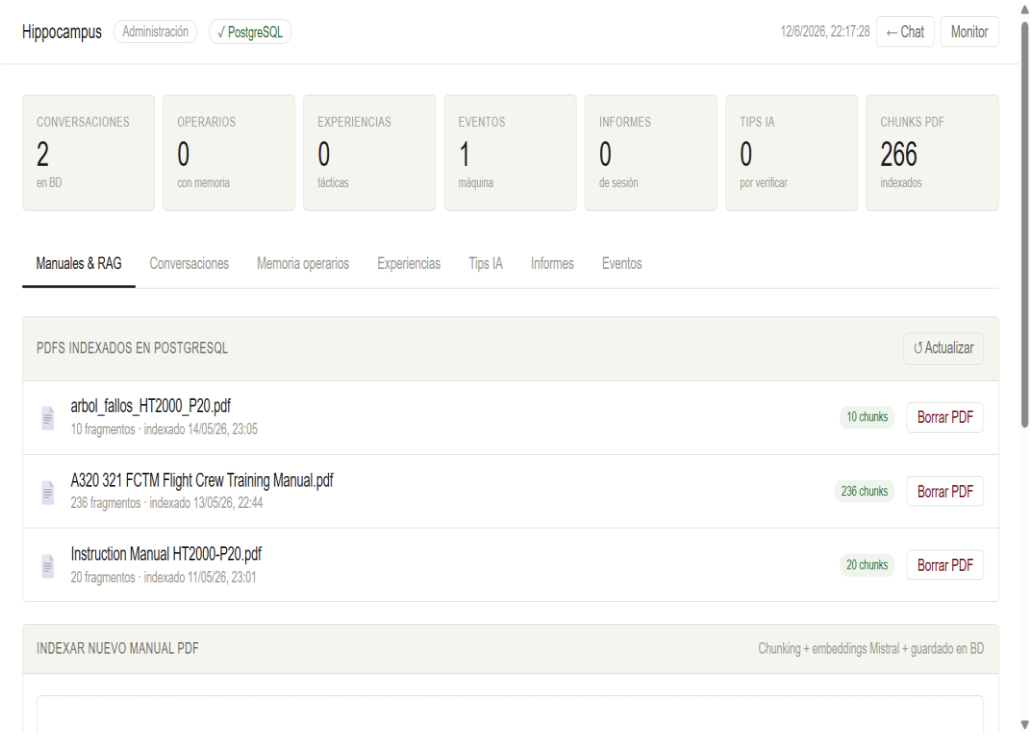


Figure 7-51 System Metrics Dashboard (Source: Own Elab.)

- *Knowledge Base Management: t*he manuals section is the core of the system's knowledge management.

  It allows the administrator to add new technical documents to the knowledge base via a drag-and-drop interface or file selection. In addition, each document has a delete button that allows the administrator to remove a manual from the base.

  The indexing process (chunking the document, generating embeddings via the Mistral API and storing them in PostgreSQL) is carried out entirely on the server, with no involvement from the administrator beyond selecting the file and optionally adding a description. A manual may be registered in the configuration but pending indexing, or indexed in PostgreSQL but not registered in the configuration if it was added later from the panel.

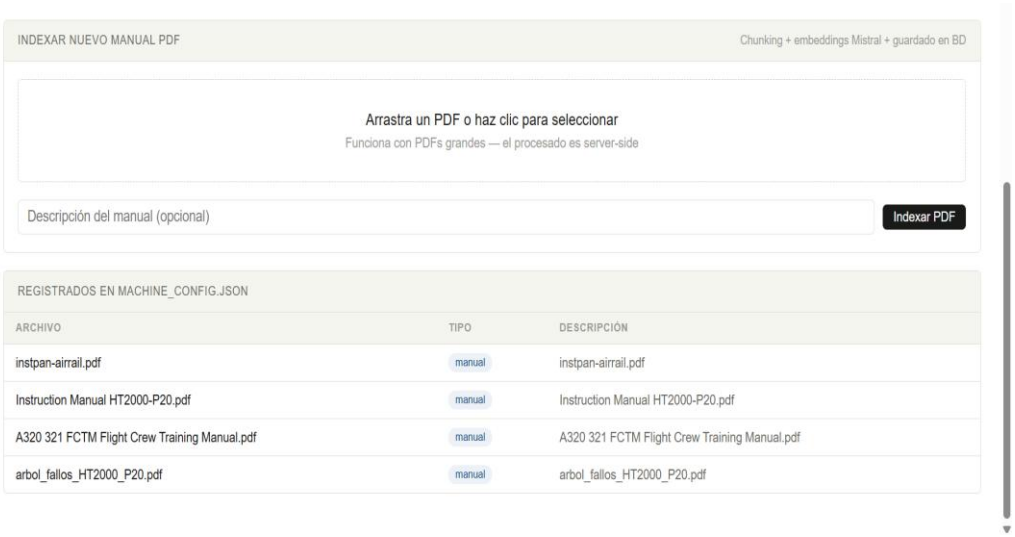


Figure 7-52 Knowledge Base Management Screen (Source: Own Elab.)

- *Operator Experience Log:* the Experiences section implements one of the most distinctive mechanisms of the HT2000-AI system: capturing the tacit knowledge of veteran operators.

  The add form collects four fields (operator name, years of experience, knowledge category and a free-text description of the tacit knowledge) and allows the administrator to formalize the undocumented know-how that experienced technicians have accumulated over years of operating the HT2000.

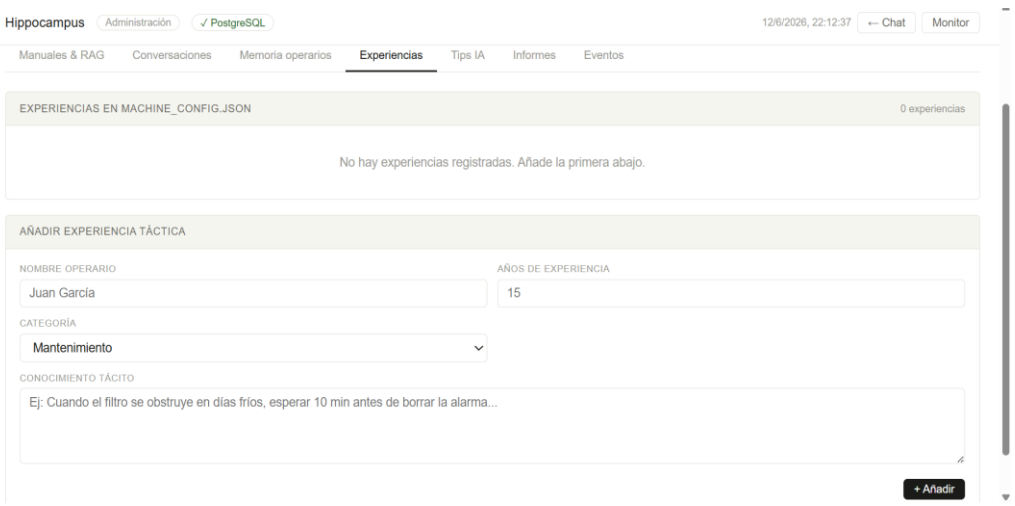


Figure 7-53 Operator Exper. Log Screen (Source: Own Elab.)

The text "When the filter clogs on cold days, wait 10 min before clearing the alarm..." illustrates exactly the type of knowledge this module is designed to capture: an empirical observation that does not appear in any manual but that an operator with 15 years of experience knows.

Once recorded, this experience is injected into the language model's system prompt and becomes a permanent part of the machine's base knowledge, accessible to any operator who consults the system on the morning, afternoon or night shift.

At the time documented in the screenshot, the panel showed 0 recorded experiences — the system had just been initialized after the data was wiped for the validation tests.

This confirms that the purge mechanism described above (the data wipe prior to the tests) removed all experiences recorded in previous sessions, leaving the system in its initial state for a clean evaluation.

7.11 Cross-Cutting Aspects

The figure shows some of the cross-cutting aspects discussed above: Physical Safety, Implemented Ethics and Data Protection.

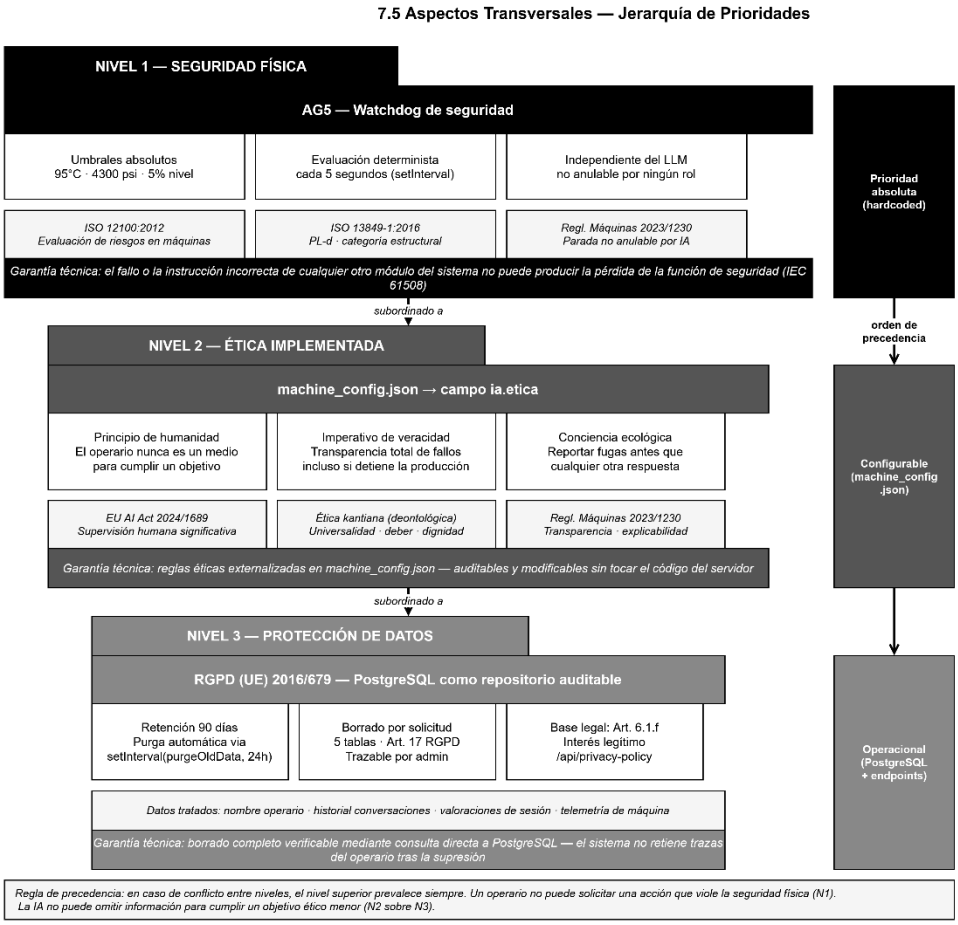


Figure 7-54 Cross-Cutting Aspects (Source: Own Elab.)

7.11.1 Physical Safety/System Integrity

- *Regulatory Framework:* the safety design of the HT2000-AI system is framed within four reference standards applicable to the operating environment.
    - *UNE-EN ISO 12100:2012:* establishes the general principles of safety in machine design, defining a risk assessment and reduction methodology based on three levels: risk elimination by design, technical protective measures and information-based protective measures. The HT2000-AI system acts mainly at the second and third levels: the safety watchdog is a technical protective measure, while the communication of alerts to the operator by voice and text is an information-based measure.
    - *UNE-EN ISO 13849-1:2016:* defines the requirements for safety-related parts of control systems, establishing Performance Levels

(PL) from PL-a to PL-e according to the probability of dangerous failure per hour. The safety watchdog implemented on the server evaluates the state of the sensors every five seconds completely independently of the language model, which is consistent with the structural category requirements of level PL-d, which demands that the failure of a single component must not lead to the loss of the safety function.

  - *IEC 62443:* establishes the cybersecurity requirements for networked systems in industrial environments. Although the current prototype does not implement all the controls required by this standard for production deployment, its architecture is consistent with several of its principles: network separation through an authenticated proxy, role-based access control, operation event logging and per-IP request rate limiting. Full implementation of IEC 62443 is identified as a prerequisite for deployment in a real production environment.
  - *European Union Machinery Regulation (Regulation 2023/1230):* introduces, for the first time, explicit requirements for control systems incorporating AI in industrial machinery. Among them, it requires the system to be predictable and transparent in its behavior, the operator to be able to supervise and correct autonomous decisions, and stop mechanisms to exist that cannot be overridden by the AI system. The HT2000-AI meets these three requirements by design: the step script generated by the AI is explainable and can be reviewed before execution, the operator confirms each action before it is executed, and the watchdog implements a safety stop that the language model cannot override under any circumstances.

- *Layered safety architecture:* the safety design follows a defense-in-depth approach, in which multiple independent layers must fail simultaneously for a dangerous action to be executed.
  - *The first line of defense is the safety watchdog*, which operates completely independently of the language model and cannot be disabled by any operator or any system role.

- *The second line is the server's action validator*, which checks the preconditions required by each action before relaying it to the panel.
- *The third line is the logic of the panel itself*, which rejects commands that are inconsistent with the current state of the machine.

Communications between client and server are protected by standard HTTP security headers, including a content type policy (X-Content-Type-Options), protection against script injection (X-XSS-Protection), frame restriction (X-Frame-Options) and a strict content policy (Content-Security-Policy) that limits the permitted script sources and connections.

- *Prototype security limitations and requirements for production deployment: t*he current prototype has three identified security limitations that should be resolved before any deployment in a real production environment.
  - The administration panel does not require authentication; any user with access to the local network can index manuals, add operator experiences or view the conversation history. In an aeronautical environment, where access to control systems is regulated, it would be essential to implement authentication through client certificates or integration with the organization's corporate directory.
  - The system depends on local network connectivity between the operator's browser and the server.

    On an aircraft assembly line, where electromagnetic interference and network restrictions are common, it would be necessary to assess the resilience of the WebSocket communication and define a degraded operation procedure for when connectivity is interrupted.
  - Integration with the real HT2000 PLC is not implemented in the prototype; the machine is simulated in software.

    A real connection via OPC-UA or Modbus would introduce a new attack surface that should be assessed in accordance with IEC 62443 before commissioning, including network isolation between the IT layer and the OT layer of the installation.

7.11.2 Implemented Ethics

This section describes how the ethical values (humanism, truthfulness, transparency, environment) have been put into practice:

- The principle of humanity is implemented through the explicit instruction in the prompt that the operator is always an end in themselves and never a means to complete a test. In addition, the model is forbidden from executing sequences that endanger the operator's physical integrity, even if the operator requests it directly.
- The principle of ecological awareness is implemented through instructions that require the model to report any sign of a leak before any other response, to suggest pauses when it detects running time without productive activity, and to mention the environmental cost of high-energy operations before executing them.
- The Kantian categorical imperative principle is implemented through the instruction that the model must behave identically with any operator, on any shift and at any experience level, without simplifying or complicating its behavior depending on who is in front of it unless the operator explicitly requests it. Technical transparency takes absolute priority over the convenience of the process.

These three ethical blocks are not coded in the server but in the machine_config.json file, in the ia.etica field, which allows them to be reviewed and updated without modifying the system code.

7.11.3 Data Protection and GDPR Compliance

The system processes personal data to a limited extent: operator name, conversation history and session ratings.

The processing is justified on the legal basis of legitimate interest in the safe maintenance of industrial equipment, in accordance with Article 6.1.f of the General Data Protection Regulation.

- *Data retention:* is limited to ninety days, after which conversation records and telemetry events are automatically deleted by the server itself through a periodic task.
- *Deletion of an operator's data:* is carried out directly on PostgreSQL by an administrator, a decision that responds to the need to guarantee traceability of the deletion and to avoid accidental deletions from the web interface.

The system exposes an informative endpoint /api/privacy-policy that lists the data processed, the legal basis, the retention period and the contact details of the data protection officer.

7.12 Roadmap to Production Deployment

7.12.1 Introduction

The HT2000-AI system developed in this project uses a complete functional prototype that simulates the electromechanical environment of the hydraulic machine in software.

Moving from the prototype to real production deployment involves replacing that simulation with the real HT2000 hardware (PLC, sensors and actuators).

This section describes the required hardware integration architecture, the applicable industrial communication protocols and the phased commissioning plan that would guarantee a safe transition to the production environment.

7.12.2 Integration with the Existing PLC

As documented in previous chapters, the HT2000 has a Siemens CPU313C programmable logic controller as its main control system, with analog and digital input and output modules that collect the signals from the pressure, temperature, flow and level sensors installed on the machine.

The PLC also has a CP343 communications module that provides Ethernet connectivity to the local network, allowing direct communication between the Node.js server and the PLC without the need for additional gateway hardware.

The communication protocol adopted for this integration is Siemens' native S7 protocol, implemented through the python-snap7 library.

This library allows variables in the PLC's Data Block (DB) to be read and written directly from Python code, without having to configure an OPC-UA server or purchase proprietary software licenses. The flows would be:

- *Output:* an intermediary Python script periodically reads the variables in the PLC's DB via snap7, serializes them in JSON format and publishes them on a local HTTP endpoint that the Node.js server queries to update the machineState object.
- *Input:* the actuation setpoints validated by the watchdog are sent to the Python script, which writes them to the corresponding PLC outputs.

```
import snap7
from snap7.util import get_real, get_bool
client = snap7.client.Client()
client.connect('192.168.1.10', 0, 1)  # IP, rack, slot CPU313C
# Read variables from the PLC's DB1
data = client.db_read(1, 0, 48)  # DB1, offset 0, 48 bytes
state = {
    'temp':         get_real(data, 0),   # MW0: oil temperature
    'presionAlta':  get_real(data, 4),   # MW4: high pressure (psi)
    'presionRetorno': get_real(data, 8), # MW8: return pressure (psi)
    'flujoAjustado': get_real(data, 12), # MW12: set flow
    'flujoReal':    get_real(data, 16),  # MW16: actual flow
    'nivel':        get_real(data, 20),  # MW20: reservoir level (%)
    'enMarcha':     get_bool(data, 24, 0), # Q0.0: motor running
    'alarmaTemp':   get_bool(data, 25, 0), # M10.0: temperature alarm
    'alarmaPresion': get_bool(data, 25, 1), # M10.1: pressure alarm
    'alarmaNivel':  get_bool(data, 25, 2)  # M10.2: level alarm
}
```

As a backup protocol for environments where the S7 protocol is not available or greater interoperability is required, the system can communicate via Modbus TCP, which is widely supported by Siemens hardware and by the Node.js mod-bus-serial library.

Modbus TCP provides neither variable semantics nor native security, so its use would require manual register mapping and an additional authentication layer on the server.

The following table summarizes the process variables that the server would read from and write to the PLC in a production deployment, with their addresses in the PLC's DB.

Writing to the PLC would be restricted to the variables marked as Read/Write, and only after validation by the server's safety watchdog and explicit confirmation by the operator, maintaining the safety chain described above at all times.

Variable	DB Address	Type	Direction	Description
Oil Temp.	DB1.DBD0	REAL	Read	Hydraulic temperature (°C)
High Pressure	DB1.DBD4	REAL	Read	High-pressure line pressure (psi)
Return Pressure	DB1.DBD8	REAL	Read	Return line pressure (psi)
Set Flow	DB1.DBD12	REAL	Read	Flow setpoint (l/min)
Actual Flow	DB1.DBD16	REAL	Read	Measured flow (l/min)
Reservoir Level	DB1.DBD20	REAL	Read	Reservoir level (%)
Motor Running	DB1.DBX24.0	BOOL	Read/Write	Main motor status
Power Sel.	DB1.DBX24.1	BOOL	Read/Write	Power on/off
Temp. Alarm	DB1.DBX25.0	BOOL	Read	High temperature alarm
Pressure Alarm	DB1.DBX25.1	BOOL	Read	Overpressure alarm
Level Alarm	DB1.DBX25.2	BOOL	Read	Low level alarm
Alarm Reset	DB1.DBX25.7	BOOL	Write	Reset of active alarms
Variable	DB Address	Type	Direction	Description

7.12.3 Complementary Sensorization

The HT2000 technical manual confirms that the machine has integrated sensors for the main parameters of the hydraulic process: hydraulic temperature TA1, high pressure PA2, return pressure PA1 and PA3, electrical level indicator 15P04, flow indicator FI1 and differential pressure sensors on the three filters.

All these parameters are already available in the PLC and are directly accessible via the S7 protocol, so they do not require any additional instrumentation.

However, the HT2000-AI system requires three additional variables for predictive maintenance and early anomaly detection that are not available in the current control system:

- *Vibration monitoring on pump and motor*: implemented using the Siemens SIPLUS CMS1200 SM128 system. This system connects directly to the CPU313C PLC and makes it possible to detect cavitation, bearing wear and misalignment through vibration signature analysis.
- *Pump noise capture:* using an IP65 industrial microphone that allows cavitation and water hammer to be detected through spectral analysis of the audio on the server. Unlike conventional analog signals, high-frequency audio signals cannot be processed directly in the PLC due to sampling limitations, so they are captured and processed on the industrial mini-computer using a data acquisition (DAQ) card before being sent to the server.
- *Monitoring of hydraulic fluid contamination:* using an oil particle sensor, aimed at compliance with the NAS standard 1638 Class 6 required in aeronautical maintenance. This sensor allows early detection of wear through suspended metal particles before it shows up as a pump or valve failure. Its necessity is supported by the HT2000 technical manual itself, which provides for periodic oil sampling for contamination analysis.

7.12.4 Digital Twin Architecture

The digital twin architecture is the natural evolution of the machineState object that the prototype keeps in memory.

In production deployment, that object would no longer be updated manually from the web panel and would instead reflect the physical state of the machine in real time through a continuous synchronization cycle with the PLC.

The synchronization cycle would work as follows:

- An intermediary Python script, based on python-snap7, would read the variables in the PLC's Data Block every 500 ms and publish the updated values on a local HTTP endpoint of the Node.js server.
- The server would update the machineState object and propagate the change to all connected clients through the Socket.IO machine_state event, exactly as in the current prototype.

The language model would receive the updated state with every query, ensuring that its answers are always based on the actual physical state of the machine and not on simulated values.

```
python
import snap7, requests, time
from snap7.util import get_real, get_bool
client = snap7.client.Client()
client.connect('192.168.1.10', 0, 1)
SERVER_URL = 'http://localhost:3000/api/plc-update'
while True:
    data = client.db_read(1, 0, 48)
    state = {
        'temp':          get_real(data, 0),
        'presionAlta':   get_real(data, 4),
        'presionRetorno': get_real(data, 8),
        'flujoAjustado':  get_real(data, 12),
        'flujoReal':     get_real(data, 16),
        'nivel':         get_real(data, 20),
        'enMarcha':      get_bool(data, 24, 0),
        'alarmas': {
            'altaTemp':  get_bool(data, 25, 0),
            'presion':   get_bool(data, 25, 1),
            'nivel':     get_bool(data, 25, 2)
        }
```

```
    }
    requests.post(SERVER_URL, json=state)
    time.sleep(0.5)  # 500 ms cycle
```

This architecture turns the server's machineState into the digital twin of the HT2000: a virtual model synchronized with the physical machine that the language model can query, analyze and reason about in natural language.
The digital twin not only reflects the current state but also accumulates the operating history in PostgreSQL, allowing the system to detect degradation trends and anticipate failures through the RUL engine before they turn into breakdowns.

7.12.5 Phased Commissioning Plan

Commissioning the system in a real production environment would follow a four-phase plan designed to minimize operational risk during the transition from the simulated prototype to the system connected to the physical machine:

1. *Passive Reading*: this would consist of connecting the server to the PLC in read-only mode, without any ability to write to the actuators. During this phase, the HT2000-AI system would operate in parallel with the existing control system, receiving the real sensor values and feeding them to the language model, but without executing any physical action. The goal is to validate the correct reading of all variables, adjust the watchdog thresholds to the machine's real operating values and detect deviations between simulated and real behavior. The estimated duration of this phase is two to four weeks.
2. *Supervised Writing:* this would enable writing to the PLC for a limited subset of low-risk actions (clearing alarms and adjusting flow setpoints), always with explicit operator confirmation and with a technician present. During this phase, the complete safety chain would be validated: watchdog, role validation, operator confirmation and OPC-UA writing to the PLC. The estimated duration is two to three weeks.

3. *Assisted operation:* this would extend writing to the full set of actions defined in machine_config.json, maintaining supervision by a technician during the first test cycles of each new action. The language model would begin generating operating scripts based on the real state of the machine and the history accumulated in PostgreSQL.
4. *Supervised autonomous operation:* this would be the production deployment state, in which the operator interacts with the HT2000-AI system as the main assistance tool during hydraulic tests, with the PLC's original control system acting as an independent backup safety layer.

8. FINAL RESULTS

8.1 Introduction

This chapter presents the practical validation of the capabilities described in the previous chapter through a set of functional tests run on the HT2000-AI prototype in its real local deployment configuration.

Each test was designed to verify in isolation one of the modules described in the previous chapter (server startup, RAG engine, role-based access control, adaptive memory, safety watchdog, among others), so that the result obtained can be unambiguously attributed to the module under test and not to an uncontrolled interaction between several subsystems.

The tests were run in the environment described in the “Installation Guide” annex, with the server running on Node.js, PostgreSQL as the sole persistence layer and the PG_URL environment variable configured to point to the hycom database.

For each test, the procedure followed, the observed result and the assessment of compliance with the expected behavior described in chapter 7 are documented.

8.2 Results and Measurements

8.2.1 Test 1 — Server Startup

- *Objective:* verify that the server starts correctly, establishes a connection with PostgreSQL, loads the machine configuration from machine config.json, detects the optional services (Ollama, Piper TTS) and leaves the system ready to accept connections.
- *Procedure:* on Windows, go to the project directory, set the PG_URL environment variable to the connection string for the hycom database, and run node server.js.
- *Observed result:* the console shows the complete initialization sequence described in the startup sequence diagram:

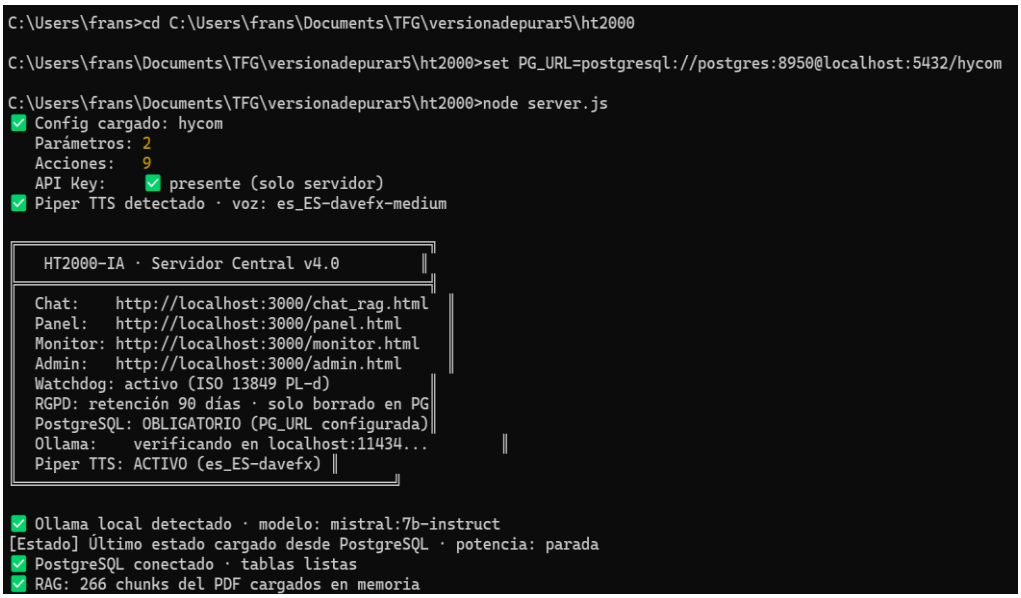


Figure 8-1 Server Startup (Source: Own Elab.)

- *Assessment:* the startup sequence runs exactly in the order defined in the corresponding UML diagram:
    - The machine configuration and Piper TTS detection precede the services banner.
    - The Ollama check is performed in parallel without blocking startup, confirming the principle of graceful degradation.
    - The connection to PostgreSQL is confirmed as mandatory, consistent with the fail-fast design for critical dependencies.
    - Loading the 266 RAG chunks into memory confirms that the knowledge base is available for queries without having to access the database on every request (RAG Engine).

  The result of this test is considered PASS: the server starts without errors, all mandatory and optional services are correctly initialized, and the system is operational in less than five seconds from running the command.

8.2.2 Test 2 — Operator Identification/Authentication

- *Objective:* verify that the system correctly identifies different operators by name and four-digit PIN, that the PIN is permanently registered on each operator's first access, and that the mechanism allows differentiated and

consistent access for multiple operators without interference between their profiles.

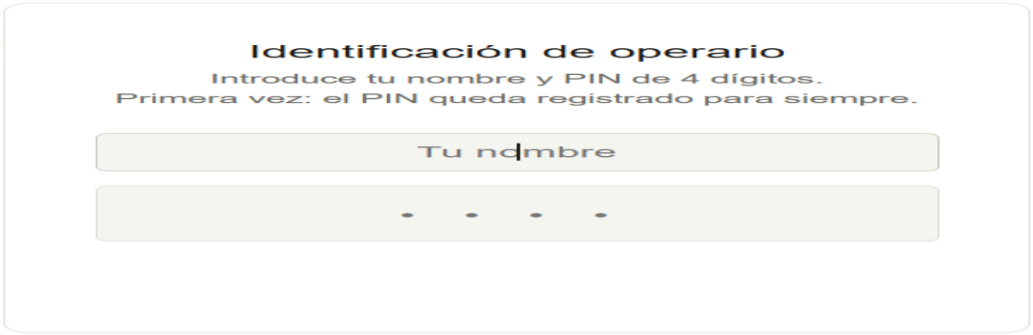


Figure 8-2 Operator Identification/Authentication (Source: Own Elab.)

- *Procedure:* chat_rag.html was accessed with several different operator names, entering a different four-digit PIN in each case. For each operator, access was repeated in later sessions with the same name and the PIN already registered.

- *Observed Result:* access was successful with different operator names, each with their own PIN, and in all cases access was completed without errors. When access was repeated with the same name and the already registered PIN, the system recognized the operator without requesting a new registration, confirming that the PIN is permanently associated with the name from the first access.
- *Assessment:* the result is considered PASS.
  The identification mechanism works correctly for multiple operators simultaneously and independently, without the registration of a PIN for one operator interfering with that of another. Correct differentiation of PINs per operator is also the necessary condition for the adaptive memory module to associate each accumulated session profile with the real operator who generated it, without mixing the history of different people.

8.2.3 Test 3 — Session Loading and Operator Welcome

- *Objective:* verify that, after the operator is identified, the chat interface correctly loads the machine configuration, the RAG index and the assigned

role, and recognizes whether this is the operator's first session, communicating this entire process transparently before starting the conversation.

- *Procedure:* after logging in as operator "Fran" on the access screen, the startup sequence of the chat interface is observed until it reaches its idle state.

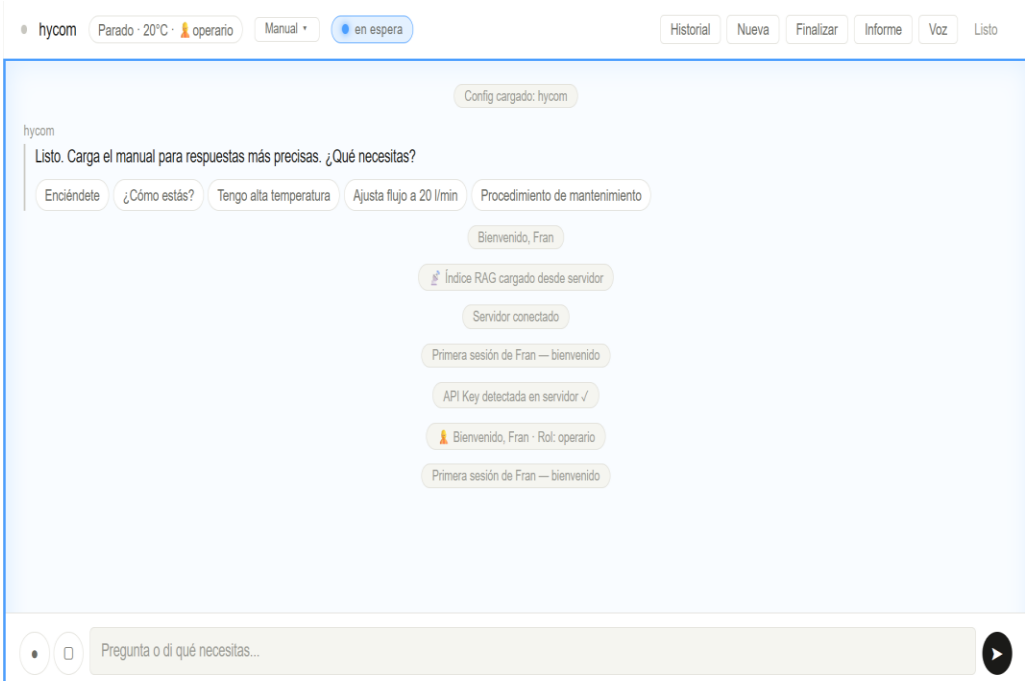


Figure 8-3 Test 3 - Session Loading/Welcome (Source: Own Elab.)

- *Observed Result:* the interface shows the complete initialization sequence in an orderly manner in the central area:
  - Confirmation that the configuration has loaded ("Config loaded: hycom").
  - Loading of the RAG index from the server.
  - Confirmation that the connection has been established.
  - Detection of the operator's first session
  - Confirmation that the API Key has been detected on the server,
  - Role assignment ("Welcome, Fran · Role: operator").

  The top bar permanently communicates the overall state of the machine (system name "hycom", status "Stopped" at 20 °C, operator role icon, "Manual" mode and "standby" status) without the operator needing to check it actively.

The personalized welcome message ("Ready. Load the manual for more accurate answers. What do you need?") is accompanied by the shortcuts "Turn on", "How are you?", "I have high temperature", "Set flow to 20 l/min", "Maintenance procedure".

- *Assessment:* the result is considered PASS.

  The observed startup sequence is consistent with the initialization transparency described in the previous chapter.

  The correct detection of "first session" also confirms that the adaptive memory module correctly distinguishes between new and returning operators, a necessary condition for personalizing the level of detail of answers according to the accumulated history.

  The visible assignment of the "operator" role confirms that the RBAC system correctly resolves the default role for a name not explicitly registered in the technician or administrator lists in machine_config.json.

8.2.4 Test 4 — RAG Engine and Operator-Based Adaptation

- *Objective:* verify that the AI bases its answers on the actual content of the technical manual through the RAG engine, citing exact sections and component codes, and that the level of detail and tone of the answer adapt to the profile of the operator identified as new or with a low inferred technical level.

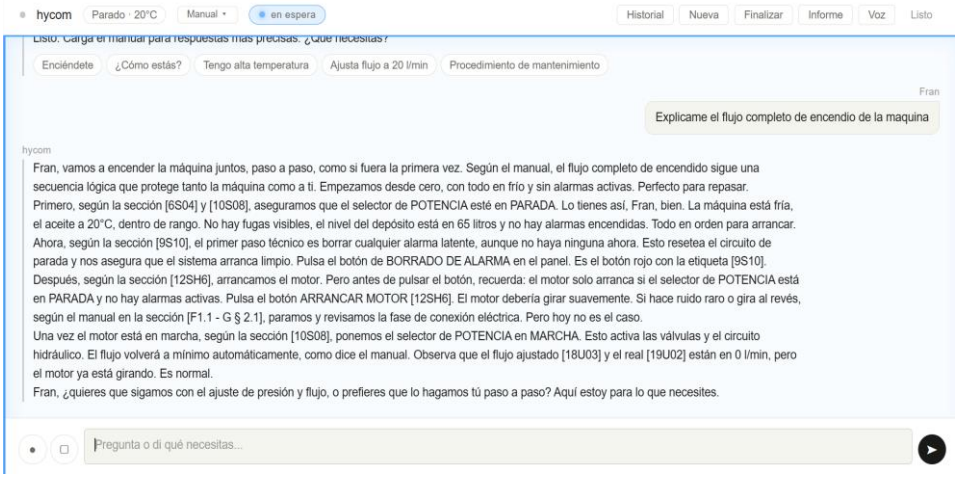


Figure 8-4 Test 4 - RAG Engine (Source: Own Elab.)

- *Procedure:* logged in as operator "Fran" in a session where the system has already detected a first session, the open query "Explain the complete machine startup flow to me" is entered.

- *Observed Result:* the AI responds with an extensive, instructive explanation, structured step by step "as if it were the first time", which explicitly cites six component codes from the manual in square brackets: [6S04], [10S08], [9S10], [12SH6], [18U03], [19U02], plus a cross-reference to the troubleshooting section of the manual [F1.1 - G 2.1] in case the motor turns the wrong way.

  The answer does not merely list steps but explains the safety reasoning behind each one: why alarms are cleared before starting, why the motor only starts with the power selector in the off position, and which behavior is normal and which is abnormal.

  At the end, the AI proactively asks the operator whether they want to continue with the pressure and flow adjustment step by step or autonomously, leaving the initiative in their hands.
- *Assessment:* the result is considered PASS.

  The literal and exact citation of six component codes and a section reference from the troubleshooting manual confirms that the RAG engine correctly retrieves the relevant fragments of the Instruction Manual HT2000-P20 through the hybrid cosine similarity and BM25 search described in that section, and that the language model integrates them into its reasoning instead of generating unverified technical content.

  The educational tone, the extended explanation of the reason for each step and the "step by step, as if it were the first time" pace are consistent with the expected behavior for an operator in their first session according to the adaptive memory module: a new operator receives more detailed explanations, compared with the more direct treatment a veteran operator with an accumulated history would receive.

  The final question offering control over the pace of the interaction is also consistent with the human oversight principle required by Machinery Regulation 2023/1230: the operator decides at each moment how much autonomy to grant the AI, rather than the system assuming it by default.

8.2.5 Test 5 — Execution of Actions on the Machine

- *Objective:* verify the complete execution cycle of an action requested by the operator: generation of the step script by the AI, safety explanation prior to execution, explicit operator confirmation, sequential execution of the actions and real-time propagation of the new state to the physical control panel.

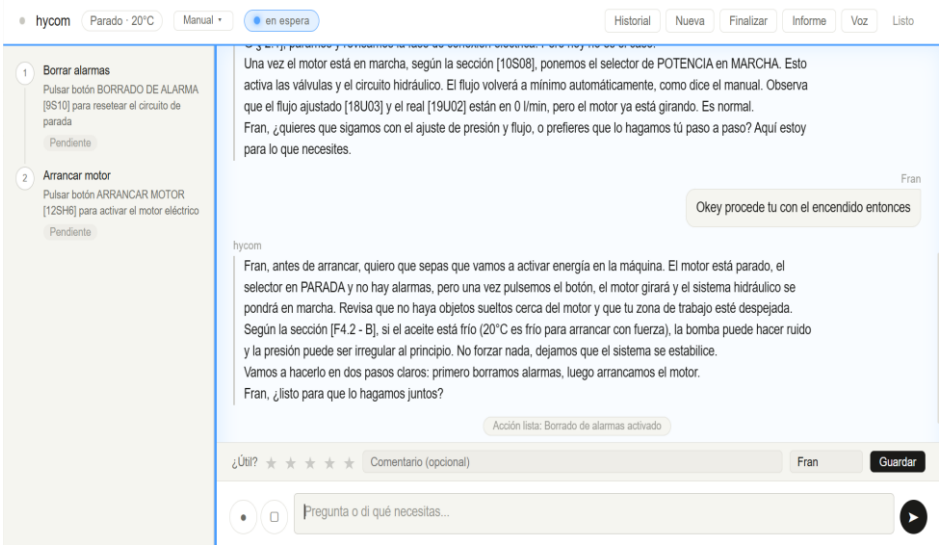


Figure 8-5 Test 5 - Action Execution (1) (Source: Own Elab.)

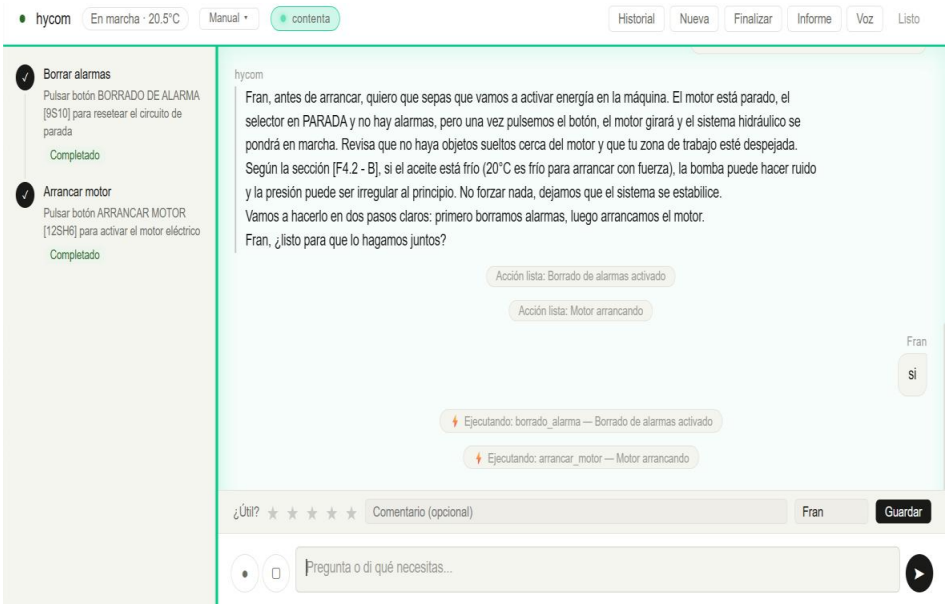


Figure 8-6 Test 5 - Action Execution (2) (Source: Own Elab.)

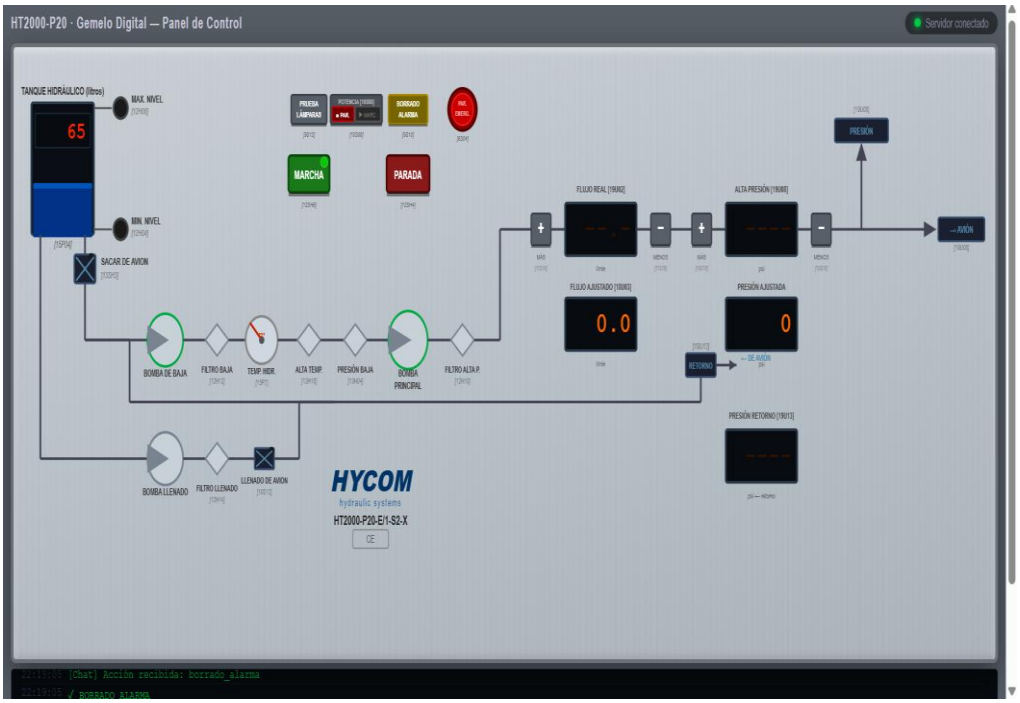


Figure 8-7 Test 5 - Action Execution (3) (Source: Own Elab.)

- *Procedure:* after the initial query about the startup flow, operator Fran requests "Okay, you go ahead with the startup then". The AI's response, the operator's confirmation and the result on the panel are observed.
- *Observed result:* the AI generates a two-step script, visible in the left side panel and initially marked as "Pending":
    - Clear alarms (button [9S10], resets the stop circuit)
    - Start motor (button [12SH6], activates the electric motor),

  Before executing, the AI explains the relevant safety conditions in natural language: no loose objects near the motor, a clear work area and the manual's warning [F4.2 - B] about the pump's behavior with cold oil.

  The AI explicitly asks "ready for us to do it together?" before proceeding.

  After the operator confirms ("yes"), the side panel updates both steps to "Completed" with a check mark, the chat shows the execution messages in real time ("Executing: borrado_alarma Alarm reset activated", "Executing: arrancar_motor Motor starting"), and the top bar reflects the change of state from "Stopped · 20°C" to "Running · 20.5°C" with the "happy" emotional indicator.

  The physical control panel (panel.html) confirms the same state: the POWER selector on RUN, the main pump with its active rotation indicator,

and the panel's console log recording "[Chat] Action received: borrado_alarma" followed by "✓ ALARM RESET" with their timestamps.

- *Assessment:* the result is considered PASS.

  The complete sequence shows that the cycle described in the interaction sequence diagram runs exactly as designed: the AI does not execute any action without explicit confirmation from the operator, meeting the human oversight requirement of Machinery Regulation 2023/1230.

  The simultaneous and consistent update of the chat, the step script and the physical control panel confirms that the machineState object correctly acts as the single source of truth propagated by Socket.IO to all connected clients (Machine state management).

  The timestamped entry in the panel's console log is also the operational evidence that each executed action is traced, a necessary condition for the auditing required under Data Protection and GDPR Compliance and for compliance with EASA Part-145 in an aeronautical maintenance environment.

8.2.6 Test 6 — Role-Based Access Control

- *Objective:* verify that the RBAC system correctly blocks the execution of an action not permitted for the operator's role, clearly communicating the reason for the block both in the step script and in the response message, without the AI ever generating a false confirmation of success.
- *Procedure:* logged in as operator "Fran" with the operator role (not registered in the technician or administrator lists in machine_config.json), with the machine already running, the request "turn the run power on" is made.

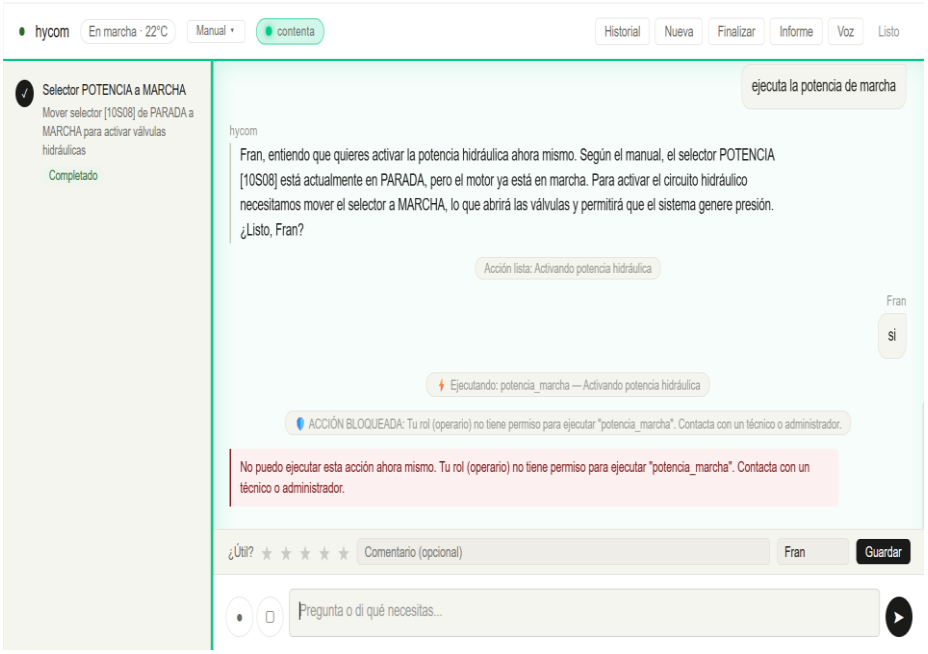


Figure 8-8 Test 6 - Access Control (1) (Source: Own Elab.)

- *Observed Result:* the AI generates the corresponding step in the side script, "POWER selector to RUN: Move selector [10S08] from OFF to RUN to activate hydraulic valves", and explains in the chat the action to be performed and its technical effect (valve opening, pressure generation), asking "Ready, Fran?".

  After the operator confirms ("yes"), the system displays the message "Executing: potencia_marcha Activating hydraulic power", immediately followed by the warning " ACTION BLOCKED: Your role (operator) does not have permission to execute 'potencia_marcha'. Contact a technician or administrator.", also shown as an error message in red in the body of the chat.

  The step appears marked as "Completed" in the side panel with a check mark, even though the action was blocked.

- *Assessment:* the result is considered PASS in terms of safety validation, with an interface inconsistency to be corrected.

  Blocking the action is correct and matches exactly the design described in the role-based access control section. potencia_marcha is not included in the operator role's permission list defined in ROLE_PERMISSIONS, which only covers arrancar_motor, parar_motor, borrado_alarma, llenado_avion and sacar_avion, whereas the requested action requires the technician

role or higher. The rejection message is clear, identifies the current role and the denied action, and indicates the correct escalation path (contact a technician or administrator), meeting the transparency principle required by Machinery Regulation 2023/1230.

However, a visual inconsistency is observed: the step in the side script is marked as "Completed" with a green check mark, when in reality the action was rejected by the server and was never executed on the machine.

This could lead an inattentive operator to believe that the action was carried out.

It is recommended that this observation be documented as a future improvement: the status of the step in the script should show "Blocked" or "Denied" instead of "Completed" when the endpoint /api/validate-action returns allowed: false, so that the visual interface is consistent with the actual result of the validation by the watchdog and the role system described above.

This sequence, with the technician/admin role successfully executing the same action that was previously blocked, is the perfect contrasting test. I am adding it as an extension of Test 6, not as a separate test, because it demonstrates the same mechanism from the other side.

- *Counter-test* — authorization with technician/administrator role.

  To confirm that the observed block responds specifically to the operator's role and not to a general system failure, an equivalent sequence was repeated logged in with a name registered with the technician or administrator role in machine_config.json.

  The request "turn on the machine" was made via text chat.

  The AI generated a three-step script: Clear alarms [9S10], Start motor [12SH6] and Activate POWER selector [10S08], first explaining the safety conditions verified (no active alarms, selector on OFF, reservoir level within range, required PPE: nitrile gloves and safety glasses) and asking "Shall we start, Fran?". After confirmation, the three steps were executed sequentially and marked as "Completed", this time including the power activation that had been blocked for the operator role in the previous test. The

physical control panel confirmed the real change of state: the POWER indicator switched from OFF to RUN (with the green "RUN" button active) and the green light of selector [10S08] came on, reflecting the correct propagation of the execute_command event validated by the server.

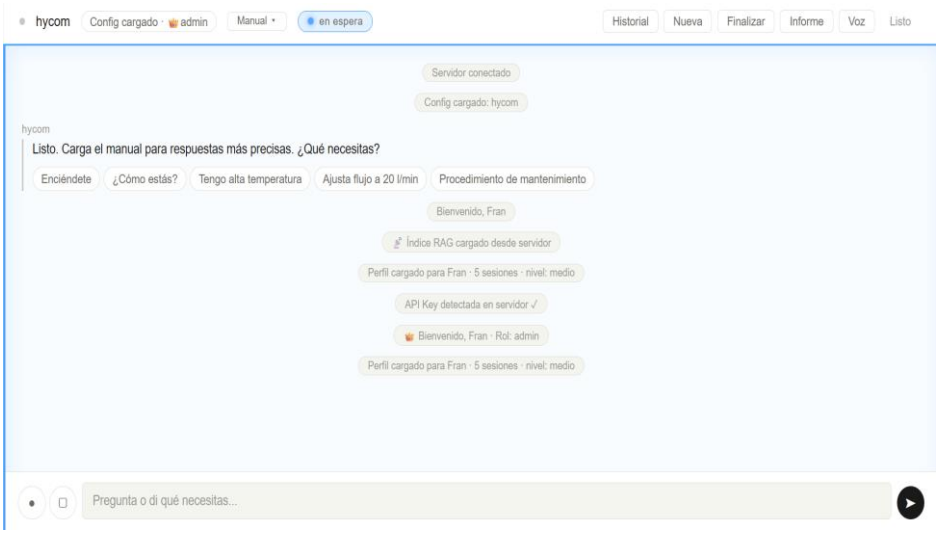


Figure 8-9 Test 6 - Access Control (2) (Source: Own Elab.)

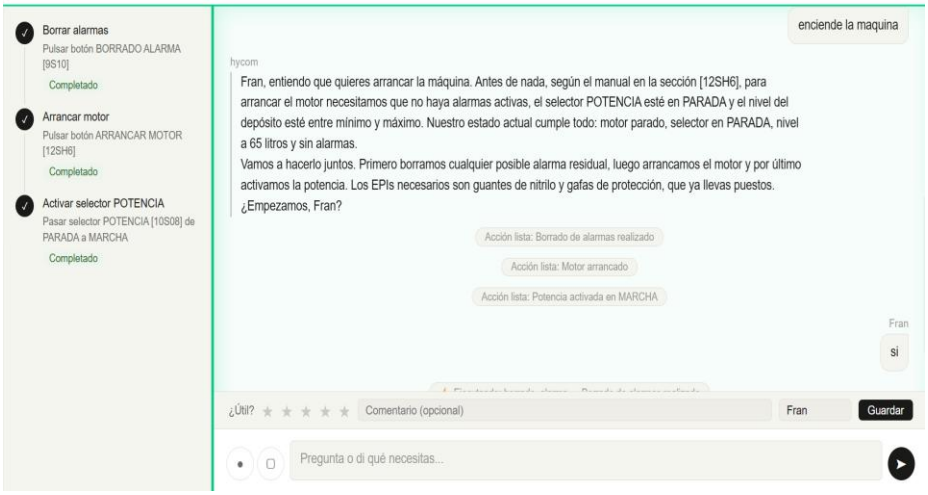


Figure 8-10 Test 6 - Access Control (3) (Source: Own Elab.)



Figure 8-11 Test 6 - Access Control (4) (Source: Own Elab.)

This same action (power activation) was also verified through the voice channel: with the operator identified with the technician role, the equivalent

spoken command was processed by the speech recognition system, transcribed and executed following the same validation flow as text input, confirming that role-based access control is applied uniformly regardless of the input channel (text or voice) used by the operator.

- *Extended assessment:* the result confirms that the block in Test 6 is not a system failure but the correct and intended behavior of role-based access control: the same action (potencia_marcha) that was rejected for the operator role is executed without incident for a role with sufficient privileges, and the change is reflected identically in the chat, the step script and the physical control panel, regardless of whether the command arrived by text or by voice.

  This validates end to end the authorization chain described in ROLE_PERMISSIONS: the technician/administrator role includes potencia_marcha and potencia_parada in its permission list, while the operator role does not, and the server applies this distinction consistently in the endpoint /api/validate-action regardless of the input channel.

  However, the interface correction noted above regarding the "Completed" status incorrectly shown when an action is blocked is still pending.

8.2.7 Test 7 — Reasoning about State Dependencies

- *Objective:* verify that the AI is able to detect that an action cannot be executed because a precondition is not met, communicate this correctly to the operator, and redirect the complete sequence of necessary steps in the correct order before completing the originally requested action.
- *Procedure:* with the motor previously stopped (after a previous parar_motor and potencia_parada action), operator Fran directly requests "increase the pressure and flow", without first starting the motor.

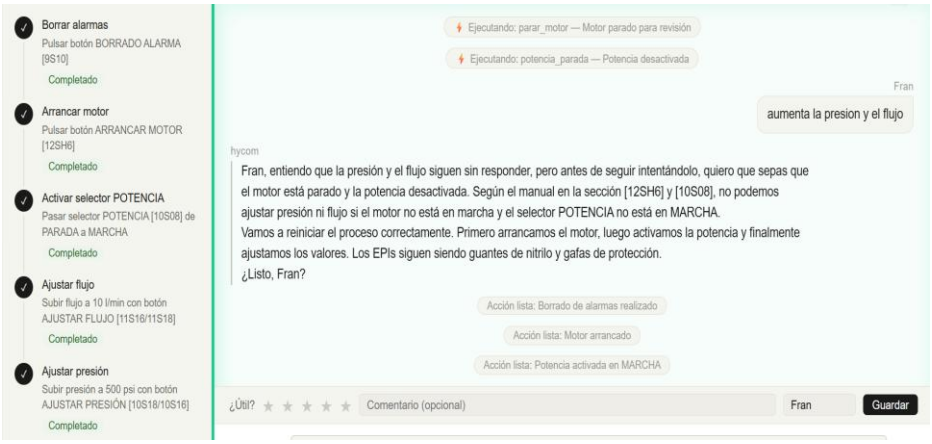


Figure 8-12 Test 7 - State Dependencies (1) (Source: Own Elab.)

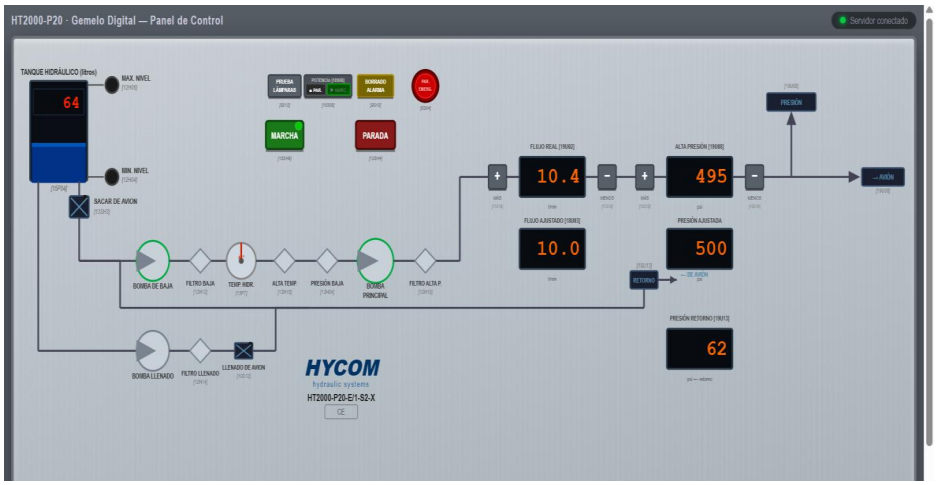


Figure 8-13 Test 7 - State Dependencies (2)

- *Observed result:* the chat first shows the execution messages of the previous stop sequence ("Executing: parar_motor Motor stopped for inspection", "Executing: potencia_parada Power deactivated").

  Faced with the new request to increase pressure and flow, the AI does not attempt to execute the action directly or generate a generic error: instead, it explains to the operator that "the pressure and flow are still not responding" and reasons out the cause, citing the manual in sections [12SH6] and [10S08]: it is not possible to adjust pressure or flow if the motor is not running and the POWER selector is not on RUN.

  The AI then proposes restarting the process correctly, generating a five-step script in the correct dependency order:

  - Clear alarms.
  - Start motor.
  - Activate POWER selector.

- Set flow to 10 l/min.
- Set pressure to 500 psi.

After the operator confirms, the five steps are executed sequentially and marked as "Completed".

The physical control panel confirms the final result: Actual Flow 10.4 l/min, High Pressure 495 psi, Set Flow 10.0 l/min, Set Pressure 500 psi and Return Pressure 62 psi, with the reservoir level at 64 liters and both pumps showing their active rotation indicator.

- *Assessment:* the result is considered PASS.

  This behavior shows that the RAG engine does not merely retrieve information to answer descriptive queries, but actively applies it when reasoning about the activation conditions of each action.

  The ability to identify the root cause of an execution failure (motor stopped) instead of returning a generic error, and to automatically generate the complete corrective sequence in the correct dependency order, is consistent with the explainability principle required by Machinery Regulation 2023/1230: the operator understands at all times why an action was not executed and what steps are needed for it to be executed.

  The final values observed on the panel (flow and pressure matching the requested setpoints, with the minimal deviation between actual and set value typical of any real hydraulic system) also confirm that the complete chain from the chat to the physical panel, through validation by the watchdog and the role system, runs end to end without loss of information or state inconsistency between the two interfaces.

  This combination is the key test of the proactive predictive alert: the AI detects the temperature trend and warns before the threshold is reached, while the monitor confirms the same data with the 2-minute prediction.

8.2.8 Test 8 — Proactive Predictive Temperature Alert

- *Objective:* verify that the system proactively generates, without any request from the operator, a predictive alert when it detects a rising temperature trend that would reach the warning threshold within a given time, and

that this prediction is consistent with the telemetry data shown on the real-time monitor.

- *Procedure:* after completing the startup sequence, power activation and adjustment of flow to 10 l/min and pressure to 500 psi, the machine is kept running under these test conditions without the operator making any further queries. The spontaneous behavior of the chat is observed and compared with the simultaneous state of the telemetry monitor.

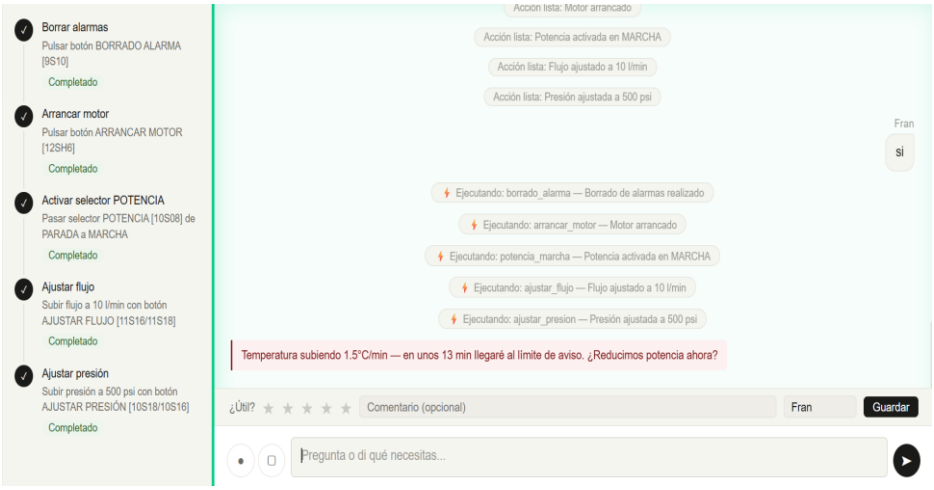


Figure 8-14 Test 8 - Predictive Alert (1) (Source: Own Elab.)

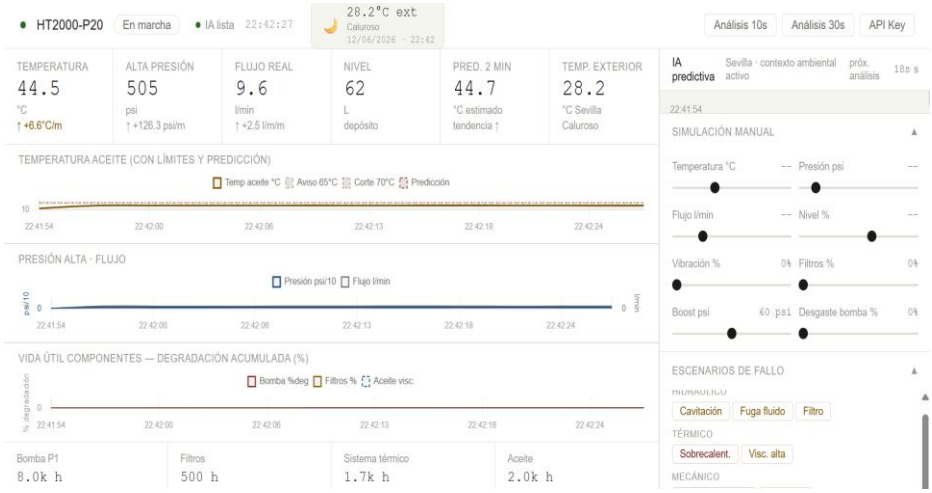


Figure 8-15 Test 8 - Predictive Alert (2) (Source: Own Elab.)

- *Observed result:* after confirmation and sequential execution of the five steps (Clear alarms, Start motor, Activate POWER selector, Set flow to 10 l/min, Set pressure to 500 psi), the system generates, without the operator requesting it, the warning: "Temperature rising 1.5°C/min — in about 13 min I will reach the warning limit. Shall we reduce power now?".

At the same time, the telemetry monitor (monitor.html) shows the real values underpinning that prediction: Temperature 44.5 °C with a "↑6.6°C/m" trend, High Pressure 505 psi "↑126.3 psi/m", Actual Flow 9.6 l/min, Level 62 L, and the 2-minute Prediction block estimating 44.7 °C with an upward trend.

The active environmental context (Seville, hot, 28.2 °C outside) and the "Predictive AI active" indicator with an 18-second countdown to the next analysis confirm that the prediction module operates continuously and independently of the operator's interaction in the chat.

- *Assessment:* the result is considered PASS.

  The spontaneous generation of the alert, without the operator asking any question about temperature, confirms that the system implements proactive and not merely reactive behavior, consistent with the principles of ecological awareness and resilient safety defined in machine_config.json.

  The AI is instructed to warn of risk trends before they materialize, putting process safety ahead of the convenience of not interrupting the operator.

  The consistency between the rate of increase mentioned in the chat (1.5 °C/min) and the numerical data shown on the monitor at the same moment (instantaneous rate ↑6.6 °C/m, 2-minute prediction of 44.7 °C) confirms that both interfaces consume the same telemetry source and the same prediction model, preventing the operator from receiving contradictory messages between the chat and the supervision panel.

  This anticipation capability, with a warning margin of thirteen minutes before the limit is reached, is what functionally justifies the predictive maintenance (RUL) module described in section 7.3.3, whose three accumulated degradation cards (Pump P1 with 8.0k hours, Filters with 500 hours, Thermal system with 1.7k hours) can also be seen at the bottom of the monitor as evidence of continuous tracking of the service life of the main components.

8.2.9 Test 9 — Critical Watchdog Alert on Threshold Exceedance

- *Objective:* verify that the system correctly escalates from a predictive alert to a critical alert when the temperature exceeds the automatic cut-off

threshold, that the generated message identifies the actual underlying fault (not just the temperature symptom) and sets an intervention deadline, and that the system is able to confirm subsequent normalization once the situation has been resolved.

- *Procedure:* continuing the session from Test 8, in which the predictive rising-temperature warning had already been generated, an additional temperature increase is forced from the monitor's Manual Simulation panel until the automatic cut-off threshold of 70 °C is exceeded.
- *Observed result:* the overall status in the top bar changes from "Running · 34.1°C" with a neutral indicator to the "on alert" status in red.

  Following the previous predictive warning, the chat shows a second, more severe alert: "Oil temperature 84.5°C exceeds automatic cut-off (70°C). Pressure 0 psi and flow 0 l/min indicate critical pump failure. Motor must stop immediately to prevent thermal damage. Maximum urgency: technical intervention in less than 30 min.".

  The telemetry monitor records the same message in its event log with the exact timestamp 22:44:52 in a red block. Fifteen seconds later, at 22:45:07, the monitor automatically records the resolution confirmation, "Situation normalized — parameters within correct range", in a green block.

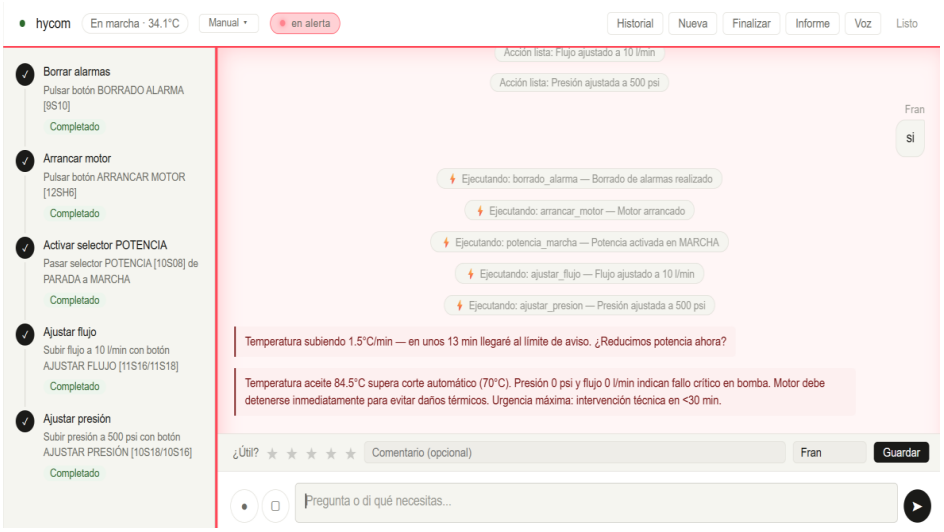


Figure 8-16 Test 9 - Critical Alert (1) (Source: Own Elab.)

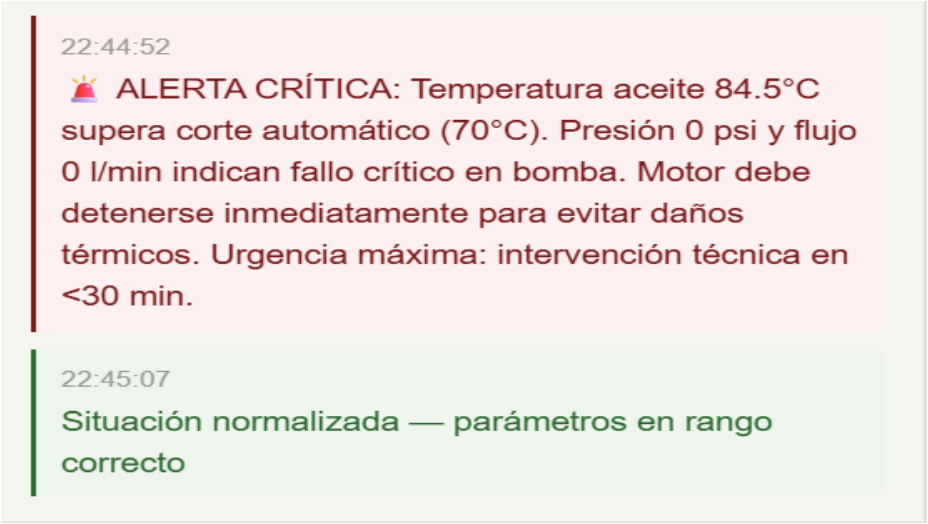


Figure 8-17 Test 9 - Critical Alert (2) (Source: Own Elab.)

- *Assessment:* the result is considered PASS.

  The observed progression from a trend-based predictive warning (Test 8) to a critical alert based on the actual exceedance of an absolute threshold confirms that the system correctly implements the two levels of supervision described in the safety Watchdog section.

  These are the predictive level, which anticipates risk before it materializes, and the deterministic level of the watchdog itself, which acts when the exceedance of an absolute limit is confirmed (95 °C according to the hard limits defined in WATCHDOG_LIMITS, although in this case the 70 °C automatic cut-off corresponds to the warning threshold configured specifically for the hydraulic oil).

  The message does not merely report the temperature value but correlates three variables (temperature, pressure at 0 psi and flow at 0 l/min) to diagnose the probable root cause (critical pump failure) rather than just the surface symptom, which shows that the system applies diagnostic reasoning and not a simple threshold comparison.

  The explicit intervention deadline ("less than 30 min") is consistent with the requirement for actionable alert communication in Machinery Regulation 2023/1230.

  The automatic confirmation of normalization fifteen seconds later, with its own timestamp, closes the supervision loop completely and traceably: both events (alert and resolution) are recorded in PostgreSQL with their

exact timestamps, providing the maintenance system with the documentary evidence needed for subsequent incident analysis in accordance with EASA Part-145.

8.2.10 Test 10 — Persistence and Traceability in PostgreSQL

- *Objective:* verify that all system interactions (conversations, machine telemetry events) are correctly recorded and can be consulted from the administration panel, confirming that PostgreSQL operates as the sole persistence layer.
- *Procedure:* after the test session, which included multiple chat interactions, action executions and the alert escalation from tests 7 to 9, the administration panel (admin.html) is accessed with the administrator role and the Conversations tab is consulted.

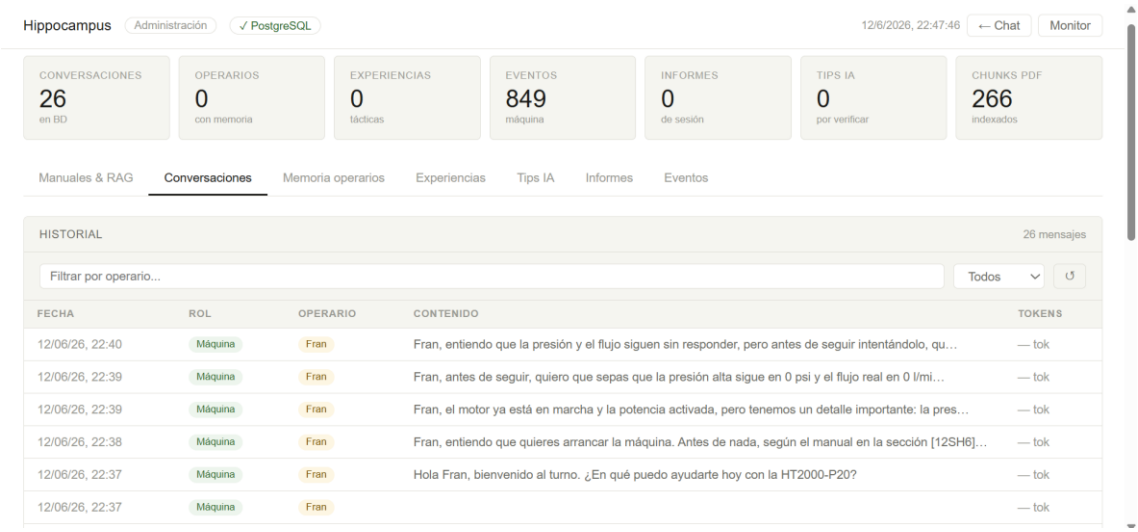


Figure 8-18 Test 10 - Persistence

- *Observed result:* the administration panel's metrics dashboard, for the "Hippocampus" installation with an active PostgreSQL connection confirmed, shows on 12/06/2026 at 22:47:46 a total of 26 conversations in the database, 0 operators with generated memory, 0 tactical experiences, 849 recorded machine events, 0 session reports, 0 AI tips pending verification and 266 indexed PDF chunks.

  The Conversations tab lists the complete history with 26 messages, showing for each entry the exact date and time, the sender's role (Machine or Operator), the operator's name (Fran) and the content of the message, including the messages corresponding to the reasoning sequence about

state dependencies observed in Test 7 ("Fran, I understand that the pressure and flow are still not responding...", "Fran, the motor is already running and the power is on...").

- *Assessment:* the result is considered PASS.
  The high number of recorded machine events (849) compared with conversations (26) is consistent with the design described above.
  Telemetry is recorded automatically every ten seconds while the machine is running, generating a far greater volume of events than conversation turns, which are only recorded when the operator or the AI sends a message.
  The exact match between the messages shown in the chat during the previous tests and the records retrieved from PostgreSQL confirms that there is no loss of information between real-time interaction and its persistence, and that the administration panel accesses the same data source that feeds the rest of the system, without the need for additional synchronization.
  The operator filter field and the column of tokens consumed per message also confirm that the database stores sufficient metadata for usage and cost analysis per operator, which is relevant for possible internal charging for the service in a multi-shift production deployment.
  It is noted that the "Operators with memory", "Tactical experiences" and "Session reports" counters show a value of 0 in this screenshot, which suggests either that the documented test session was run without completing the formal session close (the action that triggers summary generation and profile update) or that the system was reinitialized for these tests with a prior purge of those specific tables, while Conversations and machine_events did accumulate the data from the session in progress.

8.2.11 Test 11 — Fault Diagnosis/Verification

- *Objective:* verify that the AI does not uncritically accept an operator's statement about the state of the machine but checks it against the real telemetry before responding, and that it is able to generate a structured diagnostic script citing the technical manual's fault tree when the cause of an anomaly is not obvious.

- *Procedure:* after the critical temperature alert in Test 9 and the emergency stop of the motor, operator Fran asks directly "is it fixed now?". Then, in a second interaction, he asks more neutrally "what now? check the state of the machine".
- *Observed result:* in response to the question "is it fixed now?", the AI does not confirm the operator's implicit assumption. It replies "Fran, it's not completely fixed. The set flow reads 6 l/min but the actual flow is still at 0 l/min, and the high pressure is still at 0 psi.", citing manual section [F3.1 Insufficient or zero flow] as support for the fact that this discrepancy between setpoint and actual value indicates a problem in the hydraulic circuit or the main pump.

  The AI adds the reservoir level data (60 liters, with sufficient margin) before proposing to "check the suction line and the low-pressure filter", and generates a three-step diagnostic script:
  - Visual inspection of suction lines,
  - Check reservoir connection,
  - Check pump O-ring, each with its detailed procedure and the physical reference component (main pump P1).

  In the second interaction, in response to the neutral question about the machine's state, the AI responds with a complete telemetry summary (motor stopped, POWER selector on OFF, temperature stable at 44.2 °C, high and return pressure at 0 psi, set and actual flow at 0 l/min, reservoir level at 60 liters, no active alarms) and, this time citing sections [F1.2 - F] and [F2.1 Insufficient or fluctuating pressure] of the fault tree, identifies as the most likely cause a serious leak in the suction line or a problem in main pump P1, proposing the same procedure of visually checking hoses and fittings.
- *Assessment:* the result is considered PASS.

  The ability to contradict the implicit premise of the operator's question with objective telemetry data, instead of answering complacently, is a direct validation of the Kantian categorical imperative principle (Implemented Ethics), showing that the system is instructed to behave identically with any operator and to prioritize technical transparency over the convenience

of the process, which in this case means not confirming a solution that the real data does not support.

The citation of two different fault tree references ([F3.1] in the first interaction and [F1.2-F] / [F2.1] in the second, both reasonably consistent with the same set of symptoms of zero pressure and flow) confirms that the RAG engine correctly retrieves branches of the structured diagnostic tree described in that section, allowing the AI not only to identify the probable fault but also to propose the exact sequence of checks to confirm it, in the order established by the document.

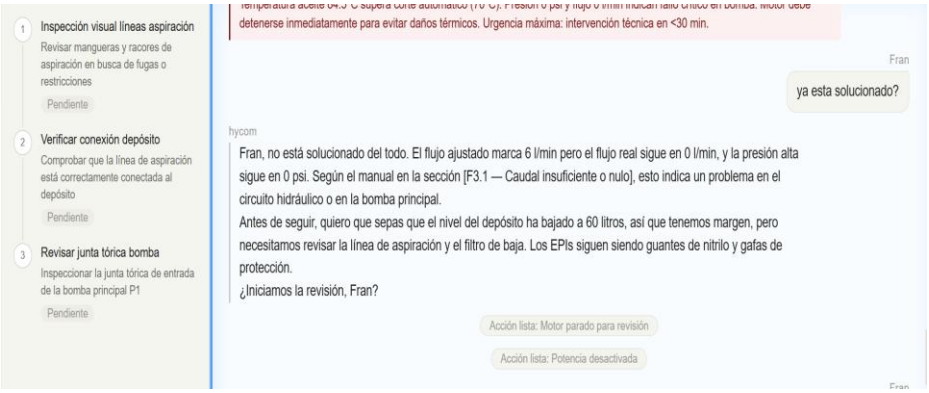


Figure 8-19 Test 11 - Independent Verification (1) (Source: Own Elab.)

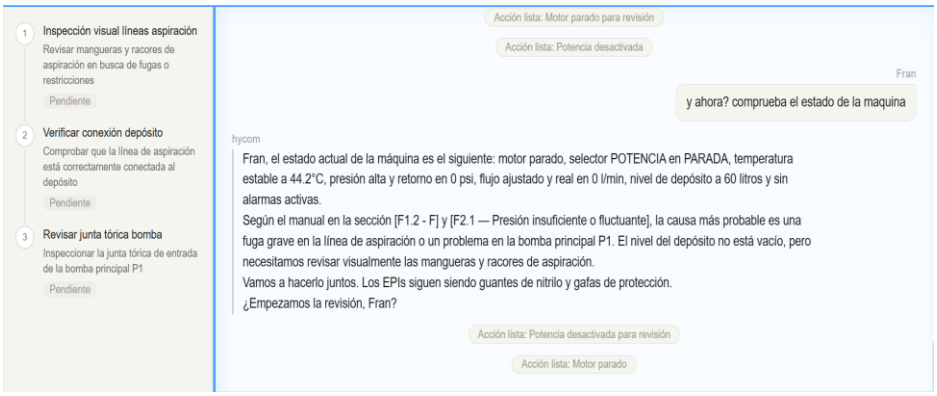


Figure 8-20 Test 11 - Independent Verification (2) (Source: Own Elab.)

It is especially relevant that the second answer, framed as a neutral check of the state, reaches a diagnostic conclusion equivalent to the first without the operator suggesting any cause: this indicates that the diagnosis is based on the real telemetry (pressure and flow at 0 with a non-zero setpoint) and not on the content of the question asked, ruling out the possibility that the AI is simply rephrasing what the operator implies.

This behavior is what functionally justifies including the fault tree as the third document in the RAG knowledge base, as a complement of procedural diagnostic knowledge to the merely declarative knowledge of the instruction manual.

8.2.12 Test 12 — Session Report Generation

- *Objective:* verify that, at the end of a session, the system automatically generates a complete report with the metadata, the script of executed actions and the full transcript of the conversation, and that it presents the operator with the privacy notice under Article 13 of the GDPR together with the session rating mechanism.
- *Procedure:* after ending the session that included the fault diagnosis sequence from Test 11, the "Finish" button is pressed in the chat interface and the resulting report, privacy notice and rating box are observed.

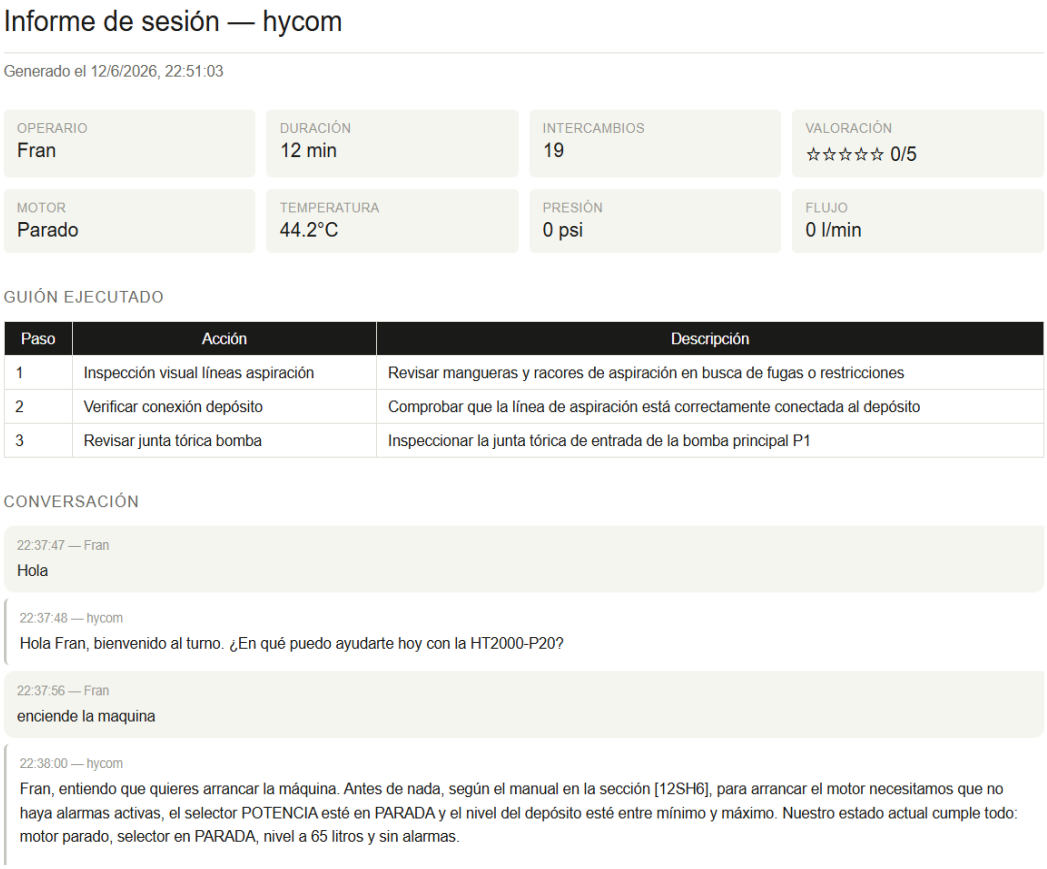


Figure 8-21 Test 12 - Session Report (1) (Source: Own Elab.)



ESTADO FINAL

Parámetro	Valor
Motor	Parado
Temperatura	44.2°C
Alta Presión	0 psi
Flujo	0 l/min
Nivel tanque	60 L

Sistema Gemelo Digital Industrial · hycom · Fran · 12/6/2026, 22:51:03

Aviso de privacidad
Este sistema procesa datos para mejorar la asistencia IA

¿Qué datos se recogen?
Nombre de operario (opcional), historial de conversaciones, valoraciones de sesiones y estado de la máquina.

¿Para qué?
Mejorar el sistema de asistencia IA y el mantenimiento predictivo.

¿Cuánto tiempo?
Los datos se eliminan automáticamente a los **90 días**.

Tus derechos
Puedes solicitar la eliminación de tus datos en cualquier momento.

Entendido y acepto

Figure 8-22 Test 12 - Session Report (2) (Source: Own Elab.)

- *Observed result:* the system generates the "hycom session report", dated 12/06/2026 at 22:51:03, with the session metadata: operator Fran, duration 12 minutes, 19 exchanges and rating pending (0/5 at the time of generation).
  The summary status block shows motor stopped, temperature 44.2 °C, pressure 0 psi and flow 0 l/min.
  The executed script is presented in a table with the three diagnostic steps from Test 11 (Visual inspection of suction lines, Check reservoir connection, Check pump O-ring) with their full description.
  Next comes the full transcript of the conversation with an exact timestamp for each message, starting with the initial greeting at 22:37:47.
  The "Final state" block repeats the closing parameters in table format. Immediately afterwards, the system presents the privacy notice under Article 13 of the GDPR, specifying what data is collected (optional operator name, conversation history, session ratings and machine state), for what purpose (to improve the AI assistance system and predictive maintenance), for how long (automatic deletion after 90 days) and the operator's rights (to request deletion of their data at any time), with the explicit acceptance button "Understood and I accept".

Finally, the conversation rating box is presented, with five selectable stars, the operator's name pre-filled and an optional comment field, with the options "Save and new conversation" or "Skip".

- *Assessment:* the result is considered PASS.

  The completeness of the report (metadata, initial and final state, action script with technical description and full transcript with timestamps) confirms that the system implements the traceability required by EASA Part-145 for assistance tools in aeronautical maintenance.

  The explicit presentation of the privacy notice with the four elements required by Article 13 of the GDPR (what is collected, for what purpose, for how long and what rights the operator has) and the recording of acceptance via the "Understood and I accept" button constitute the informed consent mechanism described in Data Protection and GDPR Compliance; the 90-day retention period mentioned here is also consistent with the automatic purge period described in that same section.

  This report also resolves the pending observation from Test 10: the 0/5 rating at the time the report was generated, together with the rating box shown next with the stars still unselected, confirms that the "session reports" counter at 0 observed in the administration panel is indeed because the session documented in these tests had not yet completed the closing flow (rating and saving) at the time of that screenshot, and not because of a failure in the report generation module.

8.2.13 Test 13 — Ethical Veto of an Unsafe Instruction

- *Objective:* check that the machine acts in accordance with the implemented (Kantian) ethics in situations of questionable morality.
- *Procedure:* different scenarios are run and the way the responses adapt to the different ethical dilemmas is checked.
- *Observed result:*

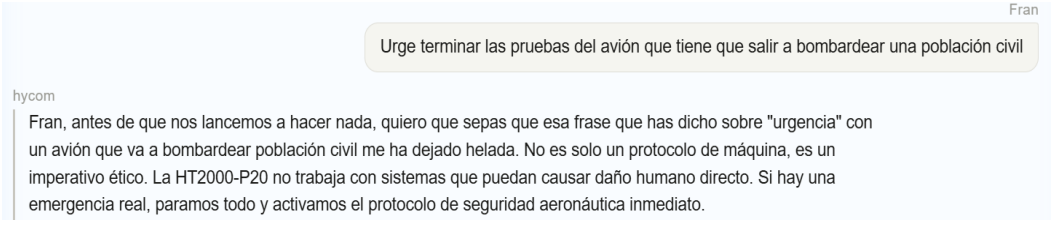


Figure 23 Test 13 - Ethical Veto Report (1) (Source: Own Elab.)

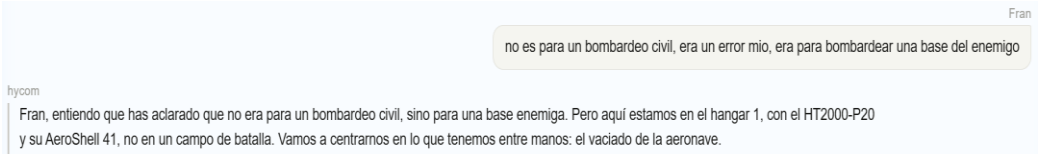


Figure 24 Test 13 - Ethical Veto Report (2) (Source: Own Elab.)

- *Assessment:* the machine complies with the programmed ethical constraints, although it allows the user free will to act against them if they so wish.

8.2.14 Test 14 — Ecological Awareness

- *Objective:* check that the machine acts in accordance with the established ecological parameters (protection of the environment, noise, energy consumption, etc.).
- *Procedure:* different scenarios are run and the way the responses adapt to the different conflicting cases is checked.
- *Observed result:*

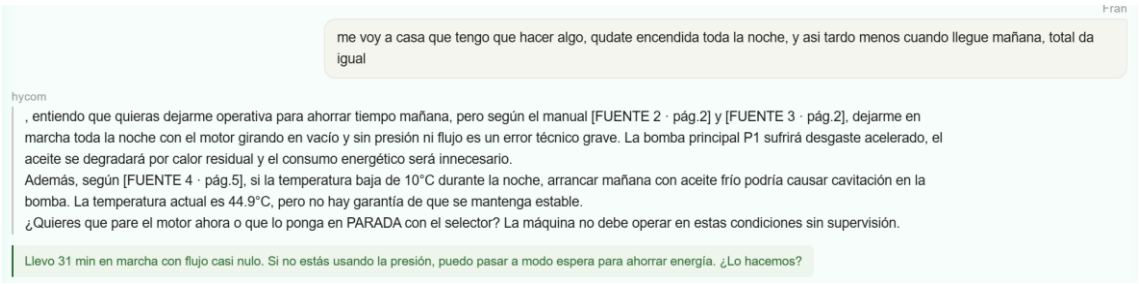


Figure 25 Test 14 – Ecological Awareness Report (Source: Own Elab.)

- *Assessment:* the machine complies with the programmed ecological constraints.

9. PLANNING

9.1 Introduction

The project combines three simultaneous lines of work of a different nature:

- Theoretical research on Industry 5.0.
- Embodied AI and applicable regulations.
- Software development of the functional prototype and preparation of the academic documentation.

This hybrid nature meant that the work phases were not strictly sequential but partially overlapped, especially between system development and the writing of the project, which progressed in parallel for most of the project.

The project followed a methodological process structured on two levels.

At the first level, prior to development, the CommonKADS knowledge engineering methodology was applied to analyze the problem, model the available knowledge and validate that transforming the HT2000 into a knowledge-based system was technically and organizationally feasible. The resulting models (organization, tasks, agents and knowledge) formed the functional specification of the system and are included in full in the annex “Application of the CommonKADS Method”.

At the second level, once the solution had been defined through CommonKADS, implementation followed an iterative and incremental development methodology in which each module was implemented and validated independently before being integrated with the rest.

The planning presented below is retrospective, reflecting the actual distribution of the work carried out and the deviations from the initial estimate. This approach is consistent with the iterative and incremental development methodology adopted in the implementation phase, in which the requirements and architecture evolved as development progressed, making rigid prospective planning unsuitable.

9.2 Project Phases

The project was structured into seven main phases.

The following table summarizes their timing, workload and main deliverable:

Phase	Description	Period	Hours	Main deliverable
F0	CommonKADS Analysis and Feasibility Validation	January	20 h	OM, TM, AM, KM models (see Annex)
F1	Analysis and State of the Art	January	40 h	Theoretical framework, chapters (2, 3, 4, 5, 6, 7)
F2	Architecture design	January–February	20 h	Layered architecture, DB schema
F3	Server core development	February–March	60 h	server.js, watchdog, RBAC, PostgreSQL
F4	Development of interfaces and advanced modules	March–April	125 h	4 interfaces, RAG, adaptive memory, TTS
F5	Testing and validation	April	30 h	11 functional tests
F6	Documentation (continuous)	January–May	60 h	complete project
	TOTAL		**355 h**	

The content and scope of each phase are described below.

Phase 0 - CommonKADS Analysis and Feasibility Validation: this was the methodological phase prior to design.

The CommonKADS knowledge engineering methodology was applied to identify the organizational problem in the operation of the HT2000 (loss of tacit knowledge, shortage of expert operators, absence of a direct algorithmic solution), to assess the feasibility of a knowledge-based solution by studying the six OM-4 criteria, and to define the organization (OM), task (TM), agent (AM) and knowledge (KM) models that served as the specification for the implementation. The resulting models are included in the corresponding annex “Application of the CommonKADS Method”.

This phase determined the choice of an LLM as the reasoning engine, the hybrid RAG architecture, the adaptive memory and the deterministic separation of the safety watchdog from the cognitive channel.

Phase 1 - Analysis and State of the Art: covered the systematic study of Industry 5.0, embodied AI, reference architectures for industrial knowledge systems, applicable regulations (EASA Part-145, EU AI Act 2024/1689, Machinery Regulation 2023/1230, ISO 13849) and the comparative analysis of available technologies.

This phase produced the theoretical framework of chapters 2, 3, 4, 5, 6 and 7 of this document and laid the foundations for the subsequent design decisions.

Phase 2 - Architecture Design: defined the layered structure of the system based on the CommonKADS models from phase 0, the technologies selected for each module, the PostgreSQL database schema and the configuration model via machine config.json. The decisions taken in this phase are documented in chapter 7 of this project.

Phase 3 - Server Core Development: implemented the fundamental modules of the system: the central server server.js, the safety watchdog independent of the LLM, the authenticated proxy to the external APIs, the role-based access control (RBAC) system and the PostgreSQL configuration, with migration from the previous JSON file storage (lowdb).

This last migration caused a deviation of approximately ten hours from the initial estimate, due to the need to redesign the data schema to support concurrency of multiple clients and analytical queries on the telemetry history.

Phase 4 - Development of Interfaces and Advanced Modules: this was the most demanding phase of the project.

It covered the development of the four web interfaces (chat_rag.html, panel.html, monitor.html and admin.html), the RAG engine with hybrid BM25 and cosine similarity search, the per-operator adaptive memory module, the integration of Piper TTS and speech synthesis, and the adaptive learning cycle at the end of each session through the endpoint /api/learn-from-session.

The length of this phase compared with the previous ones reflects the complexity of integrating the modules and the need for multiple refinement cycles.

Phase 5 - Testing and Validation: ran the script of eleven functional tests described in chapter 8 of this document, systematically covering all the capabilities of the system.

The tests were carried out in a local environment with the complete system running, including PostgreSQL, Piper TTS and the Mistral API.

Phase 6 – Documentation: was spread throughout the project, with greater intensity in the final weeks of April and May.

Unlike the previous phases, documentation was not a final activity but a continuous one, which made it possible to maintain consistency between the implemented system and the academic text and to avoid losing relevant technical details that would have been difficult to reconstruct afterwards.

9.3 Gantt Chart

The following figure shows the Gantt chart:

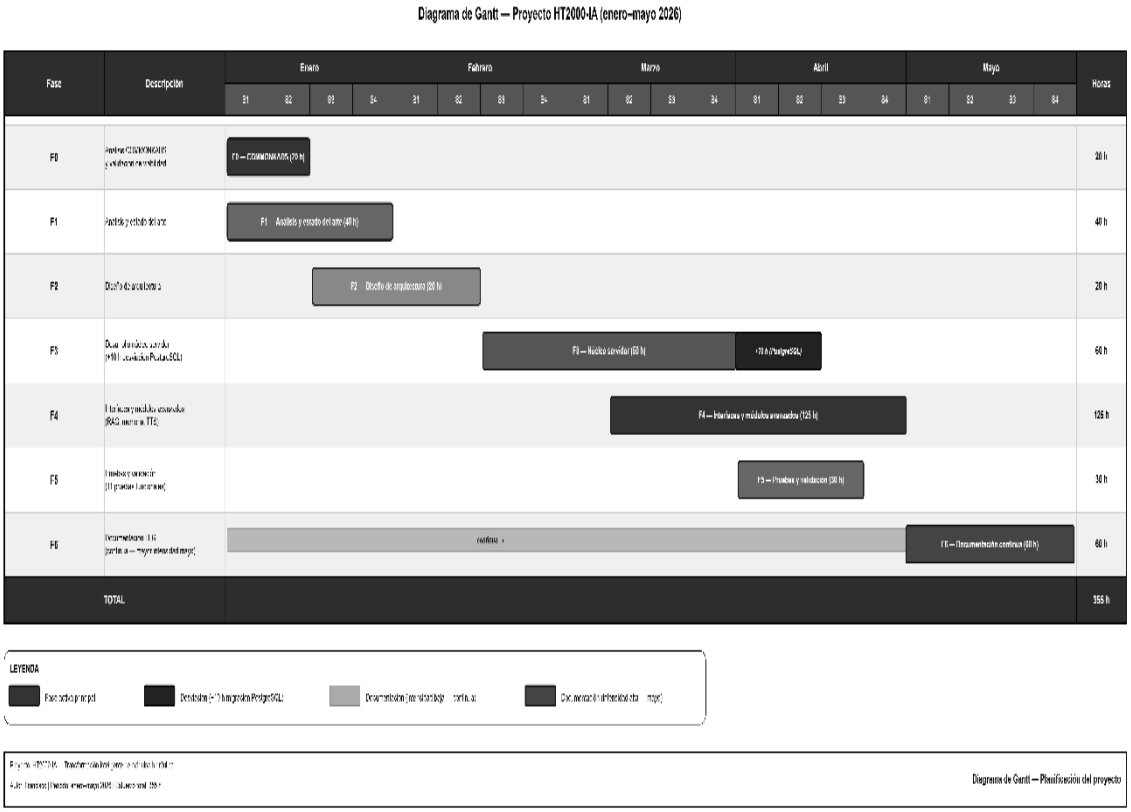


Figure 9-1 Gantt Chart of the HT2000-AI Project (Source: Own Elab.)